IMPERIAL

VERITAS

MEng Individual Project

Imperial College London

Department of Computing

Foundational Verification of Probabilistic Programs

Author:
Edwin Fernando

Supervisor:
Shing Hin Ho
Dr. Azalea Raad

Second Marker:
Prof. Nicolas Wu

June 12, 2026

Abstract

Probabilistic programming is a paradigm bringing the expressivity and clarity of abstractions of programming language theory to statistical models. These statistical models, and in general any code making use of randomness or sampling from probability distributions, are growing in use in today's software systems: cryptographic protocols, differential privacy, randomised algorithms, climate modelling, drug design, statistical physics simulation for engineering applications and probabilistic machine learning are some examples. Probabilistic programs evade conventional software testing: an outlier (correct) execution and a bug become indistinguishable. *Probabilistic Separation Logics* (PSLs) are a promising solution sitting at the intersection of Measure theory and Programming Language theory. PSLs enable foundational verification in this domain; mathematical proofs of correctness of a code implementation that assumes nothing but the axioms (foundations) of mathematics, all proven in a theorem prover. Lean is a theorem prover that has in recent years experienced unprecedented growth and excitement. It hosts the largest body of formalised mathematics, Mathlib, which has on several fronts reached the level of maturity to formalise research-level mathematics. Iris is a general framework for interactive and ergonomic proofs in a separation logic, which is actively being ported to Lean from the Rocq theorem prover. Standing on the shoulders of Mathlib and Iris, we mechanise the Lilac PSL and provide an interactive proof interface for conducting proofs in Lilac. This is the first mechanisation of a PSL with support for continuous distributions and the first mechanisation of a PSL in Lean.

Acknowledgements

Firstly I would like to thank my supervisor Shing Hin, for his endless enthusiasm, being the source of all knowledge on probabilistic stuff and his availability to discuss even unrelated topics. I must have spent half our meetings talking about topics only vaguely related to this project (but they were really interesting topics). I don't think it is possible to have found a better supervisor. I also thank Azalea for encouraging me throughout the project. I thank Bhavik, for teaching me Lean in the first place when I took his module, "Formalising Mathematics" last year. It is perhaps the best module I took in my degree, and certainly has ended up influencing my direction the most. He along with David Ledvinka also helped me in my struggle against Lean at the start of this project (including the time I ran into David in the lift and asked him about the Giry monad in Mathlib and got a nice explanation). I also thank people from the Lean Zulip, in answering my (possibly malformed) questions on Iris-lean. Markus de Medeiros and Michael Sammler provided helpful responses though it might have been apparent that I was somewhat clueless about Iris.

Contents

Chapter 1

Introduction

There has been an exciting amount of progress recently in the area of *probabilistic separation logics* (PSL). Separation logic, first introduced by O'Hearn et al. [2001], has since been tremendously successful in formally proving the correctness of heap manipulating programs [Jacobs et al., 2011; Calcagno et al., 2011; Le et al., 2022] and concurrent programs [Brookes and O'Hearn, 2016]. Iris [Jung et al., 2018] introduced a language-generic framework of separation logics for reasoning about programming languages with concurrency and higher order state, which spanned a significant class of mainstream programming languages, notably the soundness of a significant subset of the Rust programming language [Jung et al., 2017]. The popularity of Iris is due in part to the Iris Proof Mode (IPM) [Krebbers et al., 2017], an ergonomic interface embedded in a host theorem prover, for proofs in separation logic, enabling the scaling of mechanisation of systems in a separation logic. The IPM was generalised away from Iris, and modularised to provide a frontend for any separation logic in [Krebbers et al., 2018]. The application of separation logics for reasoning about *probabilistic programs* was first explored in [Barthe et al., 2019]. At a high level a probabilistic program offers sampling from a probability distribution as a primitive of the language. A persuasive notion of separation: "separation is independence of random variables" was proposed by Lilac [Li et al., 2023]. Briefly, propositions joined with the separating conjunct may not talk about inter-dependent random variables. This notion of separation has been adopted by most subsequent probabilistic separation logics.

This project provides a complete mechanisation of Core Lilac, in Lean, along with the proof mode required to carry out interactive proofs in it. We know of two similar roughly simultaneous projects, which, along with this project, are among the first mechanised probabilistic separation logics: Bluebell [Bao et al., 2025] and Amaryllis [Lohse et al., 2026].

1.1 Motivation

Here we give an informal account of what probabilistic programs are and why we would care to use them. A Probabilistic programming language is not an established or general paradigm of programming such as Functional or Imperative programming. Most generally probabilistic programs are simply programs which have a `sample` construct for sampling from probability distributions, and optionally, an `observe` for specifying a new distribution given an old distribution and some newly observed data.

Statisticians use languages such as Stan [Carpenter et al., 2017], Hakaru [Narayanan et al., 2016] and Anglican [Wood et al., 2014], to build complex statistical models succinctly using the expressivity of a programming language. Note that here, a model is simply a probability distribution over some quantity of interest, take for instance the probability of a 3d structure being correct for the given protein sequence, (other examples may come from weather forecasting, medical diagnosis, economics, statistical mechanics etc). "Running" this code corresponds to running a simulation of the model, where we can get answers such as the mean or variance, or just samples (sample protein structure to test in a lab in the example).

What distinguishes a probabilistic programming language is that it decouples the specification of

a statistical model from the algorithm for its simulation (henceforth called probabilistic inference). Inference algorithms are often highly optimised and specialised to the specific model and the interpreter of a PPL attempts to make the best choice/combination of inference algorithms for the given code (model), relieving the user of this additional work [van de Meent et al., 2021].

So in other words PPLs are a layer of abstraction which sits between a statistical model and the inference algorithms which simulate it. This abstraction unlocks the verification of correctness properties of the statistical model, which led to the development of program logics for reasoning about PPLs. This will be the main topic of this report.

1.1.1 Discrete vs continuous distributions

First let us define what discrete and general (continuous) PPLs are. A discrete PPL does not allow specifying a continuous probability distribution. This is achieved most straightforwardly by simply not including any "continuous" types, types that can support a continuous distribution such as $\mathbb{R}$, in the syntax of a PPL. A general PPL will have a type corresponding to $\mathbb{R}$, and not restrict the distributions one can construct over it. This seemingly small detail has significant consequences on the semantics of a PPL which in turn dictates the complexity of program logics for such a language. This dichotomy largely exists due to difficulties with semantics of continuous PPLs. In plain words, the existing approach for semantics of probabilistic languages did not allow for functions to be first class citizens[1]. An alternative semantics for such PPLs with higher order functions was given by [Staton, 2017] within the decade. In general the difficulties of verification of continuous PPLs is in the nuances of using measure theoretic, rather than standard set theoretic, machinery.

However, all PPLs listed above are continuous. Indeed, PPLs are only relevant when working with continuous distributions, as they remove the difficulty of working with the probabilistic inference algorithms. The inference problems that are interesting rely heavily on continuous distributions, so discrete PPLs would not be of much use. So why are program logics for discrete PPLs such as [Bao et al., 2025] and [Lohse et al., 2026] important?

1.1.2 Probabilistic programs in the wild

Probabilistic programs stretch far beyond the reaches of domain specific languages like Stan. They are in fact pervasive in modern software systems: cryptographic protocols, differential privacy, distributed systems and randomised algorithms. These were all examples where it would suffice to only support discrete probability distribution. Conventional techniques like software testing are much less applicable here; applying it naively would result in an outlier (correct) execution and a bug becoming indistinguishable. The above applications of probabilistic reasoning all require support only for discrete distributions, and generally fall firmly within traditional areas of computer science. This justifies the many logics which support only discrete distributions, including both Bluebell and Amaryllis. Examples requiring continuous distributions include Scientific computing (climate modelling, physics simulations for engineering applications) [Stuart, 2010], Bayesian data analysis and probabilistic machine learning [Murphy, 2023]. In each of these applications, the probabilistic component is centred around probabilistic inference [Ghahramani, 2015].

So domain specific PPLs like Stan, form a small subset of what program logics for probabilistic programs can be applied to. The most general feature of a probabilistic programming language, the ability to sample from a distribution can be found in the standard libraries of every major programming language, such as Python, C++ or Java.

1.1.3 Comparison with contemporary mechanisations

Each of the 3 PSL mechanisation projects (that we are aware of) of Bluebell, Amaryllis and Lilac pushes the boundaries in distinct directions. Mechanisation of PSLs is a non-trivial investment, since it also involves mechanisation of some bespoke prerequisite components.

Bluebell [Bao et al., 2025] is a PSL, novel for combining both unary and relational styles of reasoning under a single logic. Unary logics like the original PSL [Barthe et al., 2019] and Lilac

[1] In precise terminology the category of measurable spaces, in which the denotational semantics of probabilistic programs are interpreted, is not cartesian closed [Aumann, 1961]

are highly expressive for reasoning about probabilistic programs. In some situations we are only interested in proving a property regarding how two probabilistic programs relate. The outcomes of probabilistic programs follow some probability distribution. Imagine we have two probabilistic programs, and we are given a partial order over probability distributions, known as *stochastic dominance*. A relational logic can help us establish that one program *stochastically dominates* another. In order to achieve such a result using a unary logic, we would need to specify the output distribution of each program in sufficient detail using the unary logic first, and as a next step prove that they are related by the stochastic dominance relation. This may be impossible if the programs are more complex than the logic can express, and otherwise unrealistic, as we may have convoluted proof obligations for each of the programs, which is entirely unnecessary under the relational paradigm. One can imagine how relational logics may be useful in proving correctness of cryptographic protocols, where the proof of correctness often takes the form of a game between defender and adversary (both modelled as probabilistic programs). It turns out that a family of modern cryptographic protocols, known as *Succinct Non-interactive ARGuments* (SNARG) [Chiesa and Yogev, 2024], which is a proof system with probabilistic guarantees, requires a program logic with support for both unary and relational to prove its soundness [Tuma et al., 2026]. SNARGs form an active area of research and have been developed almost entirely within the decade, due to their promise of solving the computational inefficiency plaguing blockchains[2]. The ongoing mechanisation of Bluebell [Verified-zkEVM, 2025] falls under an umbrella project funded by Ethereum Foundation. In conclusion its formalisation is motivated by applications to securing the foundations of future blockchain systems. At the point of writing the ongoing bluebell formalisation has not begun integration with the Iris Proof Mode.

Amaryllis [Lohse et al., 2026] lays the groundwork for reasoning about probabilistic programs which are also capable of other effects such as heap-based memory and concurrent execution. Specifically Amaryllis is capable of reasoning about probabilistic independence and conditioning, and dynamically allocated memory within one logic. Unlike the other two formalisations, Amaryllis is formalised in Rocq. Also, unique to Amaryllis is that it not only instantiates the Iris Proof Mode (MoSeL [Krebbers et al., 2018]), but partially instantiates the Iris logic, which is a general logic for reasoning about programming languages with concurrency and higher order state[3]. Amaryllis pushes the direction of reasoning about probabilistic programs written in mainstream programming languages (as discussed in Sec 1.1.2).

Lilac [Li et al., 2023] is the only PSL among the 3 which supports continuous distributions. It is implemented in Lean, due to Mathlib (Lean's mathematical library) having the richest and fastest growing support for measure and probability theory, since it strives to support formalisation of modern research-level mathematics in Lean. An important consequence of this is that results in Mathlib are generally stated at the broadest levels of generality, such as encompassing both discrete and continuous distributions[4]. The mechanisation of Core Lilac makes the first step towards verifying the diverse set of systems which work in the continuous domain (as seen in Sec 1.1.1). Bayesian Separation Logic (BaSL) [Ho et al., 2025], extends Lilac with bayesian reasoning, which is central (to every continuous application in Sec 1.1.1), and also introducing the need for probabilistic inference (as explained in 1.1). We also claim that logics in the vein of Lilac and BaSL, are capable of tackling verification of algorithms at the heart of the enterprise: probabilistic inference. In Chapter 6, we present a novel idea for how BaSL, extended with some components which are yet to be invented, can be used to verify the correctness of a certain class of probabilistic inference algorithms, called MCMC (Markov Chain Monte Carlo). Probabilistic inference is a well-established field of research, and often the innovation of a research paper may be the usage of a novel inference algorithm. It is thus interesting to have an expressive logic for verification of various inference algorithms. The verification of inference algorithms, which may also be referred to as the interpreter/compiler of probabilistic programs, has been carried out previously [Tassarotti and Tristan, 2023; Ścibior et al., 2017]. Both previous approaches used the operational or denotational semantics directly and did not reason in a PSL. We hypothesise that verification in a PSL with separation as independence, support for conditioning and bayesian reasoning, may significantly reduce the complexity of proof obligations. If true, we envision a Lean library of verified inference

[2] As outlined here https://ethereum.org/roadmap/zkevm/. Note that SNARGs are well known by a subclass called *Zero Knowledge Proofs*

[3] This would indeed not be feasible in Iris-lean [leanprover-community, 2026] as it is just about to reach the stage of proving properties in HeapLang, the prototypical full Iris instantiation

[4] An example is given in Chapter 6

algorithms, providing confidence in algorithms belonging to an area where the only assurance of correctness emerges from an implementation being well-established or "battle-tested".

1.2 Contributions

- We mechanise the soundness of Core Lilac proof system (Lilac without conditioning) available on github. This is the first mechanisation of a probabilistic separation logic which supports continuous distributions.
- Chapter 3 overcomes difficulties in embedding Lilac into Iris, by modifying Lilac's definition of variables. We thoroughly motivate the problem by showing how a correct specification on the simplest of programs (`unif1`) cannot be proven.
- Chapter 4 presents the intuition behind some of the most difficult soundness proofs, of proof rules in Lilac.
- Chapter 5 verifies properties of fundamental probabilistic programs in Lilac using the Iris proof interface. We return to the verification of `unif1` from Chapter 3, and show that this is now easily achieved. Next we verify a specification on another program (`halve`) which involves stating and proving some new proof rules (about expectations) and custom tactics for Lilac-specific connectives through metaprogramming.
- Chapter 6 breaks away from the Lean mechanisation and reflects on what we think is a promising application of continuous PSLs. It introduces probabilistic inference (the algorithm at the core of the interpreter/compiler of a PPL), and poses rigorously, 2 research problems that stand in the way of verification of probabilistic inference algorithms using separation logic.

Chapter 2

Background

Probabilistic programming is a paradigm where we define statistical models using code. It is a means of describing complex distributions over some space in a structured way. Probabilistic programming languages (PPLs) also come with an interpreter (probabilistic inference engine) which allows the programmer to sample a term from the distribution denoted by a program and indeed sample distributions while constructing new distributions. More interestingly, Bayesian probabilistic programming languages (Bppl) also allow one to "observe" some data and update one's belief of the world (update the probability distribution one associates with some occurrence). Lilac does not support Bayesian updates, so we will not be seeing much of this important feature of probabilistic programming.

In order to understand the semantics of probabilistic programming we first need a crash course on measure theory, and then explore a compositional semantics for PPLs. We turn our attention to the theory of separation logics abstracting a prototypical instance, till we reach the core ideas, on top of which Lilac will be built in Chapter 3. Finally, we seek a fundamental understanding of our premise of foundational interactive verification: the foundations in dependent type theory, and the meta-programming which makes it "interactive".

2.0.1 Monads

The reader is assumed to have been exposed to a monadic style of programming (for example Haskell). Mathematical/categorical perspective is not strictly necessary, and is thus omitted.

Here we give a brief refresher on the computational perspective of monads. It is relevant to understand the semantics of probabilistic programming (Giry Monad) and Lean's metaprogramming library (MetaM monad).

Nevertheless the categorical/mathematical perspective on monads, though not strictly necessary to understand this work, is a valuable alternative perspective. For example the justification for the semantics of higher order probabilistic programs being an open problem till 2017 is given naturally in categorical language (the category of measurable spaces in which types are interpreted is not Cartesian closed).

From the computational perspective, a monad can be understood as a map from type A to M A. It is additionally associated with two functions Ret and bind, with the following signatures. Monads are actually monoids with Ret acting as unit and bind acting as the binary operation. The laws that Ret and bind obey are as follows which should remind one of the monoid laws.

2.0.2 Type Theory

Readers are also assumed to have some familiarity with type theory, in particular having seen inference rules.

2.1 Measure Theory

In order to understand the semantics of probabilistic programming, a formal definition for probability is needed. Measure theory underpins probability theory and an overview is given here. We want to be able to talk about the probabilities of some event occurring, which is modelled by a `random variable`. In order to define a random variable we first need to define measures and measurable spaces. A measurable space is a pair (Ω, Σ_Ω), where Ω is of type `Set`, and $\Sigma_\Omega \subseteq \mathcal{P}(\Omega)$ is called the σ-algebra. We interpret each element of Σ_Ω to be an event (such as a coin flip landing heads). We would like the σ-algebra to be closed under complements and countable unions[1]. From the presentation so far, it may seem like all singleton sets must be events but due to technical reasons ([Axler, 2020, Chapter 2]), this cannot be in certain cases where Σ is infinite.

Formally, a σ-algebra $\Sigma_\Omega \subseteq \mathcal{P}(\Omega)$ is a collection of subsets of Ω satisfying:

$$\begin{aligned} &\Omega \in \Sigma_\Omega, \\ &E \in \Sigma_\Omega \implies \Omega \setminus E \in \Sigma_\Omega, \\ &(E_i)_{i\in\mathbb{N}} \in \Sigma_\Omega \implies \bigcup_{i\in\mathbb{N}} E_i \in \Sigma_\Omega. \end{aligned}$$

A measure on a measurable space is a function $\mu : \Sigma_\Omega \to [0, \infty]$.

A probability measure on the other hand is more restrictive: $\mu : \Sigma_\Omega \to [0, 1]$. It gives the probability of an event $E \in \Sigma\_\Omega$ occurring. A similar but more general notion than what a measure gives us, is a random variable: a measurable function between two measurable spaces. We say "more general notion", since using the identity measurable function yields the measure we started with. A measurable function denoted $f : (\Omega_1, \Sigma_\Omega) \xrightarrow{\text{m}} (\Omega_2, \mathcal{G}) \; f : \Omega_1 \xrightarrow{\text{m}} \Omega_2$ is well-defined when there is an ambient measurable space structure $(\Omega_1, \Sigma_\Omega)$ and $(\Omega_2, \mathcal{G})$. f allows us to talk about the distribution of a particular attribute in the event space. For example $\Omega_1 = \{1, \ldots, 6\}^2$ may be the set of all events associated with playing 2 dice, while f may map to the sum of the two dice, $\Omega_2 = \{1, \ldots, 12\}$. A measurable function f, transforms any measure μ on $(\Omega_1, \Sigma_\Omega)$ to the pushforward measure μ' on $(\Omega_2, \mathcal{G})$. In order for this to be well-defined we require the following measurability condition to be satisfied by all measurable functions: $\forall G \in \mathcal{G}, f^{-1}(G) \in \Sigma_\Omega$. We denote the pushforward as $\mu' = f_{\#}\mu$.

A related notion to the pushforward measure, is the pullback σ-algebra. Instead of constraining the measurable function to behave according to the σ-algebra on the domain, the pullback asks what σ-algebra do we require on the domain to support some measurable function. Given a function $f : \Omega \to \Omega'$ and a σ-algebra $\mathcal{G}$ on the codomain Ω', the pullback σ-algebra on Ω is

$$f^*\mathcal{G} := \{\, f^{-1}(G) \mid G \in \mathcal{G} \,\}.$$

This is the coarsest (smallest) σ-algebra on Ω making f measurable, and we are guaranteed it is closed under complements and countable unions since $\mathcal{G}$ is a *sigma*-algebra.

A Lebesgue integral is a generalisation of the standard (Riemann) integral. Intuitively the Lebesgue and Riemann integral will coincide on the spaces such as $\mathbb{R}$, where the domain of integration is continuous. The Lebesgue integral relaxes that requirement. The Lebesgue integral of a measurable function $f : \Omega \to [0, \infty]$ with respect to a measure $\mu : \Sigma_\Omega \to [0, \infty]$ is a number defined by:

$$\int_\Omega f \, d\mu := \sup \left\{ \sum_{i=1}^{n} f(\omega_i)\, \mu(U_i) \;\middle|\; \{U_i \in \Sigma_\Omega\}_{i=1}^n \text{ disjoint cover of } \Omega \right\}.$$

To get an intuition remember that integrals are supposed to add the values of a function by breaking the domain up into infinitesimal pieces. Observe that the integral is a weighted sum of $f(\omega)$, weighted by the measure. The integral is iterating over the entire space by picking some choice of measurable disjoint sets whose union is Ω (disjoint cover of Ω). And for each measurable set (U_i) it picks a representative point ω, evaluates f on ω and weights it by how large the set is (μU_i). The supremum and infimum together enforce that the chosen disjoint cover is chosen as finely (small measure) as possible.

[1] from unions and complements we obtain intersections through de Morgan's law

2.1.1 Borel spaces and Lebesgue measures

Intuitively a measure is supposed to be a length, volume or in general a "size" in a higher dimensional counterpart. The standard borel σ-algebra on $\mathbb{R}$, $B_{\mathbb{R}}$, corresponds to our notion of those subsets of $\mathbb{R}$ which have a "size". Again we refer to Axler [2020, Chapter 2], for examples of subsets of $\mathbb{R}$ which are not measurable sets, which are rather obscure. In general most subsets that we care about: open and closed interval, singletons and of course unions and complements of the previous, are all included in $B_{\mathbb{R}}$. We define the σ-algebra, *generated* out of some $S : Set(SetA)$, $\sigma(S)$ as the smallest set containing A and closed under complements and countable unions. We can see what $B_{\mathbb{R}}$ includes by seeing various ways of defining it:

We have that the Borel σ-algebra $B_{\mathbb{R}} = \sigma(S)$, when S is:

(a) $\{(a, b) : a, b \in \mathbb{R}\}$,

(b) $\{(-\infty, a) : a \in \mathbb{R}\}$,

(c) $\{(-\infty, a] : a \in \mathbb{R}\}$,

(d) $\{(a, \infty) : a \in \mathbb{R}\}$.

More generally a Borel σ-algbera, (not necessarily on $\mathbb{R}$), is a σ algebra generated from the topology and the standard Borel σ-algebra is canonical since it is generated from the standard topology [Axler, 2020]. We are only concerned with Borel σ-algebras for $\mathbb{R}^n$ (i.e. Euclidean space) in this project.

Returning to the initial claim that a measure is intuitively the size of a set, we present the Lebesgue measure on $\mathbb{R}, B_{\mathbb{R}}$, λ, which corresponds to the sums of lengths of all intervals in a measurable subset. Precisely:

$$\lambda(E) := \inf \left\{ \sum_{i=1}^{\infty} (b_i - a_i) \;\middle|\; E \subseteq \bigcup_{i=1}^{\infty} (a_i, b_i) \right\}.$$

That is, the Lebesgue measure of a set E is the infimum, over all countable covers of E by open intervals, of the total length of those intervals. On an interval it recovers the expected value, $\lambda\big((a, b)\big) = b - a$, and it assigns measure zero to every singleton (and hence to every countable set, such as $\mathbb{Q}$).

2.1.2 Product of measurable spaces $\times_m$ and product measures $\otimes$

In the previous section we mentioned that we are interested in measurable spaces (and measures) over $\mathbb{R}^n$ for some n. Let's see how we can build the standard Borel σ-algebra and Lebesgue measure over $\mathbb{R}^n$ by taking products.

Given two measurable spaces (Ω_1, Σ_1) and (Ω_2, Σ_2), their product is the measurable space on the Cartesian product $\Omega_1 \times \Omega_2$, whose σ-algebra is generated by the *measurable rectangles*:

$$\Sigma_1 \otimes \Sigma_2 := \sigma\big(\{\, A \times B \mid A \in \Sigma_1,\ B \in \Sigma_2 \,\}\big).$$

Note we must take the generated σ-algebra: the rectangles themselves are not closed under complement or countable union. We write the resulting measurable space as $(\Omega_1, \Sigma_1) \times_m (\Omega_2, \Sigma_2)$.

Given σ-finite measures μ_1 on Σ_1 and μ_2 on Σ_2, there is a unique *product measure* $\mu_1 \otimes \mu_2$ on $\Sigma_1 \otimes \Sigma_2$ determined on rectangles by

$$(\mu_1 \otimes \mu_2)(A \times B) = \mu_1(A)\, \mu_2(B),$$

so the measure of a rectangle is the product of the side measures (for uniqueness see Axler [2020, Chapter 5A]). Specialising to $\mathbb{R}$, the n-fold product of $(\mathbb{R}, B_{\mathbb{R}})$ with itself gives the standard Borel σ-algebra $B_{\mathbb{R}^n} = B_{\mathbb{R}}^{\otimes n}$, and the n-fold product of the Lebesgue measure gives the n-dimensional Lebesgue measure, $\lambda_n = \lambda^{\otimes n}$, assigning to a *measurable rectangle* $\prod_{i=1}^{n}(a_i, b_i)$ its size $\prod_{i=1}^{n}(b_i - a_i)$ as one would expect.

2.1.3 Probability Theory

Probability theory emerges from measure theory, and is centred around the probability space which is a tuple $(\Omega, \Sigma_\Omega, \mu)$, of a set equipped with a measurable space, and a probability measure. Random variables play a key role as well so we provide a little more intuition on their relation to the measurable space in their (co)domain. To bolster the intuition that the measurability condition is less restrictive over a finer measurable space in the domain, imagine a coarse measurable space in the domain. The pre-image of the function is less likely to be a measurable set in the domain, and so less likely to satisfy the measurability condition. Hence coarser measurable spaces in the domain are more restrictive. Take the premise that we want to be as permissive as possible in the random variables we are allowed to define. Then we want their domain to be as general as possible, so we take a measurable space which is very fine grained; in other words it is very large as it contains a large number of sets. In practice this is often achieved by equipping the domain with the standard Borel measurable space, though this is not necessarily the finest (greatest) measurable space available (indeed, a greatest measurable space does not exist for general Ω).

Two random variables $X : \Omega \to \Omega_1$ and $Y : \Omega \to \Omega_2$ on a probability space $(\Omega, \Sigma_\Omega, \mu)$ are *independent* when knowing the value of one tells us nothing about the other, that is, their joint distribution factors into the product of their distributions:

$$\mu\big(X^{-1}(A) \cap Y^{-1}(B)\big) = \mu\big(X^{-1}(A)\big)\,\mu\big(Y^{-1}(B)\big) \qquad \forall A \in \Sigma_1,\ B \in \Sigma_2.$$

Finally, the *expectation* of a real-valued random variable $X : \Omega \to \mathbb{R}$ is its average value weighted by the probability measure, given by the Lebesgue integral

$$\mathbb{E}[X] := \int_\Omega X \, d\mu.$$

2.2 Semantics of Probabilistic Programs

We first present Appl [Li et al., 2023, appendix A], a probabilistic programming language without Bayesian updates. Syntactically it looks like a basic WHILE language. In Appl, there isn't an explicit `sample` construct. Sampling is realised in the semantics of sequencing as explained later. Though the language is first order, PPLs still have a non-trivial, novel denotational semantics. The central difference is that:

1. Types are interpreted in `Meas`, the category of measurable spaces rather than just `Set`. Concretely each type is interpreted as a measurable space (A, Σ_A).
2. Probabilistic programs have a monadic semantics, interpreted in the Giry monad, which expresses probabilistic computation as an effect. The Giry monad is a monad in `Meas`.

The interesting aspects of Appl, follow from these two points which we will explain thoroughly.

A note on notation: we deliberately overload a single letter, writing A both for a syntactic type of Appl and for its denotation $[\![A]\!]$, the measurable space it is interpreted as. Which is meant is always clear from context, since the syntactic type constructor is written $\mathbf{G}$ while its semantic counterpart, the Giry functor, is written $\mathcal{G}$.

In Fig 2.1a we see the types of Appl which as noted must have as denotation a measurable space rather than a set, as given in Fig 2.1b. We recognize many familiar σ-algebras in the denotation such as the Borel σ algebra on the reals and the product measurable space (see Sec 2.1). Note that in discrete spaces such as $[\![\text{bool}]\!]$, the power set is a valid measurable space, and clearly the greatest one (ordered by subset inclusion). In our PPL we would like to express measures and manipulate measures. Understanding the type constructor $\mathbf{G}$, is key to answering this question.

The type constructor $\mathbf{G}A$ is interpreted as the type of probability measures ProbMeasure$[\![A]\!]$, the ~~`Set`~~ `MeasurableSpace` of measures over $[\![A]\!]$, denoted $[\![\mathbf{G}A]\!] = \mathcal{G}\,[\![A]\!]$ So in order for $\mathbf{G}A$ to be a valid Appl type, there must exist some canonical `MeasurableSpace` $\Sigma_{\mathcal{G}\,[\![A]\!]} \subseteq \mathcal{P}(\Sigma_A \to [0,1])$. This is indeed true [Giry, 1982][2] established that $\mathbf{G} = \mathcal{G}$ is a functor, the Giry functor.

[2] An excellent blogpost introducing the Giry monad can be found here https://jtobin.io/giry-monad-foundations

$$A, B ::= A \times B \mid \text{bool} \mid \text{real} \mid \mathbf{G}\, A \mid 1$$

(a) Syntax

$$\begin{aligned}
[\![A \times B]\!] &= [\![A]\!] \otimes [\![B]\!] \\
[\![\text{bool}]\!] &= (\{T, F\}, \mathcal{P}(\{T, F\})) \\
[\![\text{real}]\!] &= (\mathbb{R}, \mathcal{B}_{\mathbb{R}}) \\
[\![\mathbf{G}A]\!] &= \mathcal{G}\,[\![A]\!] \\
[\![1]\!] &= \text{the one-point space}
\end{aligned}$$

(b) Denotational semantics

Figure 2.1: Types in Appl

A natural next question is how these measures can be created and manipulated. To answer that we first introduce the syntax and denotational semantics of terms in Appl, in Fig 2.2. Our semantics is compositional, meaning we allow for the denotation of *open* terms, terms which have free variables. To allow this, Appl uses the standard technique of introducing a variable context. This is shown in Fig 2.2b, where a novelty of lilac is that $[\![M]\!]$ is not just a function from the context, but a measurable function from the context. We are starting to see the probabilistic nature of Appl. The denotations of open terms of Appl type A are *random variables* of type $[\![A]\!]$.

Let us present the more straightforward terms in Appl. In Fig 2.2c, it is apparent that $\rho : [\![\Delta]\!]$, so with the variable construct we are able to choose a variable from the context, denoted $\rho(X)$. As in a standard language, we are able to create tuples, take projections on them, perform standard binary operations, and denote true and false. `flip` and `unif` are specific to a PPL, both allowing us to construct terms of type $\mathbf{G}\, A$. `flip p : G bool` and `unif[0,1] : G real`. Plainly, `unif[0,1]` denotes the uniform distribution on the interval $[0, 1]$, while `flip p` denotes the Bernoulli distribution, which is just the distribution of a (possibly biased) coin flip.

We have so far seen constructs which are standard in any programming language, (noting that their interpretation does have some added measurability structure), and we have also seen how to construct primitive measures using `unif[0,1]` and `flip p`. But expressivity of Appl for manipulating measures stems from its monadic term constructors `ret` and `bind`. We pursue a layered presentation in understanding the semantics of these constructs, first presenting the semantic `ret` and `bind` operations of the Giry monad, where we do not have to worry about variable contexts. Then the actual denotation of the two constructs. We will revisit it in Sec 3.1, where we mechanise it in Lean, in point free form to make it amenable to equational reasoning (used plentifully in the proofs to come).

2.2.1 The Giry monad

Our intuition is that syntactic `ret` and let-binding should have something to do with the Giry monad's operations, denoted as:

$$\text{ret}_{\mathcal{G}} : A \to \mathcal{G}\, A \qquad\qquad \gg\!= : \mathcal{G}\, A \to (A \to \mathcal{G}\, B) \to \mathcal{G}\, B$$

Here we are assuming A and B have the form $A = [\![A]\!]$ and $B = [\![B]\!]$ Let us see how we can construct these operations and validate the monad laws. We define $\text{ret}_{\mathcal{G}}\ x = \text{dirac}\, x$. The Dirac measure concentrates all its probability at a single point. Given a measurable set E, $\text{dirac} x E =$ if $x \in E$ then 1 else 0.

Now the type $(A \to \mathcal{G}\, A)$ is defined as Kernel $A\, B$. Mathlib uses this type as the primitive on which most of its probability theory library is built. We give a diagrammatic exploration of probabilistic programs to see why kernels are so compelling:

$$
\begin{aligned}
p &\in [0,1] \\
r &\in \mathbb{R} \\
n &\in \mathbb{N} \\
\oplus &\in \text{Arith} := \{+, -, \times, /, \hat{}\} \\
< &\in \text{Cmp} := \{<, \leq, =\} \\
M, N, O ::= {} & X \mid \text{ret } M \mid X \leftarrow M;\ N \mid \\
& (M, N) \mid \text{fst } M \mid \text{snd } M \mid \\
& \text{T} \mid \text{F} \mid \text{if } M \text{ then } N \text{ else } O \mid \text{flip } p \mid \\
& r \mid M \oplus N \mid M < N \mid \text{unif } [0,1] \mid \\
& \text{for}(n, M_{\text{init}}, i\ X.\, M_{\text{step}})
\end{aligned}
$$

(a) Syntax

$$
\begin{aligned}
\Delta ::= {} & \cdot \mid \Delta, X : A \\
[\![\Delta]\!] = {} & A_1 \times_m \cdots \times_m A_n \\
[\![M : A]\!] : {} & [\![\Delta]\!] \xrightarrow{\text{m}} [\![A]\!]
\end{aligned}
$$

(b) The type of denotation of terms in a variable context

$$
\begin{aligned}
[\![X]\!]\rho &= \rho(X) \\
[\![\text{ret } M]\!]\rho &= \text{ret } [\![M]\!]\rho \\
[\![X \leftarrow M;\ N]\!]\rho &= v \leftarrow [\![M]\!]\rho;\ [\![N]\!][X \mapsto v] \\
[\![(M, N)]\!]\rho &= ([\![M]\!]\rho,\ [\![N]\!]\rho) \\
[\![\text{fst } M]\!]\rho &= \pi_1([\![M]\!]\rho) \\
[\![\text{snd } M]\!]\rho &= \pi_2([\![M]\!]\rho) \\
[\![\text{T}]\!]\rho &= \text{T} \\
[\![\text{F}]\!]\rho &= \text{F} \\
[\![\text{if } M \text{ then } N \text{ else } O]\!]\rho &= \begin{cases} [\![N]\!]\rho, & [\![M]\!]\rho = \text{T} \\ [\![O]\!]\rho, & [\![M]\!]\rho = \text{F} \end{cases} \\
[\![\text{flip } p]\!]\rho &= \text{Ber}\, p \\
[\![r]\!]\rho &= r \\
[\![M \oplus N]\!]\rho &= ([\![M]\!]\rho) \oplus ([\![N]\!]\rho) \\
[\![\text{unif } [0,1]]\!]\rho &= \text{Unif}\, [0,1] \\
[\![\text{for}(n, M_{\text{init}}, i\ X.\, M_{\text{step}})]\!]\rho &= \text{loop}(1,\ [\![M_{\text{init}}]\!]\rho,\ \lambda k.\ [\![M_{\text{step}}]\!]\rho[i \mapsto k,\ X \mapsto v]) \\
\text{where loop}(k, u, f) &= \begin{cases} \text{ret } u, & k > n \\ v' \leftarrow f(k, v);\ \text{loop}(k+1, v', f), & \text{otherwise} \end{cases}
\end{aligned}
$$

(c) Denotational semantics

Figure 2.2: Terms in Appl

$$\iota : A \to \mathcal{G}\, B \tag{2.1}$$
$$\kappa : B \to \mathcal{G}\, C \tag{2.2}$$
$$\kappa \underset{B}{\circ} \iota : A \to \mathcal{G}\, C \tag{2.3}$$

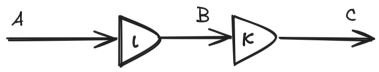


Figure 2.3: A simple probabilistic program

For a measurable space (A, Σ_A), a probability measure is of type $\mu : \Sigma_A \to [0, 1]$. This gives a distribution on A. The intuition is that given μ we can sample some value from it. Each triangle below is a kernel, and the types of its input and output are indicated. We compose the kernels along the type B as $\kappa \underset{B}{\circ} \iota$. The types of the individual kernels and composed kernel of Fig 2.3, is given below:

The interpretation of the kernel ι is that for every (deterministic) value of A it gets, it will give you a (probabilistic value) distribution over type B. So composition here, is sampling from the $\mathcal{G}\, B$, getting a deterministic value of type B and passing that to the kernel κ. This sampling is done ranging over all the values of B in proportion to their distribution, and taking a weighted average of the values of type $\mathcal{G}\, C$. More precisely, this is the integral: $\kappa \underset{B}{\circ} \iota = \lambda a.\, \int \lambda b.\, \kappa(b)\, d\iota(a)$

Notice that $\circ$ has a type very similar to the $\gg\!=$ (in the Giry monad). $\circ : (B \to \mathcal{G}\, C) \to (A \to \mathcal{G}\, B) \to \mathcal{G}\, C$. If we just swap the arguments and partially apply an $a : A$ on ι, we see that the bind of the Giry monad composes a measure with a kernel like so $\iota(a) \gg\!= \kappa : \mathcal{G}\, C$ We now take a step back, and have a look at a more general problem.

Having gained some intuition for how $\gg\!=$ works let's provide its definition. In Lean, it is defined as follows in Mathlib:

```
def bind (μ : Measure α) (κ : α → Measure β) : Measure β :=
  join (map κ μ)
```

`map` refers to taking a pushforward, so $\mathrm{map}\, \mu\, \kappa = \kappa_{\#}\mu : \mathcal{G}\, (\mathcal{G}\, \beta)$. This makes sense; for each $a : \alpha$ we get some $\mathcal{G}\, \beta$ following κ, so the measure on α naturally turns into a measure on measures on β. `join`[3] collapses the type: $\mathrm{join} : \mathcal{G}\, (\mathcal{G}\, \beta) \to \mathcal{G}\, \beta$. It takes some measurable set $E : \Sigma_\beta$ and the measure it returns is the weighted sum of all the measures in $\kappa_{\#}\mu : \mathcal{G}\, (\mathcal{G}\, \beta)$. Precisely it is the integral

$\mathrm{join}\, \mu'\, E = \int (\lambda\, \mu \mapsto \mu(E))\, d\mu'$

Unfolding the definitions we have: $(\mu_\alpha \gg\!= \kappa)\, E = \int (\lambda\, \mu_\beta \mapsto \mu_\beta(E))\, d(\kappa_{\#}\mu_\alpha)$.

We prove one of the monad laws to check that these definitions make sense. Let us prove that $\mathrm{join}\, (\mathrm{ret}\, \mu) = \mu$. Take an arbitrary measurable set E, and we prove through extensionality:

$\mathrm{join}\, (\mathrm{ret}\, \mu) E = \int (\lambda\, \mu' \mapsto \mu'(E))\, d(\mathrm{dirac}\ \mu)$

$$\begin{aligned} &\mathrm{join}\, (\mathrm{ret}\, \mu) E \\ &\qquad = \int (\lambda \mu' \mapsto \mu'(E))\, d(\mathrm{dirac}\, \mu) \\ &\qquad = (\lambda \mu' \mapsto \mu'(E))\ \mu \\ &\qquad = \mu(E) \end{aligned}$$

As expected. We justify the step dropping the integral, with the fact that diracμ concentrates all of its probability on μ. Viewing the integral as a weighted sum, a weight of 1 is given to μ. Next we take a step back from Appl and consider a wider setting.

[3] `bind` is derived from join in a usual categorical presentation of a monad

```
let x = t in          let y = u in
let y = u in    =     let x = t in
v                     v
```

Figure 2.4: Commutativity of let bindings

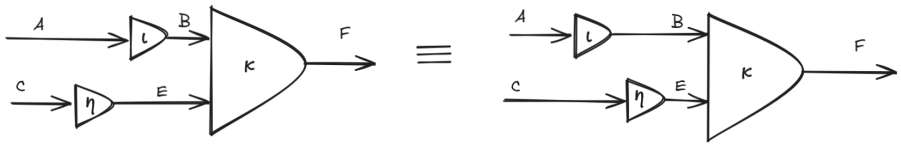


Figure 2.5: Commutativity of let bindings diagrammatically

2.2.2 The let binding problem

Staton [2017] provided a compositional semantics for first-order Bayesian probabilistic programming language (Bppl), which forms the foundation of program logics, for modern probabilistic programming languages. Note that Bppl, is Appl with the `observe` construct, and its semantics is more general than that of Appl. However, we choose to present a fundamental difficulty plaguing the semantics of Bppl, which was solved in [Staton, 2017] through the introduction of `s-finite kernels`. It will give us a picture of the surrounding landscape of probabilistic program semantics, the difficulty in establishing even the most basic properties of programs in this paradigm, and a topic to further build our intuition of *kernels*.

In search of the right semantics we lay down the criteria, that we would like to validate the following basic commutativity property that programmers expect of a let binding. Specifically, when neither x is free in u nor y is free in t:

We can represent each of these programs as a kernel having two inputs; one coming from each let binding[4]. To clarify, these kernels have multiple deterministic values they take in order to return a single probabilistic value (a measure). The commutativity property is illustrated by the equality of the two programs in Fig 2.5.

$\kappa \underset{B}{\circ} \iota$ is given by the following on the left and right respectively

$\lambda a\ c. \int \lambda e.\ \left(\int \lambda b.\ \kappa(b, e)\ d\iota(a)\right) d\eta(c)$

$\lambda a\ c. \int \lambda b.\ \left(\int \lambda e.\ \kappa(b, e)\ d\eta(c)\right) d\iota(a)$

Though the integrals may look intimidating, all that's happening is swapping the order of integration. Fubini's theorem allows us to swap the order of integration under certain conditions. Though we presented the kernel as a single type, there is in fact a taxonomy of different types of kernels, based on the restrictions on the measure of their output type. In Appl, we are actually using *Markov kernels* of type $A \to \text{ProbMeasure}\, B$, kernels where the output must be a probability measure. In general a kernel has type $A \to \text{Measure}\, B$. Fubini's theorem holds for Markov kernels [Borgström et al., 2017], but Bppl programs can't restrict themselves to Markov kernels[5]. Staton [2017] shows that this theorem holds, if instead of general kernels, we restrict ourselves to `s-finite kernels`. A kernel $\kappa : A \to \mathcal{G}(B)$ is finite if there is a finite $r \in [0, \infty)$ such that, for all a, $\kappa(a, B) < r$. κ is s-finite if there is a sequence $\kappa_1 \ \ldots\ \kappa_n \ \ldots$ of finite kernels and $\Sigma_{i=1\ldots\infty}\ \kappa_i = \kappa$.

In short, topological transformations of these diagrams corresponding to the motivating example in 2.4 do not change the semantics.

Mathlib's probability theory "API" provides many theorems with s-finiteness of kernels as a side-condition since it operates at the greatest level of generality, so it is useful to know about them

[4] In our actual semantics this is not fully accurate since at every line all preceding let bindings (the full context) are visible. But here we have taken care to state that the variable introduced by the first let binding is not visible in the second one, so the diagrams presented are a valid simplification of the actual semantics

[5] This is due to Bayes' theorem yielding unnormalised measures which are in general computationally intractable to normalise (convert to a probability measure)

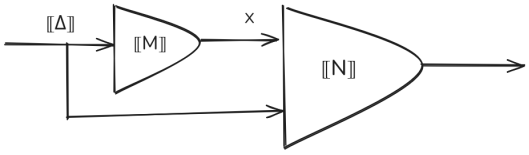
even if in this project we are only concerned with Markov kernels.

2.2.3 Semantics of let binding

We are now in a position to understand the whole semantics of Appl terms in Fig 2.2c in particular that of let bindings. Note that we refer to a term as a program if it has a type $\mathbf{G}\,ty$. In this definition M and N must be programs.

$[\![X \leftarrow M;\ N]\!]\rho = v \leftarrow [\![M]\!]\rho;\ [\![N]\!][X \mapsto v]$

The denotation of a program is precisely a Markov kernel, which allows us to understand the semantics simply as a diagram:



Here we are copying the value $\rho : [\![\Delta]\!]$ into both kernels. We can define this copying operation with a copying kernel which is provided by Mathlib. As can be seen kernels provide a primitive that is ideally suited to compositionally building complex probabilistic models. Which is exactly what we do in our probabilistic programming languages!

2.3 Separation Logic

Separation logic has been extremely productive in the last two decades in reasoning about imperative programming languages, after first being introduced in [O'Hearn et al., 2001]. The classical separation logic introduced a compositional state, specifically a heap (modelled as a function $\mathbb{N} \to Val$), which was required along with a more usual variable environment, to interpret any assertion. This heap or more generally resource, had an algebraic structure to it: the most fundamental one being a partial commutative monoid (PCM), with the binary operator dictating when it is valid to combine two heaps. Many more complex algebras for resources were studied in later works [Calcagno et al., 2007; Pym et al., 2004]. Separation logic was identified as a natural tool to reason about imperative programs mutating heap based memory at a suitable level of abstraction. It then also found success in reasoning about concurrent programs [Brookes and O'Hearn, 2016].

2.3.1 Classical separation logic

The first separation logic was proposed by O'Hearn et al. [2001] for reasoning about programs which manipulate a heap. Before we move to probabilistic separation logic which is the main topic of the report, we first present separation logic in a simpler setting. Intuitively, separation logic is just first order logic with the additional connectives of the separating conjunction and separating implication (notation: $*$ and $-\!\!*$). These separating connectives are defined with respect to a *resource*. In the original separation logic we have a simple resource which is a finite partial function from a set denoted Locations to a set Values (fig 2.6a). The notation $h_0 \star h_1$ is a partial binary operation denoting the union of h_0 and h_1 if they are disjoint. So we see that in order for a separating conjunct to hold we need to split the resource appropriately such that each part satisfies a conjunct. This is the principle behind all separation logics. Each proposition "owns" unique resources. One can imagine how each proposition makes assertions about parts of a program, and it is made explicit which parts of the heap, that part of the program "owns" and is thus able to modify. In short the logic allows reasoning in a resource-compositional manner. And specifically classical separation logic reasons about "memory" in languages like C, where the programmer must consciously manage the memory.

The other two *spatial connectives* are emp and $-\!\!*$. emp denotes the heap is an empty mapping, acting as the identity element to a monoid formed with $*$. $-\!\!*$ is to $*$ what $\to$ is to $\wedge$. Intuitively,

$P \mathrel{-\!\!*} Q$ holds under a heap q_- if P holds under some heap p for which the combined resource $q = p \star q_-$, satisfies the goal Q.

Next observe the definitions of *forall quantification*. We quantify over the set `Values` that the given variable used to define P can take. Though we haven't introduced the programming language that this logic reasons about, note that the variables form the bridge between the language and logic, since these variables are shared between programs and assertions we make about the program. We do not go into the specifics of introducing the programming language for this particular logic, since that takes us too far afield.

$$\text{Heaps} \triangleq \text{Locations} \rightharpoonup_{\text{fin}} \text{Values} \qquad \text{Stores} \triangleq \text{Variables} \rightharpoonup_{\text{fin}} \text{Values}$$

(a) Basic sets and structures

$$\begin{array}{rcll} E, F, G & ::= & x, y, \ldots \mid 0 \mid 1 \mid E + F \mid E \times F \mid E - F & \text{Numeric Expressions} \\ B & ::= & \text{false} \mid B \Rightarrow B \mid E = F \mid E < F & \text{Boolean Expressions} \\ P, Q, R & ::= & B \mid E \mapsto F & \text{Atomic Formulae} \\ & \mid & \text{false} \mid P \Rightarrow Q \mid \forall x.P & \text{Classical Logic} \\ & \mid & \text{emp} \mid P * Q \mid P \mathrel{-\!\!*} Q & \text{Spatial Connectives} \end{array}$$

(b) Syntax of separation logic

$$\begin{array}{lcll} s, h & \models & B & \text{iff } B \\ s, h & \models & E \mapsto F & \text{iff } [\![E]\!]_s \in \text{dom}(h) \text{ and } h([\![E]\!]_s) = [\![F]\!]_s \\ s, h & \models & \text{false} & \text{never} \\ s, h & \models & P \Rightarrow Q & \text{iff if } s, h \models P \text{ then } s, h \models Q \\ s, h & \models & \forall x.P & \text{iff } \forall v \in \text{Values}.\ [s \mid x \mapsto v], h \models P \\ s, h & \models & \text{emp} & \text{iff } h = [\,] \text{ is the empty heap} \\ s, h & \models & P * Q & \text{iff } \exists h_0, h_1.\ h_0 \star h_1 = h,\ s, h_0 \models P \text{ and } s, h_1 \models Q \\ s, h & \models & P \mathrel{-\!\!*} Q & \text{iff } \forall h'.\ \text{if } h' \star h \text{ exists and } s, h' \models P \text{ then } s, h \star h' \models Q \end{array}$$

(c) Semantic interpretation

Figure 2.6: Definitions of separation logic

2.3.2 Zoo of separation logics

In this section we see increasingly abstract presentations of separation logic to spot patterns amongst them, and give definitions of characteristics that help classify separation logics. Identifying these characteristics and "telling" Iris about it, lets us effectively use the proof automation/engineering developed by the Iris framework for our probabilistic separation logic (which will be presented later).

Abstract resources

It was hinted in the previous section that emp and $*$ are like a monoid. In general, what we mean by a "resource" can be modelled with some algebraic structure. The spatial connectives $*$ and $\mathrel{-\!\!*}$ are both defined using the binary operation $\star$. At the core of a separation logic is the partial commutative monoid (PCM) formed by emp and $\star$. A partial monoid is just a monoid accounting for the fact that $\star$ is a partial operation. So we guard all the laws with existence checks. This

turns out to be quite messy in a formalism, as seen for example in the `assoc` law. The below code shows the formalisation used taken from Bppl.Lilac.Krm.

```
notation a:arg " ⋆ " b:arg ⇒ PcmBase.binop a b

class Pcm (α : Type*) extends One α where
  binop : α → α → Option α -- partial binary operation denoted '⋆'
  one_mul : ∀ a : α, 1 ⋆ a = a
  comm : ∀ a b, a ⋆ b = b ⋆ a
  assoc : ∀ a b c : α,
    ((⋆) <$> (a ⋆ b) <*> (some c)).join = ((⋆) <$> (some a) <*> (b ⋆ c)).join

class Krm (α : Type*) extends Pcm α, Preorder α where
  le_mul_mono : ∀ x x' y y' p' : α,
    x ≤ x' → y ≤ y' →
    (x' ⋆ y' = some p') → ∃ p, (x ⋆ y = some p) ∧ p ≤ p'
```

Lean follows a Haskell-like syntax. Above we define the typeclass representing PCM and KRM. `Pcm` extends `One`, which gives a $(1 : \alpha)$ which we interpret as `emp`. `assoc` is using functor syntax (from haskell) and `join : Option (Option α) → Option α`. This code is included to define precisely what is meant by partial monoid and krm, and to familiarise the reader with lean syntax.

The more interesting development is the order structure in the algebra. This can be motivated by wanting to model an affine separation logic (defined in the next section). Intuitively an affine separation logic always allows a larger heap to validate an assertion if a smaller heap does so too. This was not the case for the classical separation logic presented in the previous section which is a linear separation logic. Concretely only the single celled heap $\{1 \mapsto 42\}$ would validate the assertion $1 \mapsto 42$, and importantly $\{1 \mapsto 42, 2 \mapsto 42\}$ would not validate the same assertion. However, in an affine separation logic, any heap which contains $\{1 \mapsto 42\}$ would validate the assertion. Each of these flavours of separation logic are useful in different contexts. Classical separation logic is useful in a language with manual memory management while affine separation logic is used in a language with garbage collection. We see that we want an ordering in our algebra to express this "contains" relation or more generally how "informative" a resource is. So we introduce a preorder to the algebra, turning the PCM into a Kripke resource monoid (KRM) [Galmiche et al., 2005]. Note that KRMs also encompass PCMs/linear case, if we simply set the preorder to equality. For the affine heap-based separation logic, the preorder is subset inclusion on the domain of heaps. The standard connectives of separation logic (those connectives shared by all separation logics) can be generated from a KRM, which is the approach taken in the Lilac formalisation.

Logic of Bunched Implications

Abstracting even further, we would like to get rid of explicit mentions of the resource and the satisfiability relation $(s, h \models P)$ and talk instead about entailments of the form $\vdash P$ or $P \vdash Q$. Indeed, *logic of bunched implications* (BI) as presented by [O'Hearn and Pym, 1999] is what classical separation logic is based on. It can be described as a logic in which the familiar "intuitionistic" connectives of first order logic are combined with emp, $*$ and $-\!\!*$ (called spatial connectives, alluding to heaps), in the same logic. We can recover the notion of resources by our interpretation of logical entailment: $P \vdash Q$ is defined as (using connectives in the meta-logic) $\forall s, h (s, h \models P \Rightarrow s, h \models Q)$. The definition of $\vdash$ is in general specific to each logic, but in practice (for separation logic) usually has the above shape. Having established this greater level of abstraction, we can more succinctly define some characteristics of different separation logics which will come in handy when instantiating the Iris framework. All subsequent figures in this section are reproduced from [Krebbers et al., 2018].

First let's precisely define the relation between $*$ and $-\!\!*$ (identical to $\wedge$ and $\to$)

$$\text{emp} * P \dashv\vdash P$$

$$P * Q \vdash Q * P$$

$$(P * Q) * R \vdash P * (Q * R)$$

$$*\text{-Mono}\quad \frac{P_1 \vdash Q_1 \qquad P_2 \vdash Q_2}{P_1 * P_2 \vdash Q_1 * Q_2}$$

$$*\text{-Intro}\quad \frac{P * Q \vdash R}{P \vdash Q -\!\!* R}$$

$$*\text{-Elim}\quad \frac{P \vdash Q -\!\!* R}{P * Q \vdash R}$$

Next we define the affine propositions: P as any proposition satisfying $\forall Q.\ P * Q \vdash Q$. In other words P can always be dropped. An affine separation logic is one where all propositions are affine. Affine propositions can also be equivalently defined as $P * \text{True} \dashv\vdash P$ or $\text{True} \dashv\vdash \text{emp}$. Another class of propositions is persistent propositions: those propositions which may be duplicated to establish the goal. So for instance if P is persistent then $P \vdash P * P$. Intuitionistic propositions are a final class of propositions which are both affine and persistent. Working with intuitionistic propositions in the premise feels like working with the familiar first order ("intuitionistic") logic, since we don't at all need to keep track of how many times these propositions are used. Because of this Iris treats intuitionistic propositions specially, keeping them separate from the rest of the "spatial" propositions in the premise. So a user of the Iris proof-mode will see an "intuitionistic context", following rules familiar to them from using the host theorem prover (Rocq or Lean), and additionally the spatial context, which they will need to treat more carefully. Examples follow in the next section.

2.3.3 Iris and MoSeL

Iris is a general framework for tactic based proofs in a separation logic. Iris (specifically MoSeL) can be understood according to the layers of abstraction it provides.

```
KRM -> BI                                  ->                  Iris Proof Mode
    |_ Probability Specific connectives -> typeclass instances -|
```

This is sort of the general structure of how Iris is meant to be used by someone instantiating it. You start with a KRM or similar algebraic structure which is the resource model, from which there is a pre-determined construction of a BI (those connectives common to all separation logic). Given this Iris proof mode will be aware of the standard properties of a separation logic. For example, it is able to prove the following goal, note it has implicitly applied both commutativity of the separating conjunct Todo: example 1 We are also able to introduce both variables and premises with the same command, to build a context of everything we have to prove our goal. Todo: example 2 What is unique about the Iris proof mode when compared to Lean or Rocq, is that it has an additional spatial context. Todo: example with spatial context In short it is engineered to provide an interface which mimics the natural reasoning style of the user. The IPM understands separation logic and endeavours to automate common boring tasks.

Though these seem like rather basic conveniences they are quite difficult to provide in a manner extensible to all separation logics. In order to understand how the proof mode works (as we will be instantiating Iris), it is first necessary to understand tactic based proofs in general.

2.4 Theorem Proving

2.4.1 Dependent type theory

Examples in this section are based on the excellent introduction to the calculus of inductive constructions by Paulin-Mohring [2014].

Foundational theorem proving is proving theorems with only the axioms (foundations) of mathematics assumed. Likewise, foundational software verification is proving theorems (properties) of programs.

Here we focus on the theory of foundational theorem proving and in the next (2.4.2) we focus on how these ideas scale to large projects in practice. Note that we are interested in "proving theorems" as software verification is just a particular instance of theorem proving (using program logics, which is covered in detail in the rest of this report).

We now focus on a specific "Foundation of Mathematics" which has proved ideal for theorem proving in practice, as displayed by the most successful theorem provers - Rocq, Lean, Agda, Idris - all being based on dependent type theory. Dependent type theory has the remarkable property of encoding both propositions (synonymous to theorems) and proofs on one side and types and programs on the other, as part of the same language. These two sides are in fact

formally indistinguishable. The Curry-Howard isomorphism (or propositions as types method) is an observation that the rules governing logic (introduction elimination rules), are much the same as the inference rules governing typing judgements. So the two can be used interchangeably. Certain types in our type theory will be interpreted as propositions, and terms of that type will be proofs of that proposition!

Below is a description of Calculus of Constructions (CoC), a flavour of dependent type theory on which Rocq (CoC and its successors were developed for Rocq) and Lean are based. Agda is based on Martin Lof Type Theory (MLTT) which differs in a matter of predicativity. We will explain and revisit this briefly at the end.

Pure CoC consists of very few rules and constructs and it is very surprising that it is a valid foundation for all mathematics. We start with a typed lambda calculus in which both the terms and the types are given by the *same* syntax. This is known as a pure type system (PTS). Instead of function/arrow type we have the dependent product, $\Pi(x : A), B(x)$, where B may be defined using x. Additionally, (as there is no separate syntax for types) all types have *sort*. In other words, every construction in the calculus must have a type, which itself will have a type and so on, infinitely. A has type $\mathbf{Type_1}$ (or *sort* $\mathbf{Type_1}$, to acknowledge that A is itself a type). There is actually an infinite hierarchy of these types: $A : \mathbf{Type_1} : \mathbf{Type_2} : \mathbf{Type_3} \ldots$, where the subscripts are indices known as *universes*.

The type of the identity function on A is given by $\Pi(\_ : A), A$. We use the notation $A \to A$ in this case since it is not dependent. For completeness the identity function is typed $(\lambda x : A \mapsto x) : \Pi(\_ : A), A$ As a first example of a dependent type take the type of the polymorphic identity function, which takes a type and gives the identity function for that type: $\Pi(A : \mathbf{Type_i}), A \to A$. So $A \to A$ (or $\Pi(\_ : A), A$) is using A in its definition and is therefore dependent. Again for completeness we type the term which is the polymorphic identity function: $\lambda A : \mathbf{Type} \mapsto \lambda x : A \mapsto x : \Pi(A : \mathbf{Type_i}), A \to A$. We will not go into the detail of universe polymorphism which strictly speaking should be considered since the index i is unquantifed in the above expressions.

We now move to expressing logic in CoC, but before doing so we introduce the sort $\mathbf{Prop} : \mathbf{Type_1}$, so that the (infinite) set of sorts is given by $\mathcal{S} := \{\mathbf{Prop}\} \cup \bigcup_{i \in \mathbb{N}} \{\mathbf{Type}_i\}$. **Prop** is special and is *impredicative* as we will see in 2.4.1. We map first order logic propositions to types of sort **Prop**. For example, if $A : \mathbf{Prop}$ and $B : \mathbf{Prop}$, then logical implication $A \Rightarrow B$ corresponds to the type $A \to B$. This is justified by the meaning of implication being able to prove B given a proof of A. Forall quantification (also) corresponds to dependent product types: given some $T : \mathbf{Type_i}$ and $P : \mathbf{Prop}$, the logical statement $\forall x : T, P(x)$ corresponds to the type $\Pi(x : T), P(x)$. This also makes sense since to prove a universally quantified statement we need a proof of $P(x)$ for every x, which is what a term of type $\Pi(x : T), P(x)$ would give. We will use the notation $\forall x : T, P$ for $\Pi(x : T), P(x)$, freely, as is standard in Lean.

Dependent type theory as a foundation is an example of *constructive mathematics* because proofs are always built by assembling the required data as seen in above example. With that intuition in mind, let's show how some other logical connectives are represented:

As seen in this example, the standard recipe in constructive mathematics is to specify a proposition by what is required to prove it. In the next example we see representation also arises from what the connective enables us to prove. Take False $:= \forall(P : \mathbf{Prop}).\ P : \mathbf{Prop}$. First notice that if we have a proof of False, that is a term of type $\forall(P : \mathbf{Prop}).\ P$, we would get the proof of any proposition P, out of that, encoding false entails anything. Also, it is not possible to construct a term of type $\forall(P : \mathbf{Prop}).\ P$, ie, there is no proof of False. A strange thing that one might notice is that we are quantifying over everything in **Prop**, in order to obtain something of type **Prop**.

Predicativity

In this section we continue with more examples of logic using dependent type theory since it comes up in a different context in the definition of Iris, which took a while to figure out.

Let's now introduce all the typing rules. Remember $\mathcal{S} := \{\mathbf{Prop}\} \cup \bigcup_{i \in \mathbb{N}} \{\mathbf{Type}_i\}$.

We notice that there are two typing rules for the dependent product: `PI-PROP` for sort **Prop** and `PI-TYPE` for all other sorts. `PI-TYPE` is saying that the dependent product falls into a sort which is the max of the sort of its domain and codomain. While there is also a special case rule `PI-PROP`,

$$\frac{\Gamma \vdash}{\Gamma \vdash \mathbf{Prop} : \mathbf{Type}_1}\ \textsc{Type-Prop} \qquad \frac{\Gamma \vdash}{\Gamma \vdash \mathbf{Type}_i : \mathbf{Type}_{i+1}}\ \textsc{Type-Form}$$

$$\frac{\Gamma \vdash x : A \in \Gamma}{\Gamma \vdash x : A}\ \textsc{Var} \qquad \frac{\Gamma \vdash A : s \qquad x \notin \Gamma \qquad s \in S}{\Gamma, x : A \vdash}\ \textsc{Ctx-Ext}$$

$$\frac{\Gamma, x : A \vdash t : B}{\Gamma \vdash \mathrm{fun}\, x : A \Rightarrow t : \Pi x : A, B}\ \textsc{Fun-Intro} \qquad \frac{\Gamma, x : A \vdash B : \mathbf{Prop}}{\Gamma \vdash \Pi x : A, B : \mathbf{Prop}}\ \textsc{Pi-Prop}$$

$$\frac{\Gamma \vdash A : \mathbf{Type}_i \qquad \Gamma, x : A \vdash B : \mathbf{Type}_j}{\Gamma \vdash \Pi x : A, B : \mathbf{Type}_{\max(i,j)}}\ \textsc{Pi-Type} \qquad \frac{\Gamma \vdash f : \Pi x : A, B \qquad \Gamma \vdash a : A}{\Gamma \vdash f\, a : B[a/x]}\ \textsc{App}$$

$$\frac{\Gamma \vdash t : A \qquad \Gamma \vdash B : s \qquad A \preceq B}{\Gamma \vdash t : B}\ \textsc{Conv}$$

Figure 2.7: Inference rules for CoC [Paulin-Mohring, 2014]

which says that regardless of the domain, if the codomain is in **Prop**, then the Π-type stays in **Prop**. We say that the sort **Prop** is impredicative, because elements in it are defined by quantifying over **Prop** itself! Remembering False := $\Pi(P : \mathbf{Prop}).\ P : \mathbf{Prop}$, we see exactly this. And suppose that we instead were to type this with the rule `PI-TYPE` and $\mathbf{Prop} \triangleq \mathbf{Type}_0$, then the domain, $\mathbf{Prop} : \mathbf{Type}_1$ and codomain $P : \mathbf{Prop}$ would give the Π-type sort $\mathbf{Type}_1$, making **Prop** *predicative*. However, we would like False : **Prop**, so the impredicativity of **Prop** is desirable. Let's also recall the definition of forall quantification which also depended on predicativity: $\forall x : T, P$, we can quantify over values in much larger universes or quantify over all **Prop**s and still end up with a type in **Prop**. Impredicativity is powerful but fragile. It can be shown that if $\mathbf{Type}_1$ is also impredicative, then this type theory would become inconsistent [Coquand, 1986].

As a final example lets have a look at $A \wedge B \triangleq \forall(P : \mathbf{Prop}).\ (A \to B \to P) \to P : \mathbf{Prop}$. Or making things a little more explicit: $\wedge := \lambda(A\ B : \mathbf{Prop}) \mapsto \forall(P : \mathbf{Prop}).\ (A \to B \to P) \to P : \mathbf{Prop} \to \mathbf{Prop} \to \mathbf{Prop}$, where we are first using lambda abstractions and then a Π-type construction.

Proof Relevance

Finally, Proof Relevance separates the type theories of Lean and Rocq. Lean is proof irrelevant, meaning terms of types in Prop are definitionally equal[6]. So for instance if we have a structure bundling data and a proof of a property like so in Lean:

```
structure MeasurableFun (α β : Type*) [MeasurableSpace α] [MeasurableSpace β] where
  /-- underlying function -/
  toFun : α → β
  meas: Measurable toFun
```

the equivalence of two `MeasurableFun`s is determined solely by the equality of the `toFun`, since `meas` is a (dependent) proof and will be definitionally equal if `toFun` is equal. In practice this is incredibly convenient, and is applied automatically throughout the codebase to simplify proof objectives. It was always used in an elegant abstraction in establishing that certain constructions form a KRM, which we will detail later.

The choices made in Lean's type theory to bring about proof irrelevance do have some major theoretical drawbacks such as leading to undecidability of definitional equality [Carneiro, 2019, see chapter 3], which however don't seem to matter too much in practice.

Calculus of Inductive Constructions

Mention here that 1) Inductive families... 2) strict positivity

[6]Rocq on the other hand is proof relevant by default. However, after noticing that Lean's proof relevance was indispensable for some projects, they decided to introduce a sort `SProp`, which is proof irrelevant

In this section we hope to have gotten across that theorem proving in a dependent type theory is exactly type checking of terms constructed by the user. This is all that is needed to motivate the next section on how this is partially automated and scaled in Rocq and Lean (going into more Lean specific detail). We also introduced the notions of predicativity and proof relevance, both of which were encountered over the course of the project. Interestingly they also cleanly separate 3 of the most significant proof assistants. They will be mentioned briefly where relevant but are not particularly important in understanding most of this report.

2.4.2 Metaprogramming

We now know that constructing a proof involves assembling many subproofs according to the typing rules to obtain a term of a desired Type (actually Prop). It is however extremely time-consuming and error-prone to go down to this level of detail when constructing a proof. Pen and paper proofs operate at a much higher level, and the use of metaprogramming in Proof assistants, is motivated by the desire to remove the mental overhead of mechanised proofs, and move closer to the style of pen and paper proofs.

"Tactics" are metaprograms meant for constructing proof terms. A Tactic block is started in Lean using *by* syntax: in our code if we wanted a term (proof) of type $A : Prop$, we can write $(by \ldots) : A$ (though the type A, would usually be inferred by lean). So how does the *by* tactic make it easier to construct the term? It provides a proof context, of premises and goals, which is actually additional state wrapped in a monad called `TacticM`. Inside a tactic block a sequence of `TacticM` monads are composed, each monad constructed using one of Lean's ecosystem of tactics, incrementally modifying the proof state until "all goals are solved". And at the end of a tactic block, using the information acquired in the monadic state a term of the desired type should be constructed using the information collected in the TacticM monad.

We have been pretty vague about what this "proof context" and "goals" actually are so far. In lean there are a few more constructors for terms than in the simply typed lambda calculus, on top of lambda abstraction and application. Terms are called `Expr` in lean. We introduce the constructor `mvar : MVarID → Expr`. For building an expression given a metavariable. This is at the heart of how tactics work. Tactics communicate to each other what has been proven and what is left to prove using metavariables. We will illustrate this through a simple example reproduced from [leanprover community, 2024, chapter 4]. In the proof of the following theorem we show the evolution of the proof context, line by line.

```
example {α} (a : α) (f : α → α) (h : ∀ a, f a = a) : f (f a) = a := by
  apply Eq.trans
  · apply h
  · apply h
```

Metavariables are used to represent unknown terms of known types, which may be holes in larger terms or a goal we want to prove (term of a certain Prop that we are looking to construct). As soon as the tactic block starts with `by`, a metavariable `?m1` is created which is unassigned but has type `f (f a) = a`. Now we look at the state after the first `apply`. The types of `Eq.trans` and `?m1` can be unified, so the tactic succeeds and we have new proof state as follows:

To a user it would look like this:

```
...
⊢ f (f a) = ?b

...
⊢ ?b = a

...
⊢ α
```

But it's helpful to see the state of metavariables explicitly:

```
?m1 := Eq.trans α (f (f a)) ?b a ?m2 ?m3
?m2 : f (f a) = ?b
?m3 : ?b = a
```

```
?b
```

where `Eq.trans : ∀ (α : Sort u), ∀ (a b c : α), a = b → b = c → a = c`.

The old goal `?m1` has been assigned (albeit with holes in the assignment). Two new goals ?m2 and ?m3 were created and an additional mvar ?b, which is not a Prop was created as well. After the second `apply`, which acts on the goal ?m2, we assign ?m2 and also assign ?b through the unification process.

```
?m1 := Eq.trans α (f (f a)) ?b a ?m2 ?m3
?m2 := h (f a)
?m3 : ?b = a
?b  := f a
```

Only ?m3 needs to be assigned which is taken care of by the final `apply`. At the end of the tactic block, we obtain the term `Eq.trans α (f (f a)) (f a) a (h (f a)) (h a)` which is typed as `f (f a) = a`, as desired!

Chapter 3

Mechanising Lilac I: Definitions

In this chapter we take the reader through the mechanisation task that lay ahead, and the challenges encountered when closely following the definitions in the paper. We thoroughly motivate the modifications on the definitions in light of compatibility with the iris proof mode. This chapter is about getting the definitions right and the unexpectedly lengthy process one might go through in validating that the choices made are "correct".

3.1 Syntax and Semantics of Appl

To reason about Appl (see Sec 2.2), we first need to define Appl in Lean. A first attempt at doing this may be inspired by how compilers are written: define an AST for the syntax of the language, and a typechecker for accepting or rejecting AST terms. Following such an approach would be painful given our goal of specifying properties of programs. We would always have to guard for the possibility that a program is not well-typed, when writing a specification on it, and would lead to a proliferation of exception handling. Fortunately we are working in a dependently typed language, which is expressive enough to circumvent the problem by letting us define a syntax (AST) that is well-typed by construction.

We need to disambiguate references to types in Lean and types of Appl (which are in-fact terms in Lean). We will refer to the former as types in the *meta language* and to the latter as types in the *object language*.

There are several standard ways to define object languages which are well-typed by construction, which all differ in the crucial problem of representing (object language) variables. This seemingly innocent problem of how to represent variables is one of the biggest issues encountered in this project, and standard techniques don't quite work for Appl, because of its non-standard interpretation of (object language) types.

The final (Lean) representation chosen for Appl follows a technique called dependent de Bruijn indices [Chlipala, 2013, see chap 17, Reasoning about programming language syntax]. The reference also covers Parametric Higher Order Abstract Syntax which was heavily considered, along with the simple string based approach to variables, which is standard in Iris projects. For both alternatives it is not straightforward or perhaps not even possible to express that the denotation of a program is not an ordinary function from a variable context but a measurable function. Among these, only de Bruijn indices make this map explicit, so that we can express measurability.

First we present the types of Appl along with its denotational semantics, taken from Bppl.Lilac.Appl.

```
inductive Ty where
  | prod : Ty → Ty → Ty
  | bool
  | real
```

```
  | exp : ℕ → Ty → Ty -- Tyⁿ
  | G : Ty → Ty -- Giry monad

@[reducible] def Ty.den : Ty → MeasCat -- category of measurable spaces
  | prod ty₁ ty₂ => .of (ty₁.den × ty₂.den)
  | bool => .of Bool
  | real => .of ℝ
  | exp n ty => .of (∀ (_ : Fin n), ty.den) -- using MeasurableSpace.pi
  | G ty => .of (ProbabilityMeasure (ty.den))
```

This mirrors the definition in the paper [Li et al., 2023] with no deviations. Note that the denotation of Appl types is `MeasCat`, the category of MeasurableSpace, rather than `Type` as for most other object languages. It is crucial to annotate `@[reducible]`, which tells the typechecker to unfold `Ty.den` when checking if two types are definitionally equal. The importance of this will be made clear once we describe the denotation of terms.

First we describe the machinery for dependent de Bruijn indices. De Bruijn indices are a system of variable naming that facilitates guarantees by the type system that there is no usage of undeclared variables. The idea is to use natural numbers as names, where 0 refers to the nearest (rightmost) binding, 1 refers to the next one and so on. $(\lambda.\,\lambda.\#1 + \#0)$ would represent $(\lambda x.\,\lambda y.x + y)$ for example.

First we upgrade to a typed lambda calculus: $\mathbb{N}$ is replaced with `List Ty` and an index like #0 for example is a $ty : Ty$ at the head of the list. *Dependent* de Bruijn indices remove the worry that we may still refer to an index that is too high and thus unbound. We use inductive type, `Member`, which is a proof of membership in a `List Ty`, as indices with which we refer to variables.

```
inductive Member : α → List α → Type
  | head {a as}  : Member a (a::as)
  | tail {a b bs}: Member a bs → Member a (b::bs)
```

Note that `a` is syntax for an inferred argument. Here is the definition of terms. Note how `var` is defined using a `Member` as an index.

```
inductive Term : List Ty → Ty → Type
  | var : Member ty ctx → Term ctx ty -- a variable can never be unbound
  | ret : Term ctx ty → Term ctx (.G ty)
  | bind : Term ctx (.G ty₁) → Term (ty₁ :: ctx) (.G ty₂) → Term ctx (.G ty₂)
  | pair : Term ctx ty₁ → Term ctx ty₂ → Term ctx (.prod ty₁ ty₂)
  | fst : Term ctx (.prod ty₁ ty₂) → Term ctx ty₁
  | snd : Term ctx (.prod ty₁ ty₂) → Term ctx ty₂
  | T : Term ctx .bool
  | F : Term ctx .bool
  | ite : Term ctx .bool → Term ctx ty → Term ctx ty → Term ctx ty
  | r : ℝ → Term ctx .real
  | arith : Arith → Term ctx .real → Term ctx .real → Term ctx .real
  | cmp : Cmp → Term ctx .real → Term ctx .real → Term ctx .bool
  | flip : (p : ℝ≥0) → (h : p ≤ 1) → Term ctx (.G .bool) -- Ber p
  | unif01 : Term ctx (.G .real) -- Uniform[0,1]
```

In Lean's dot notation, `.G` will elaborate into `Ty.G` since the expected type at that location can be inferred. We can anticipate that the semantics of a term $te : .Gty$ (object language typing), would be a Kernel. `Arith` and `Cmp` are types for the syntax of operators like $+$, $-$, $\leq$, etc, so we omit them. The syntax follows straightforwardly from Fig 2.2a, presented in sec 2.2. In the Lean code, we also see that the syntax is typed (for example `arith` only takes terms of types `Ty.real` as arguments), so that all terms are correct by construction. Below we provide the denotation of selected terms:

```
@[simp] def Term.den : Term ctx ty → List.TProd (⟪·⟫) ctx -m→ ty.den -- measurable
  | var mem => ⟨fun env ↦ env.get mem, List.TProd.measurable_get mem⟩
  | ret X => ⟨fun env ↦ probDirac (X.den env), measurable_probDirac.comp X.den.2⟩
  -- In `bind` we are working exactly with kernels
  | @bind _ ty₁ ty₂ M N =>
```

```
    letI T := List.TProd (⟪·⟫) ctx
    letI k₁ : Kernel T (⟪ty₁⟫ × T) := (toMK M.den ×ₖ Kernel.id)
    letI k₂ : Kernel (⟪ty₁⟫ × T) ⟪ty₂⟫ := toMK N.den
    toDen (k₂ ∘ₖ k₁)
  | pair M N ⇒ ⟨fun env ↦ (M.den env, N.den env), Measurable.prod M.den.2 N.den.2⟩
  | fst M ⇒ ⟨fun env ↦ (M.den env).fst, Measurable.fst M.den.2⟩
  | T ⇒ ⟨fun env ↦ true, measurable_const⟩
  | ite P M N ⇒ ⟨fun env ↦ if P.den env then M.den env else N.den env,
      Measurable.ite (P.den.2 (measurableSet_singleton true)) M.den.2 N.den.2⟩
  | r x ⇒ ⟨fun _ ↦ x, measurable_const⟩
  | flip p hp ⇒ ⟨fun _ ↦ ⟨(bernoulli p hp).toMeasure, inferInstance⟩, measurable_const⟩
  | unif01 ⇒ ⟨fun _ ↦ ⟨lebI, inferInstance⟩, measurable_const⟩
```

Firstly, if `Ty` was not marked as `@[reducible]`, each denotation would not type check. In `Term.den : Term ctx ty → L ty.den` needs to be normalised to the actual denotation of a `Ty`. For example, if `Ty.real` does not reduce to $\mathbb{R}$ then in the semantics of `Term.r`, we have $x : \mathbb{R}$, but we want $x : Ty.real$ which will not type-check. This kind of type-level computation has its limits and during the project we had faced significant difficulties with being able to prove that two types are equal, but the reduction not happening automatically (called *definitional equality*), and thus the terms could not be typed as desired.

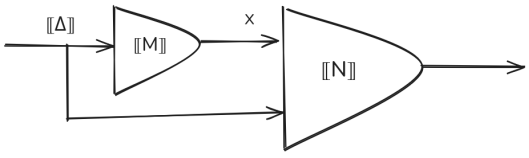
Note that we use the notation $\langle\!\langle ty \rangle\!\rangle$ for the denotation of an APPL type in the lean codebase since the standard $[\![ty]\!]$ was already in use in some dependencies.

Unfolding `List.TProd` gives `rs.foldr (fun r β ⇒ ⟪r⟫ × β) PUnit`. A measurable space is also getting constructed as the product type of the denotations is iteratively taken. A measurable space over `List.TProd` is provided by Mathlib in this fashion, if each of the components of the list has associated with it a measurable space. The main reason we are using dependent de Bruijn indices, even though there are drawbacks to readability of the proof interface, is because it is clear how to define a measurable space over a heterogeneous list.

The reason we are seeing the awkward notation of λ-functions (`fun env ...`), being enclosed in tuples $\langle \ldots \rangle$, is because we need to prove each of the functions from the context are measurable. They are of type `List.TProd (⟪·⟫) ctx -m→ ty.den` where the $-m\!\!\rightarrow$ is notation for `MeasurableFun`, which is the structure (flat inductive type) that these tuples elaborate into:

```
structure MeasurableFun (α β : Type*) [MeasurableSpace α] [MeasurableSpace β] where
  /-- underlying function -/
  toFun : α → β
  meas: Measurable toFun
```

and we can refer to the underlying function as the first field .1 and the measurability condition as the second field .2 which is what one may notice throughout the code. We are invoking Mathlib's measure theory API to get these measurability proofs. The most interesting definition of `Term.den` is that of `bind`. Since we know that the arguments of bind are typed $MN : \mathrm{G\_}$, we know they are both kernels, so we cast them into the kernel type provided by Mathlib and construct the following diagram (copied from Sec 2.2.3), in a point-free style.



Notes on the notation: `toMK` is short for "to Markov Kernel", and is simply a cast[1]. Given two kernels P and Q, the kernel product $P \times_m Q$, copies the inputs and return the output of each

[1]We attempted using an implicit coercion, which is implemented using typeclasses in Lean, but the required types could not be inferred automatically

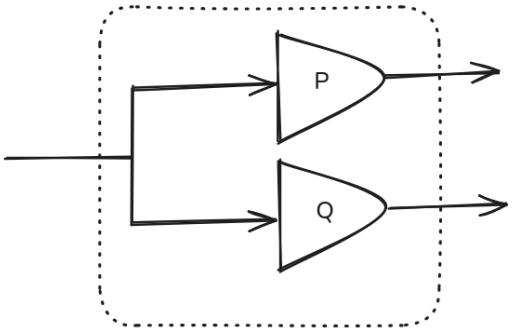


Figure 3.1: Kernel product between kernels P and Q

of P and Q as a tuple (product measure on a product measurable space). Again we represent it diagrammatically, this time within a dotted box in Fig 3.1:

Finally, `Kernel.id` the identity kernel, can achieve the effect of just passing on its deterministic input if we define it as Kernel.id x = dirac x. Our original definition was the definition from the paper, but it was very difficult to prove the measurability of the construct; there was no suitable Mathlib lemma, and proving measurability from scratch is involved, requiring finding relevant helper lemmas in Mathlib, and was long. Thankfully, measurable functions compose to measurable functions, and we only had to realise this perspective, of the definition of `bind`, before it became more natural to state the definition using kernels compositionally and get measurability for free!

3.2 Lilac

Lilac is a logic that is designed to reason about Appl programs at a level of expressivity that would be natural to a user, who could be a statistician or programmer. Its goals are to be able to express the kinds of specification on Appl, for us to be convinced that a program is doing what it is supposed to do, as well as provide sound proof rules to start from the definition of a program and be able to prove the specifications that we made. The following are a few core intuitions behind Lilac. Lilac aims to reason about random variables, which is at the level of abstraction natural for human reasoning (as opposed to descending down into measure theory and Lebesgue integrals, which form the semantics of Appl). Lilac's notion of separation is completely different to that of classical separation logic, with separation between two Lilac propositions meaning the mutual independence between random variables referenced in each proposition. We will explain this in detail soon in Sec 3.2.1. But first we need to define the resources at the heart of Lilac. The resource here, instead of heaps, are Probability spaces:

```
structure PSpace (Ω : Type*) where
  ms : MeasurableSpace Ω
  μ : Measure[ms] Ω
  is_prob : IsProbabilityMeasure μ
```

Essentially this is just a Probability Measure, a measure which assigns measure (or probability) 1 to the universal set, and all the data required to define it (a `MeasurableSpace`). So by analogy with classical separation logic, a Probability Space dictates the state of a probabilistic program. This is achieved by the values of variables in a probabilistic program being modelled based on this probability measure. Concretely, variables in the logic are measurable functions (aka random variables) from the $\Omega \xrightarrow{m} ty.den$ for some $ty : Ty$. This means that the distribution (the measure) over $ty.den$ is a pushforward of μ from the resource. Hence, the resource is controlling the distribution of every random variable which makes up expressions which makes up probabilistic programs.

3.2.1 Separation is probabilistic independence

Now we can present the notion of separation defined over probability spaces. One of the slogans is "separation is probabilistic independence". Probability spaces admit a binary operation, say $p \star q$ which can guarantee that the random variables defined through the resource p and those constructed through q are independent. The definition of this binary operation is in a sense a generalisation of independence of random variables: (note we use $\mathcal{E}$ $\mathcal{F}$ and $\mathcal{G}$ for measurable spaces)

Definition 2.2 [Li et al., 2023] (Independent Combination). Let $(\Omega, \mathcal{E}, \mu)$ and $(\Omega, \mathcal{F}, \nu)$ be probability spaces over the same ambient sample space Ω. A probability space $(\Omega, \mathcal{G}, \rho)$ is an independent combination of $(\Omega, \mathcal{E}, \mu)$ and $(\Omega, \mathcal{F}, \nu)$ if (1) $\mathcal{G}$ is the smallest σ-algebra containing $\mathcal{E}$ and $\mathcal{F}$, and (2) ρ witnesses the independence of μ and ν in the sense that for all $E \in \mathcal{E}$ and $F \in \mathcal{F}$ it holds that $\rho(E \cap F) = \mu(E)\nu(F)$.

The 2nd condition is familiar since it's the standard definition of independence of two events E and F. The novelty is that, independence is lifted from single events to all possible events occurring in two different measurable spaces. And that's how we get separation; we can just say an arbitrary random variable from p and q respectively are independent. Remember that we desire a partial binary operation to define a Kripke resource monoid (KRM). It is non-trivial to prove that the independent combination when it exists, is unique, required to show that independent combination is a function not a relation. The "pi-lambda-theorem" is used to prove both this and several other laws of KRM are satisfied.

3.2.2 Measurable spaces and random variables on the Hilbert cube

Probability spaces are defined parametrically over some type Ω, but Lilac makes a specific choice for Ω in order to define the notion of a finite footprint. We will return to finite footprint later in Sec 4.3, but here we first define $\Omega = [0,1]^{\mathbb{N}}$. Henceforth, denote the interval, $[0,1]$ as I. This type is known as the Hilbert Cube. The Hilbert Cube is a countably infinite sequence of intervals I, in other words $\mathbb{N} \to I$. First we must establish a measurable space over $I^{\mathbb{N}}$, to turn this into a probability space. Intuitively this would be obtained by taking the standard borel σ-algebra, Σ_I over each dimension and taking the product measurable space iteratively over all dimensions. In other words a fold over $\mathbb{N}$. Mathlib offers a more general way to construct this measurable space, as it defines `Measurable.pi`, which does not require a countable index, but works on any Π-type (function type). Like so:

```
instance MeasurableSpace.pi {X : δ → Type*} [m : ∀ a, MeasurableSpace (X a)]
    : MeasurableSpace (∀ a, X a) := ...
```

This is the standard borel σ-algebra, $\Sigma_{I^{\mathbb{N}}}$ on the Hilbert Cube. Each *measurable rectangle* is a cartesian product over infinite dimensions. And the general measurable set is constructed using unions or complements of the rectangles. This is the finest (or largest) Measurable Space over the Hilbert cube that we will work with. Remember we have one measurable space associated to each probability space, so there will be many different measurable spaces constructed over the Hilbert cube, but they will all be coarser (smaller) than the $\Sigma_{I^{\mathbb{N}}}$. Motivated by the aim of reasoning about random variables, $\Omega \xrightarrow{\text{m}} A$ (where A is some `ty.den`) we define: $RV\,A \triangleq I^{\mathcal{N}} \xrightarrow{\text{m}} A$such that the map is finite footprint. At the moment the finite footprint is an additional restriction that does not concern us. Measurable Spaces are implicit in the standard mathematical notation, so note that RV is defined w.r.t $\Sigma_{I^{\mathbb{N}}}$ (`MeasurableSpace.pi`) on $I^{\mathbb{N}}$.

Remember that $\mathcal{G}A$ is the space of measures over A.

3.2.3 Ownership is measurability

Let's provide some more intuition on how probability spaces can only admit certain random variables (thereby achieving its mutual independence requirements with variables from the other spaces), This pertains to the 2nd slogan "ownership is measurability". To do so, let's introduce the syntax and semantics of the logic:

$$
\begin{aligned}
S, T &\in \mathbf{Set} \\
A, B &\in \mathbf{Meas} \\
P, Q ::= {} & \top \mid \bot \mid P \wedge Q \mid P \vee Q \mid P \to Q \mid P * Q \mid P \mathrel{-\!\!*} Q \mid \\
& \forall x{:}S.\, P \mid \exists x{:}S.\, P \mid \forall_{\mathrm{rv}} X{:}A.\, P \mid \exists_{\mathrm{rv}} X{:}A.\, P \mid \\
& E \sim \mu \mid \mathrm{own}\, E \mid E \stackrel{\circ}{=} E \mid \mathbb{E}[E] = e \mid \mathrm{wp}(M, X{:}A.\, Q)
\end{aligned}
$$

Notice that we have two sets of quantifiers: one for deterministic variables and one for random variables. Over the course of the mechanisation, I noticed that Core Lilac (without the conditioning modality) does not use *deterministic variables*. This simplifies our presentation here, omitting deterministic contexts and variables, and the two quantifiers $\forall$ and $\exists$ above. The following rules are helpful to understand, how to construct terms in the syntax. We have already omitted deterministic context and quantifiers here:

$$
\begin{gathered}
\Delta ::= \cdot \mid \Delta, X : A \\
[\![X_1 : A_1, \ldots, X_n : A_n]\!] = A_1 \otimes \cdots \otimes A_n
\end{gathered}
$$

$$
\frac{\Delta \vdash P \qquad \Delta \vdash Q}{\Delta \vdash P \oplus Q}
\qquad
\frac{\Delta, X : A \vdash P}{\Delta \vdash \forall_{\mathrm{rv}} X{:}A.\, P}
\qquad
\frac{\Delta, X : A \vdash P}{\Delta \vdash \exists_{\mathrm{rv}} X{:}A.\, P}
\qquad
\frac{E \in [\![\Delta]\!] \xrightarrow{\mathrm{m}} A \qquad \mu \in \mathcal{G}A}{\Delta \vdash E \sim \mu}
$$

$$
\frac{E \in [\![\Delta]\!] \xrightarrow{\mathrm{m}} A}{\Delta \vdash \mathsf{own}\, E}
\qquad
\frac{E_1 \in [\![\Delta]\!] \xrightarrow{\mathrm{m}} A \qquad E_2 \in [\![\Delta]\!] \xrightarrow{\mathrm{m}} A}{\Delta \vdash E_1 \stackrel{\circ}{=} E_2}
\qquad
\frac{E \in [\![\Delta]\!] \xrightarrow{\mathrm{m}} \mathbb{R} \qquad e \in \mathbb{R}}{\Delta \vdash \mathbb{E}[E] = e}
$$

$$
\frac{M \in [\![\Delta]\!] \xrightarrow{\mathrm{m}} \mathcal{G}A \qquad \Delta, X : A \vdash Q}{\Delta \vdash \mathrm{wp}(M, X{:}A.\, Q)}
$$

Figure 3.2: Simplified Typing rules of Lilac

where we use this notation $\oplus \in \{\wedge, \vee, \to, *, \mathrel{-\!\!*}\}$

Lilac like any other separation logic has additional connectives beyond the standard connectives specified by BI that is required of it. The first row of the syntax above are standard connectives in BI, so their semantics is identical to that of classical separation logic, replacing heaps with probability spaces. The second row of quantifiers is explained later when explaining iris proof mode, which requires a different treatment of quantifiers to keep track of variables when carrying out proofs in the logic. The connectives in the third row are called logic (or probability) specific connectives and where we make interesting assertions about the state of a program. To understand the relation between the logic-specific connectives and the standard connectives, think of the syntax of the logic as a tree, the logic specific connectives tend to be the leaves, while the standard connectives are like nodes (they are mostly binary). The nodes are there to compose together information expressed by the leaves.

The central probability specific connective is `Own E`. Fig 3.3 shows the semantics (or *satisfiability relation* in this context). Notice that both the (distributed as) and $\mathbb{E}$ connectives have the semantics of `Own E`, E is $\mathcal{F}$-measurable, with an additional conjunct. This leads to the intuition that a lilac proposition (henceforth `LProp`) must "own" a random variable before saying anything about it. In probabilistic separation logic we interpret ownership of a random variable as enforcing that no other separating conjunct may own a random variable which is not independent. But how do we enforce this using the semantics? Why is ownership `Own E`, measurability E is $\mathcal{F}$-measurable? First let us clarify that we already know that E is a measurable function $RVE \triangle I^{\mathbb{N}} \xrightarrow{\mathrm{m}} A$ so is `Own E` redundant? No. RVE defines measurability w.r.t to $\Sigma_{I^{\mathbb{N}}}$, where $\Sigma_{I^{\mathbb{N}}} \geq \mathcal{F}$, This measurability w.r.t $\mathcal{F}$ is a stronger condition. To see this simply take some function and notice that its preimage on a set in the codomain, if contained in $\mathcal{F}$, must also be in $\Sigma_{I^{\mathbb{N}}}$.

To provide an intuition for why ownership is defined as E is $\mathcal{F}$-measurable, let's consider a scenario where some dimension in the Hilbert cube, n, is used to define a random variable. Take $\omega \in I^{\mathbb{N}} \triangleq$

$\mathbb{N} \to I.$ and $X : RV\mathbb{R} := \lambda\omega \mapsto \omega n$. This projection is a random variable. Now we can study under what measurable spaces of $I^{\mathbb{N}}$ "own X" holds. We split the Hilbert cube at dimension n into 3 chunks: $0 \ldots n-1, n, n+1 \ldots \infty$, and talk about the measurable spaces in each piece. The product measurable spaces of these 3 pieces will form the whole Hilbert cube measurable space again. We do not care at all about what σ-algebra is present in the first and the last chunk, but only the σ-algebra at dimension n matters. If at n we have the trivial σ-algebra, $\{\phi, I\}$, then X is certainly not measurable. In fact, we cannot have any σ-algebra smaller than the borel σ-algebra on I, Σ_I, to validate `Own X`.

And if two probability spaces $(I^{\mathbb{N}}, \mathcal{E}, \mu)$, $(I^{\mathbb{N}}, \mathcal{F}, \nu)$ have the same σ-algebra at dimension n, $\mathcal{E}|_n = \mathcal{F}|_n$, Then the independent product of these two spaces will not exist in general. To see this take for a moment the simpler probability spaces $(I, \Sigma_I, \mu|_n)$ and $(I, \Sigma_I, \nu|_n)$, by focussing our attention on the nth dimension. The independent combination would be $(I, \Sigma_I, \lambda x \mapsto \mu|_n\ x * \nu|_n)$. But the last element of that tuple is not a measure! It would not satisfy the additivity condition on measure that taking the measure of a union of measurable sets is the sum of measure of the sets. The takeaway is that identical σ-algebras cannot form independent combinations, and in this example we illustrated how measurability was unique ownership of a dimension of the Hilbert cube. Measurability over coarser σ-algebra than Σ_I is useful as well: take $Y : RVBool := \lambda\ \omega \mapsto \omega\ n \leq 0.5$. To validate `Own Y`, a much coarser σ-algebra on dimension n than Σ_I would suffice: $\phi, [0, 0.5], [0.5, 1], I$. In general measurability can express complex ownership patterns: single dimensions can be split across multiple separating conjuncts, and multiple dimensions can be mixed.

$$
\begin{aligned}
(\mathcal{F}, \mu) \vDash \mathrm{own}\, E &\iff E \text{ is } \mathcal{F}\text{-measurable} \\
(\mathcal{F}, \mu) \vDash E \sim \mu' &\iff E \text{ is } \mathcal{F}\text{-measurable} \ \wedge\ \mu' = E_{\#}\mu \\
(\mathcal{F}, \mu) \vDash \mathbb{E}[E] = e &\iff E \text{ is } \mathcal{F}\text{-measurable} \ \wedge\ \mathbb{E}_{\omega\sim\mu}[E(\omega)] = e \\
(\mathcal{F}, \mu) \vDash E_1 \stackrel{\circ}{=} E_2 &\iff F \in \mathcal{F} \ \wedge\ \mu(F) = 1 \ \wedge\ F \cup (E_1, E_2)^{-1}(A) \in \mathcal{F} \text{ for all } A \in \mathrm{cod}(E_1) \otimes \mathrm{cod}(E_2), \\
&\quad \text{where } F = \{\omega \mid E_1(\omega) = E_2(\omega)\} \\
(\mathcal{F}, \mu) \vDash \mathrm{wp}(M, X\!:\!A.\, Q) &\iff \text{for all } \mathcal{P}_{\mathrm{frame}} \text{ and } \mu \text{ with } \mathcal{P}_{\mathrm{frame}} \star \mathcal{P} \sqsubseteq (\Sigma_\Omega, \mu) \text{ and all } D_{\mathrm{ext}} : \mathrm{RV}\,[\![\Delta_{\mathrm{ext}}]\!], \\
&\quad \text{there exists } X : \mathrm{RV}\, A \text{ and } \mathcal{P}' \text{ and } \mu' \text{ with } \mathcal{P}_{\mathrm{frame}} \star \mathcal{P}' \sqsubseteq (\Sigma_\Omega, \mu') \text{ such that} \\
&\quad \begin{pmatrix} \omega \leftarrow \mu; \\ v \leftarrow M(\omega); \\ \mathrm{ret}\big(D_{\mathrm{ext}}(\omega), v\big) \end{pmatrix} = \begin{pmatrix} \omega \leftarrow \mu'; \\ \mathrm{ret}\big(D_{\mathrm{ext}}(\omega), X(\omega)\big) \end{pmatrix}, \\
&\quad \text{and } (X), \mathcal{P}' \vDash Q
\end{aligned}
$$

Figure 3.3: Probability Specific Connectives in Lilac

3.2.4 Variables

In order to reason about programs, the logic must be able to express facts about the program effectively. In other words, the language and the logic need to "communicate" somehow. In this section we explain the interface between the language and the logic. Interfaces between different components, namely: language & logic and logic & iris, are important in a formalisation as it guides and motivates choices in how key definitions are made.

Generally speaking a logic expresses propositions about a program in the language by, having access to the program variables through some mechanism. Before we go into this mechanism, there is a novelty in the way Lilac has a different perspective on variables in the language vs the logic. A variable in the language may have the type `x : ty` (Appl type judgement) for some `ty : Ty` (meta language type judgement). One would then expect that if the logic were to construct an `LProp` in which this variable occurs, it would have type `x : ty.den`. However, as we saw above the type of variables in the logic is instead `X : RV ty.den`. The conversion from `ty.den` to `RV ty.den` is a reflection of the perspective taken in probabilistic reasoning. Let's explain with an example. Take

$$x \leftarrow \text{unif } [0,1];\ y \leftarrow \text{unif } [0,1];\ \text{ret } (x,y) \qquad \text{(UNIF2)}$$

$$\{\top\}\ \textsc{unif2}\ \{(X,Y).\ X \sim \text{Unif}[0,1] * Y \sim \text{Unif}[0,1]\}$$

In the program (according to the semantics of bind) x and y are sampled, so they are fixed values. One can consider multiple runs through the program where the values of the samples are different, but locally x and y are deterministic values. This is in contrast with the logic where X and Y are random variables. We consider the full distribution of values x and y could have taken across all *runs* of `UNIF2`.

The link between the language and the logic is established using a connective in the logic, which takes the denotation of a program and some other assertions regarding it. Lilac uses the *weakest precondition* (wp) connective, but let's first have a look at the more intuitive `Hoare triple` connective. It takes 3 arguments, represented syntactically as $\{P\}[\![M]\!]\{X.Q\}$. P (precondition) is an `LProp` and $[\![M]\!]$ is the denotation of some program M. Q is an `LProp` with the (random) variable X free in it. If `M : Ty.G ty` (Appl type judgement), then `X : RV ty.den`. Since Appl is a functional language, the only variable that we have access to in the logic is that representing the outcome of the entire program. Though in practice multiple variables can be accessed by the logic by destructuring a tuple output by the program as in the example of UNIF2.

Todo: how this destructuring works even with RV? (Not important will probably get dropped at the end)

The alternative connective, `wp`, is constructed with the denotation of a program and only one `LProp`, the postcondition. Syntactically: $\text{wp}[\![M]\!]\{X.Q\}$. `wp` is defined to be the weakest precondition on the program M, such that the postcondition Q holds. In a consistently defined semantics the following equivalence constructing a Hoare triple in terms of `wp` holds: $\{P\}[\![M]\!]\{X.Q\} \triangleq P \mathrel{-\!\!*} \text{wp}[\![M]\!]\{X.Q\}$. The advantage of using `wp` is that it is more amenable to automation and since it only takes in a single proposition. It is easier to infer postconditions in proof rules which involve going from one `wp LProp` to another, and makes the prospect of applying these rules easier as less (often no) parameters will need to be explicitly specified. An example of a rule converting between `wp LProp`s is `wp_bind`:

$$\text{wp}([\![M]\!], X.\, \text{wp}([\![N]\!], Y.\, Q)) \ \vdash\ \text{wp}([\![X \leftarrow M;\ N]\!], Y.\, Q)$$

3.3 Instantiating Iris

Theorem proving can be split into two activities: writing definitions and proving theorems. The former doesn't seem difficult or time-consuming. It sounds reasonably easy to come up with a skeleton for the codebase, based on a paper. After a while working on this project I have realised this is not true, and indeed this is well known by the theorem proving community, and is also true for professional mathematics and software engineering. Let's elaborate on the latter. Writing computer code is parallel to writing definitions (of data and algorithms). So definitions are indeed important and time-consuming in Lean as well. There is no parallel to writing theorems though, in conventional software engineering.

A milestone was reached when the definitions of the language and logic were written down, with a set of abstractions to convince myself this development was sound. These definitions were composed of a separate syntax and semantics for both the language and the logic, using dependent de Bruijn indices for both. These definitions would have sufficed to prove formally the correctness of all the proofs in the paper. However, the goal of the project is not only to validate the soundness of the logic, but also to create a usable tool for proving properties of actual probabilistic programs ergonomically (using tactics). The best way to do so is to instantiate the Iris framework. And the interface provided by Iris was not compatible with current definitions. Below we explain why it wasn't compatible and how Iris' interface is motivated.

3.3.1 Deep vs shallow embeddings

When defining a language or logic, within a meta-language (lean in our case), we are presented with 2 choices. A deep embedding; inductively define the syntax, and the denotational semantics becomes an inductive definition over the syntax. For clarity, here is an excerpt from a deep embedding of the Lilac logic:

Todo: complete this (code from c3ca63f), just explain the code and say why it is literally the satisfiability relation, say what HC is

```
inductive LProp : List Ty → List Ty → Type
  | top : LProp ds rs -- ds: deterministic context, rs: random variable context
  | and  : LProp ds rs  → LProp ds rs → LProp ds rs
  | sep  : LProp ds rs  → LProp ds rs → LProp ds rs
  | forall (d : Ty) : LProp (d :: ds) rs → LProp ds rs
  | forall_rv (r : Ty) : LProp ds (r :: rs) → LProp ds rs
  | exists_rv (r : TyRand) : LProp ds (r :: rs) → LProp ds rs
  ...

def LProp.denote {ds : List TyDet} {rs : List TyRand}
    (P : LProp ds rs) (γ : HList (⟦·⟧) ds) (D : RV (List.TProd (⟦·⟧) rs)) (φ : PSp HC) : Prop :=
  match P with
  | top  ⇒ True
  | and P Q ⇒ P.denote γ D φ ∧ Q.denote γ D φ
  | sep P Q ⇒ ∃ φ₁ φ₂ : PSp HC, φ₁ ⋆ φ₂ ≤ some φ ∧ P.denote γ D φ₁ ∧ Q.denote γ D φ₂
  | forall d P ⇒ ∀ x : ⟦d⟧, P.denote (x :: γ) D φ
  | forall_rv r P ⇒ ∀ X : RV ⟦r⟧, P.denote γ (X ; D) φ
  | exists_rv r P ⇒ ∃ X : RV ⟦r⟧, P.denote γ (X ; D) φ
```

Figure 3.4: Excerpt of Deep Embedding of Lilac

On the other hand, a shallow embedding fuses the inductive structure of the syntax and the semantic denotation function together. The denotations of terms in our object language get expressed directly in the meta language, and we lose the ability to distinguish entities in the object language from the meta-language. The previous point, in concrete terms is that we lose the ability to do induction over the object language now. Here is an excerpt of Lilac as a shallow embedding:

The shallow embedding had a much smaller surface and generally allowed the logic to piggyback on the forall and exists quantifiers of the meta language.

3.3.2 BI in Iris

Remember that in Sec 2.3.2 we explained that BI (logic of bunched implications) is just an abstraction which defines the standard connectives shared by every separation logic. Here is the definition of BI as a typeclass in iris-lean (a dependency of this project).

```
/--
Require the definitions of the separation logic connectives and units on a carrier type `PROP`.
-/
class BIBase (PROP : Type u) where
  Entails : PROP → PROP → Prop
  emp : PROP
  pure : Prop → PROP
  and : PROP → PROP → PROP
  or : PROP → PROP → PROP
  imp : PROP → PROP → PROP          -- implies
  sForall : (PROP → Prop) → PROP
  sExists : (PROP → Prop) → PROP
  sep : PROP → PROP → PROP          -- separating conjunct
  wand : PROP → PROP → PROP         -- magic wand
```

```
  persistently : PROP → PROP
  later : PROP → PROP
```

Firstly `Prop` is the universe of propositions, part of Lean's syntax. While `PROP` denotes the proposition in the object logic (Lilac in our case). The typeclass is called `Prop` since there is another typeclass `BI` extending `BIBase` which states the axioms of BI. As explained in 2.3.2, the instantiation of `Entails` can interpret a BI as a separation logic. `Entails` says what it means for a `PROP` to be true by translating into the meta language's `Prop`. `persistently` and `later` are omitted since the former is an orthogonal concern and the latter is irrelevant. `emp` can be understood as the unit of the monoid with which we will instantiate BI. There are several binary operators. The most important connectives are `sForall` and `sExists`. The original definition from iris-rocq is more straightforward:

```
  iForall : ∀ A : Type u', (A → PROP) → PROP
  iExists : ∀ A : Type u',  (A → PROP) → PROP
```

We quantify over the type A of variables that we want to quantify over, since Iris can't know what type of variables a specific logic may want to quantify over, and indeed as in Lilac there may be multiple types (Lilac has two of each quantifier). We have named the above quantifier `iForall`, with `i` short for indexed, since when quantifying over types we refer to the types (like `A`) as *type indices*. `iForall` and `iExists` are impredicative (see 2.4.1), meaning we quantify in universes larger than that of the type being constructed. `u'` is universe level, a natural number. Indeed, impredicativity is required here to provide complete flexibility in what variables an instantiating logic can then quantify over.

Now let's look at `sForall` and `sExists`, with `s` short for *set*. Note that in theorem provers, Sets (say `Set A`) are represented as `A → Prop`. So the signature for the quantifiers says that given a set of `PROP` we construct a `PROP`. The reason for this less intuitive version of quantification is that iForall (iExists) can be defined in terms of sForall (sExists). And the quantified types are not restricted to a predetermined universe `u'`.

3.3.3 First attempt: deeply embedded logic instantiation

There is a result in Lilac [Li et al., 2023, see Appendix B.10] called the substitution lemma which uses induction over the syntax of Lilac propositions. Supposing that induction over the syntax of the logic is important/necessary, a next step is to try to instantiate `BIBase` above with the deep embedding from Fig 3.4. We may try something like so:

```
instance (ds rs: List Ty) : BIBase (LProp ds rs) where
  Entails P Q    := ∀ γ D φ, P.denote γ D φ → Q.denote γ D φ
  and            := LProp.and
  ...
```

Where we simply instantiate each of the constructors with syntax, and call the denotation function in `Entails`. This works nicely until we get to the instantiation of the quantifiers. We now need some syntactic version of `sForall` and not just the `forall/forall_rv` from Fig 3.4. Though the latter is all we need for our logic, instantiating Iris demands the flexibility to quantify over any type, not just the object language types. Let's introduce the following syntax into our language:

```
inductive LProp' : List TyDet → List TyRand → Type
  | sForall : (LProp' ds rs → Prop) → LProp' ds rs
  ...
```

This looks like a valid lean definition, but it is not, since it fails a syntactic requirement of inductive types called *strict positivity*. The type being defined inductively may only appear in *positive* locations, which means it is not allowed to occur in the input of an arrow type. Even if we suppose the `iForall` definition was used in BI, we still end up with nuanced problems with mixing semantics into the syntax because of the quantifiers. We spare further details, but the conclusion is that the impredicativity of quantifiers in `BI` disallows usage of a deep embedding of the logic.

3.3.4 Shallow embedding of the logic

3.4 Deparametrisation of LProp

A major concern throughout the mechanisation was the definition of `LProp`, and how amenable it might be to variable substitution. For example, remember that we were forced to forego the convenience of an induction principle over the lilac connectives (or constructors in this context). This was indeed due to the generality of iris without which we wouldn't be able to use it in the first place. This seemingly insignificant issue tended to have a significant effect on the decisions made. We will first explain why the notion of substitution as defined in the paper, is incompatible with iris.

Substitution is somewhat complicated to think about in Lilac, because random variables are actually functions. At high a level, the takeaway from this section is that when designing a separation logic, we are designing an abstraction. The resource is what we are abstracting over so

- We must not require access to the resource directly for any manipulation at the program logic level.
- `LProp` should only take as argument the entity we abstract over which is the resource.

Both principles were broken by the original formalisation, and led to increasingly convoluted fixes which ultimately led to a dead end. The original formalisation did however use the substitution as defined in the paper at points, so it seemed to be the right path, despite the growing complexity. In retrospect, keeping in mind the above principles, would have led to never needing to consider and partly implement convoluted definitions for `LProp` and the proof rules, and the final definition that we have converged at is far simpler and intuitive.

3.4.1 Lilac substitution

Lilac has two contexts, the random and deterministic contexts. The latter is only required when the logic supports reasoning about conditioning. Remember that conditioning takes random variable and fixes its values, i.e. makes it deterministic. Let us give a concrete example. Here is a proof rule about conditioning (which we do not mechanise in this project):

$$(\underset{x \leftarrow X}{\mathbf{C}}\ \mathbb{E}[E] = e\ ) \ \wedge\ \mathbb{E}[e[X/x]] = v \ \vdash\ \mathbb{E}[E] = v \qquad \text{C-Total-Expectation}$$

It says that taking an expectation $\mathbb{E}[E]$ over all the random variables in E, can be broken down into the taking the expectation first over all the random variable but X, and then taking the expectation of the resultant value just over X. In the standard mathematical notation it is $\mathbb{E}[\mathbb{E}[E \mid X]] = \mathbb{E}[E]$ which is a standard result. What is important to us, is that the conditioning modality $\underset{x \leftarrow X}{\mathbf{C}}$, acts like a binder to a deterministic value, x, akin to $\forall x$. A binder introducing a deterministic value is not present anywhere in Core Lilac, ocurring only in the extended version.

So we will focus on the random contexts. Take the quantifier $\forall_{rv}$:

$$\gamma, D, \mathcal{P} \vDash \forall_{rv} X : A.\ P \qquad \text{iff} \qquad \text{for all } X : \text{RV } A (\gamma, (D, X), \mathcal{P}) \vDash P$$

We are quantifying over RV A, which means variables in Lilac are defined to have this type. Remember RV A means $I^{\mathbb{N}} \xrightarrow{\text{m}} A$ (with additional finite footprint property, not relevant here). Substitution is the process of transforming from one `LProp` to another by specifying a mapping from one variable to another. However, definition of substitution given in the paper looks like this:

$$S \in [\![\Delta']\!] \to [\![\Delta]\!]$$

$$
\begin{aligned}
\top[S] &= \top \\
\bot[S] &= \bot \\
(P \wedge Q)[S] &= (P[S] \wedge Q[S]) \\
(P \vee Q)[S] &= (P[S] \vee Q[S]) \\
(P \to Q)[S] &= (P[S] \to Q[S]) \\
(P * Q)[S] &= (P[S] * Q[S]) \\
(P -\!\!* Q)[S] &= (P[S] -\!\!* Q[S]) \\
(\forall_{\mathrm{rv}} X : A.\, P)[S] &= \forall_{\mathrm{rv}} X : A.\, P[S \times 1_A] \\
(\exists_{\mathrm{rv}} X : A.\, P)[S] &= \exists_{\mathrm{rv}} X : A.\, P[S \times 1_A] \\
&\ldots
\end{aligned}
$$

It is helpful to have the following mental picture of the meaning of an `LProp` as a function $I^{\mathbb{N}} \xrightarrow{\mathrm{m}} [\![\Delta]\!] \xrightarrow{\mathrm{m}} A$, pick an element ω of $I^{\mathbb{N}}$ (which is like sampling a random number). Once we do this the randomness is gone and the maps forward are deterministic. ω maps to values of variables, $[\![\Delta]\!]$, as an intermediate step. Then these variables deterministically map to some expression of type A. With this in mind note that there's a mismatch in what we would like substitution to be, replacing $X : I^{\mathbb{N}} \xrightarrow{\mathrm{m}} A$ with $Y : I^{\mathbb{N}} \xrightarrow{\mathrm{m}} A$ which is used in an `LProp`, with the above definition of substitution, which only starts at the intermediate $[\![\Delta]\!]$ step.

At a high level the Iris proof mode (indeed any proof mode) works by splitting an `LProp` into a context of premises and variables, and a goal. The variables are of type RV A and if the substitutions are not with RV A as domain, the types just won't line up.

3.4.2 Definitions of LProp, random variables and `wp` rules

The definition of `LProp` after moving to a shallow embedding had the signature[2]:

$$Lprop := [\![\Gamma]\!] \to (I^{\mathbb{N}} \xrightarrow{\mathrm{m}} [\![\Delta]\!]) \to (I, \Sigma_{I^{\mathbb{N}}}, \mu) \to Prop$$

In lean that would be:

```
def LProp {ds : List Ty} {rs : List Ty} :=
  (γ : HList (⟪·⟫) ds) → (D : RV (List.TProd (⟪·⟫) rs)) → (φ : PSp HC) → Prop
```

As noted before $[\![\Delta]\!]$ is never used without the conditioning modality, so we omit it here. We have also chosen to omit the monotonicity of `LProp`, w.r.t to the ordering on resources, as that is irrelevant here. Subsequent presentation of connectives constructing `LProp`, will be suitably modified.

```
def LProp {rs : List Ty} := (D : RV (List.TProd (⟪·⟫) rs)) → (φ : PSp HC) → Prop
```

Remember that for each `r` in `rs`, `⟪r⟫` is the denotation of a type which is a type equipped with a measurable space. And that `List.TProd (⟪·⟫) rs` is the product measurable space of all the `r` denotations.

According to the original definition of variables in the logic, which we denoted E showing up prominently in probability specific proof rules own$E$, $E \sim \mu$ and $E_1 \stackrel{\circ}{=} E_2$:

$E : [\![\Delta]\!] \xrightarrow{\mathrm{m}} A$

Even at this stage we can see problem coming (in retrospect) because the forall quantifiers of the logic use a different notion of variables $E : \mathrm{RV}\ A$, which is the point of interface with iris, so the iris will also use $E : \mathrm{RV}\ A$. Compare the definition of `own` with that of `forall_rv` in the logic:

```
def own {ty : Ty} (E : List.TProd (⟪·⟫) rs -m→ ⟪ty⟫) : LProp rs :=
  ⟨fun D Ω ↦ Measurable[Ω.ms] (E ∘ D), ...⟩

def forall_rv (ty : Ty) (P : LProp (ty :: rs)) : LProp rs :=
```

[2]The code in this section is obtained from an older commit https://github.com/edwin1729/bppl-logic/tree/cd19b5ff454abaf05f959ba8abfe0961518d02d3

```
  fun D Q ↦ ∀ X : RV ⟪ty⟫, P.1 (X ; D) Q
```

In `own`, E starts from context `List.TProd (⟪·⟫) rs`, while `forall_rv`, starts earlier from $(I^{\mathbb{N}}, \Sigma_{I^{\mathbb{N}}})$, both mapping to `⟪ty⟫`. This is the first problem with our definition. This is the problem which turns out to have no workaround. Let's suppose that we need to go back and forth between one representation of variables to another. Given `E : List.TProd (⟪·⟫) rs -m→ ⟪ty⟫` and `D : HC -m→ (List.TProd (⟪·⟫) rs)` we can simply compose them to get `E ∘ D : HC -m→ ⟪ty⟫`. Note that we only have access to `D` explicitly like this when working outside the abstraction of the logic, i.e. in soundness proofs but not in proofs within the lilac logic. Going the other way is not possible, and that's the irreparable issue.

There's also a 2nd problem due only to my choice in formalisation: quantification takes an `LProp` with parameters `ty :: rs` and shrinks the context to `rs`. This makes sense, since under a forall quantification, there is one more free variable than there is outside. The problem is that while reasoning about a single program we jump between multiple types (`LProp (ty :: rs)` to `LProp (rs)`).

But the iris type signatures for connectives which remember looked like:

```
class BIBase (PROP : Type u) where
  and : PROP → PROP → PROP
  ...
```

generally had connectives where the types of the arguments and output were all the same. This is something to keep in mind as a difficulty though not an insurmountable problem as we can work around with some bookkeeping. But as the name of this section, "Deparametrisation" suggests, ultimately we remove this `rs`, parameter, the need for which vanishes when we uniformly use $E : \mathrm{RV}\ A$ everywhere, which massively simplifies the system.

To show how exactly this set of definitions is irreparably flawed, we try to verify the simplest specification possible, Unif1

$$x \leftarrow \mathrm{unif}\ [0,1];\ \mathrm{ret}\ x \qquad (\textsc{Unif1})$$

$$\{\top\}\ \textsc{Unif1}\ \{X.\ X \sim \mathrm{Unif}[0,1]\}$$

To prove this, we will require 3 rules: `wp_bind`, `wp_unif` and `wp_ret`

$$\textsc{W-Ret}\quad Q[[\![M]\!]/X] \vdash \mathrm{wp}([\![\mathrm{ret}\ M]\!], X.\, Q) \qquad \textsc{W-Bind}\quad \mathrm{wp}([\![M]\!], X.\, \mathrm{wp}([\![N]\!], Y.\, Q)) \ \vdash\ \mathrm{wp}([\![X \leftarrow M;\ N]\!], Y.\, Q)$$

$$\textsc{W-Unif}\quad (\forall_{\mathrm{rv}}\, X : A.\ X \sim \mathrm{Unif}[0,1] \mathrel{-\!\!*} Q) \ \vdash\ \mathrm{wp}([\![\mathrm{unif}\ [0,1]]\!], X : A.\, Q)$$

We can see substitution being used in `wp_ret`. Let's go step by step through the proof showing the Lean definition of each of the 3 proof rules, and the iris proof states after each rule is applied. Below are the definitions of the proof goal and the proof in iris upto where no more progress can be made

```
abbrev unif1 : Term [] Ty.real.G :=
  unif01.bind (
  ret (var head)
)

/-- Variable at the head of the random environment list. -/
def fst_rv : ValRand ds (A::rs) ⟪A⟫ :=
  ⟨fun r_env ↦ r_env.get .head , List.TProd.measurable_get .head⟩
```

```
lemma unif1_prop : ⊢ (wp unif1.den iprop(fst_rv ~ (WP.unif01_sem)) : LProp [] []) := ...

abbrev post1 : LProp [] [] := wp unif1.den iprop((fst_rv) ~ WP.unif01_sem)

theorem unif1_spec : iprop(⊢ post1) := by
  iapply wp_bind
  iapply wp_unif
  iintro %x
  irewrite ⊢ substLProp (wp ⟦ret (var head)⟧ (dropSnd iprop(fst_rv ~ unif01_sem))) x
  change ⊢ substLProp (fst_rv ~ unif01_sem) x -∗
           substLProp (wp ⟦ret (var head)⟧ (dropSnd (fst_rv ~ unif01_sem))) x
  iintro H
```

Step 1 : `wp_bind`

Our proof state starts like this:

```
⊢
⊢ wp ⟦unif1⟧ (fst_rv ~ unif01_sem)
```

note that `fst_rv` is picking out the first element of the random variable context which is at the moment a singleton list $[X] = X;;[]$. So it's just picking out `X` if X is the outcome of running `unif1`. `unif01_sem` stands for unif01 semantics and is simply a uniform measure on the unit interval $[0, 1]$.

Firstly we apply `wp_bind`, defined in Lean as:

```
lemma wp_bind (Q : LProp ds (B::rs)) (M : Term rs A.G) (N : Term (A::rs) B.G) :
    wp ⟦M⟧ (wp ⟦N⟧ (dropSnd Q)) ⊢ wp ⟦M.bind N⟧ Q := ⟨soundness proof⟩
```

Notice the usage of `dropSnd` defined as:

```
abbrev dropSnd (P : LProp ds (r :: rs)) : LProp ds (r :: r' :: rs) :=
  fun D Ω ↦ P.1 (⟨fun (x, _y, z) ⇒ (x,z) , measurable_dropSnd⟩ ∘ₘ D) Ω
```

This is because the `Q` within a `wp`, receives an extra argument in the context (see 3.2). The `Q` is the same though between the left and right hand sides of the $\vdash$. So we introduce this `dropSnd` casting function which converts `Q` into an `LProp` taking a bigger context. Internally a measurable map `⟨fun (x, _y, z) ⇒ (x,z) , measurable_dropSnd⟩` is applied on the bigger context `D`, to drop the 2nd variable and pass that to `Q`. Why are we dropping the 2nd variable (not 1st)? The context that Q recieves on the lhs of `wp_bind` looks like $[Y;;\ldots]$ while on the rhs `(dropSnd Q)` will receive a context like $[Y;;X;;\ldots]$. `Q` does not have access to X on the lhs which is why it is dropped, on the rhs. As we can see we have to rather tedious and unintuitive bookkeeping with such "casting" functions to make the types match, if `LProp` is parametrised.

Now our state looks like this:

```
⊢
⊢ wp ⟦unif01⟧ (wp ⟦ret (var head)⟧ (dropSnd (fst_rv ~ unif01_sem)))
```

Step 2: `wp_unif` Now we are in position to apply the `wp_unif` rule, given in lean as:

```
lemma wp_unif (Q : \texttt{LProp} ds (Ty.real :: rs)) :
    forall_rv Ty.real (fst_rv ~ unif01_sem -∗ Q) ⊢ wp ⟪Term.unif01⟫ Q := ⟨soundness proof⟩
```

This is a pretty direct translation of the informal law. A `wp` `LProp` running just the `unif01` program can be turned into an `LProp` which uses the `forall_rv` and $-\!\!*$ connectives. Both `wp` and `forall_rv` introduce a variable into the random context. `wp` says that the variable is introduced as the outcome of the program `unif01`. The right hand side says, that is the same as requiring that some arbitrary random variable is distributed according to `unif01_sem`.

Applying this we obtain the proof state:

```
⊢
⊢ forall_rv Ty.real (fst_rv ~ unif01_sem -∗
    (wp ⟦ret (var head)⟧ (dropSnd (fst_rv ~ unif01_sem))))
```

This is an interesting situation where both the left and right hand side of the $-\!\!*$ contain the `LProp (fst_rv ~ unif01_sem)`. However, remember we are working with de Bruijn indices so `fst_rv` does not always refer to the same variable. `fst_rv` always refers to the nearest variable introduction, which on the right hand side comes from `wp` not `forall_rv`. So on the rhs we refer to a different variable obtain from running `ret (var head)`, but with more application of proof rules, we should hopefully be able to show that these two variables are in fact equal.

Step 3: iintro Now we would like to apply the intro tactic of iris called `iintro`. In general, an intro tactic can pull the input of a forall quantification or implication in the goal into the premise in the proof state (Remember that both forall quantification and implication are exactly the same: they are both dependent products 2.4.1). The `iintro` tactic mimics this acting on iris' $\forall$ (`iForall` or `sForall`), $-\!\!*$ and $\to$. But the goal we have uses our custom `forall_rv` connective. If we want to use Iris' proof mode, we will need to convert this `forall_rv` to `iForall`. Iris does in fact provide extensible tactics, i.e. `iintro` can be extended to work on other custom connectives like `forall_rv`. This was the original approach, which adds another level of indirection to this presentation. To make our presentation more understandable let's remove that level of indirection and use `iForall` directly in the definition of `wp_unif` and never use `forall_rv` in the first place. Here's an adapted definition for `wp_unif`

```
lemma wp_unif (Q : LProp ds (Ty.real :: rs)) :
  iprop(∀ (X : RV ⟪Ty.real⟫), substLProp (fst_rv ~ unif01_sem-* Q) X) ⊢ wp ⦃Term.unif01⦄ Q := ...
```

We added the `iprop` syntax specifier here to make it clear that what it wraps is an `LProp`, specifically $\forall$ is overloaded and here refers to iris notation `iForall : (α → LProp rs) → LProp rs`. The second interesting thing is a direct consequence of the signature of `iForall`: it doesn't know to allow a larger context underneath it (we would have liked to have `iForall : (α → LProp (r::rs)) → LProp rs`). So we again resort to manual book-keeping with substLProp defined as follows:

```
abbrev substLProp (P : LProp (A::rs)) (X : RV ⟪A⟫) : LProp rs :=
  fun D Ω ↦ P.1 (X ;; D) Ω
```

There is nothing unexpected in this definition

The `;;` syntax is supposed to look just like the cons operator, but remember that `X` and `D` are actually measurable functions to a value and list respectively. It is unimportant but here is the definition for completeness:

```
def fun_prod (f : α -m→ β) (g : α -m→ γ) : α -m→ β × γ :=
  ⟨fun a ↦ (f a, g a), Measurable.prod f.2 g.2⟩
```

If we apply our modified `wp_unif`, instead of the original version in **step 3**, we will instead get this proof state after application:

```
⊢
⊢ ∀ X, substLProp (fst_rv ~ unif01_sem -*
    (wp ⟦ret (var head)⟧ (dropSnd (fst_rv ~ unif01_sem)))) X
```

And after the next tactic `iintro %x`, we get the slightly different proof state where for the first time we are actually using the proof mode in a non-trivial manner!

```
x : RV ⟪Ty.real⟫
⊢
⊢ substLProp (fst_rv ~ unif01_sem -* (wp ⟦ret (var head)⟧ (dropSnd (fst_rv ~ unif01_sem)))) x
```

We see that our goal now has `substLProp` at the outermost level instead of an SL connective, so we can't apply any proof rules. The intuition here is that thinking of an `LProp` as constructed using a tree of connectives, we should be able to just push the `substLProp` down to the leaves while applying the substitutions as encountered. Succinctly `substLProp` is just a fold over a tree. Unfortunately, this tree is not an inductive data structure (in other words it not a deep embedding but a shallow one). There are workarounds to this use the typeclass mechanism which can be made to emulate induction over shallow embeddings. But all of this is not the actual problem! We realise that the base case of this fold, the action of `substLProp` on the leaves, is not definable. Since we need to convert between the two type signatures of variables in the impossible direction in order to achieve the substitution on leaves.

Let's return to our concrete example. We push the substLProp, through the $-\!\!*$ connective, with a rewrite tactic. This is the `change` in our proof. After this and `iintro H` we have the state:

```
x : RV ⟪Ty.real⟫
⊢ ∗H : substLProp (fst_rv ~ unif01_sem) x
⊢ substLProp (wp ⟦ret (var head)⟧ (dropSnd (fst_rv ~ unif01_sem))) x
```

Let's focus on `∗H : substLProp (fst_rv ~ unif01_sem) x`. Now we have exactly a substLProp on a "leaf" connective $E \sim \mu$. Conceptually `x` and `fst_rv` are the same variable. However `x : HC -m> ℝ` and `fst_rv : (List.TProd (⟪·⟫) rs) -m> ℝ` and we can't apply the substitution to remove the substLProp and make $\sim$ the outermost connective. If we can't do that then we can't reason using the proof rules of Lilac. In the proof mode, we have hit a dead end, and the only way to proceed is descend back into the meta logic. The separation logic is supposed to provide a higher level of abstraction but clearly our instantiation of Iris was flawed as we can't stay in the abstraction and manage to prove the simplest objective.

3.4.3 ignore

The whole point of the variable contexts was supposed to be that, it will allow substitution: we want to begin with some $(I^{\mathbb{N}}, \Sigma_{I^{\mathbb{N}}} \xrightarrow{\text{m}} [\![\Delta]\!])$ and turn it into $(I^{\mathbb{N}}, \Sigma_{I^{\mathbb{N}}} \xrightarrow{\text{m}} [\![\Delta']\!])$, where in Δ' one of the variables would have changed. As stated in the paper, we can specify a substitution as a measurable map $[\![\Delta]\!] \xrightarrow{\text{m}} [\![\Delta']\!]$. The difficulty is the measurability of this map. We can't simply map an index in the list from one value to another, because of the way $[\![\Delta]\!]$ is defined in lean as `List.TProd (⟪·⟫) rs`, unfolding `TProd` gives `rs.foldr (fun r β ⇒ ⟪r⟫ × β) PUnit`. A measurable space is also getting constructed as the product type of the denotations is iteratively taken. It's possible to define a measurable function which may peel off the head, or cons a new element, or both, but creating a measurable map works deeper inside the list, and then proving properties of the then rather complicated function is not attractive. On studying Lilac further it looked as if we only really need to manipulate the head of the list, so this opens up the possibility of not using the dependent de Bruijn indices method (mapping from a list), which is what we opt for in the end.

saying that the random variable obtained by sampling the program $[\![retM]\!]$ is just always the denotation of M, $[\![M]\!]$. $[\![M]\!]$ substitutes the variables X which is free in the `LProp` Q, so substitution is essentially removal of free variables. Initial attempts at defining *wp_ret* tried to adapt the proposed at the $[\![\Delta]\!]$ level, but this led to a painfully convoluted system before finally hitting a dead end with this approach. We try to outline the old design with the old definitions of `LProp`. At a high level, the above notion of substitution can be made to work

The development got quite far with the `LProp` being parametrised with the type of the deterministic context. However, that started causing serious issues when stating the proof rules, because of the book-keeping that needs to be done with always keeping track of the number of variables in context. Not to mention this bookkeeping is quite difficult since we always need to keep proving that the the contexts are a measurableSpace.

3.5 New definitions

In the previous two sections we have motivated the need for two key changes to the original attempt at defining the logic Lilac. Here we present the definitions as they currently are on the latest version of the Lean code. We also comment on the apparent redundancy of the `congruence` *proof rule, and speculate on the suitability of these definitions for the conditioning modality which is future work*

Chapter 4

Mechanising Lilac II: Proofs

Having overcome the fundamental change in the definitions, consolidate inconsistencies and instantiating Iris, we present here the 2nd aspect of a mechanisation project: A LOT of theorem proving. We present the soundness proofs of Lilac proof rules drawing attention to 2 of them in particular: `wp_bind` *and* `wp_unif`*, due to each of them influencing the definition of the* `wp` *connective (in order to validate their soundness).* `wp_bind` *motivated the use of Mathlib's Probability Kernels, with kernel style definitions and proofs.* `wp_unif` *the central and most complex proof in the entire project, motivates the creation of an entire sub-library of measure theoretic results about measurable spaces obtained through finite footprint on the Hilbert Cube.*

Todo: Proof irrelevance for closed_krm_subtype also for conditioning Todo: Pi-lambda theorem not suitable for BaSL

4.1 A Tour of the `wp` Connective

Before we proceed with having a detailed look at any `wp` proof rule in particular, we first need to gain a strong understanding of the definition of the `wp` connective. We restate and motivate the definition of the `wp` connective in this section[1]:

$$
\begin{aligned}
(\mathcal{F},\mu) \vDash \mathrm{wp}(M, X\!:\!A.\, Q) \iff\ & \text{for all } \mathcal{P}_{\mathrm{frame}} \text{ and } \mu \text{ with } \mathcal{P}_{\mathrm{frame}} \star \mathcal{P} \sqsubseteq (\Sigma_{I^{\mathbb{N}}}, \mu) \text{ and all } D_{\mathrm{ext}} : \mathrm{RV}\,[\![\Delta_{\mathrm{ext}}]\!], \\
& \text{there exists } X : \mathrm{RV}\,A \text{ and } \mathcal{P}' \text{ and } \mu' \text{ with } \mathcal{P}_{\mathrm{frame}} \star \mathcal{P}' \sqsubseteq (\Sigma_{I^{\mathbb{N}}}, \mu') \text{ such that} \\
& \begin{pmatrix} \omega \leftarrow \mu; \\ v \leftarrow M(\omega); \\ \mathrm{ret}\big(D_{\mathrm{ext}}(\omega), v\big) \end{pmatrix} = \begin{pmatrix} \omega \leftarrow \mu'; \\ \mathrm{ret}\big(D_{\mathrm{ext}}(\omega), X(\omega)\big) \end{pmatrix}, \\
& \text{and } \mathcal{P}' \vDash Q[X]
\end{aligned}
$$

The `wp` connective is the most crucial and intricate part of the semantics of Lilac. At the heart of the definition is the requirement that running a program with $\mathcal{P}$ as the resource, will cause a modification of the resource to $\mathcal{P}'$, such that 1) there are no unnecessary changes to $\mathcal{P}$, and 2) $\mathcal{P}'$ is able to generate the output of the program M. These clauses are captured respectively, by the

[1] This is identical to the version presented before in Sec 3.2. However, in both place we have made an omission of a redundant part of the original definition. Particularly the do-notation based equation, returns a triple rather than a tuple on both sides of the equation. The component we have omitted, $D(\omega)$ can always be obtained by our choice in instantiation of D_ext and is hence made redundant.

$$\frac{P \vdash Q}{\mathrm{wp}(M, X, P) \vdash \mathrm{wp}(M, X, Q)}\ \textsc{wp-consequence}$$

$$\frac{X \notin F}{F * \mathrm{wp}(M, X, Q) \vdash \mathrm{wp}(M, X, F * Q)}\ \textsc{wp-frame}$$

$$\mathrm{wp}(M, X, P) \vee \mathrm{wp}(M, X, Q) \vdash \mathrm{wp}(M, X, P \vee Q)\ \textsc{wp-disjunction}$$

$$Q[\![M]\!/X] \vdash \mathrm{wp}([\![\mathrm{ret}\ M]\!], X, Q)\ \textsc{wp-ret}$$

$$\mathrm{wp}([\![M]\!], X, \mathrm{wp}([\![N]\!], Y, Q)) \vdash \mathrm{wp}([\![X \leftarrow M;\ N]\!], Y, Q)\ \textsc{wp-bind}$$

$$(\forall\, X : \mathrm{RV}\ \mathbb{R}.\ X \sim \mathrm{Unif}[0,1] \mathrel{-\!\!*} Q) \vdash \mathrm{wp}([\![\mathrm{unif}\,[0,1]]\!], X : A, Q)\ \textsc{wp-unif}$$

$$(\forall\, X : \mathrm{RV\ Bool}.\ X \sim \mathrm{Ber}\, p \mathrel{-\!\!*} Q) \vdash \mathrm{wp}([\![\mathrm{flip}\ p]\!], X : A, Q)\ \textsc{wp-flip}$$

$$\mathrm{wp}([\![M]\!], X : A.\, \mathrm{wp}([\![N]\!], Y : A.\, Q(\text{if } E \text{ then } X \text{ else } Y))) \vdash \mathrm{wp}([\![\text{if } E\ M\ N]\!], X : A.\, Q)\ \textsc{wp-if}$$

$$I(1, e) * (\forall i : \mathbb{N}.\ \forall_{rv}\, X : A.\ \{I(i, X)\}\ M\ \{X'.\, I(i+1, X')\}) \vdash \mathrm{wp}([\![\mathrm{for}(n, e, i\, X.\, M)]\!], X : A.\, I(n+1, X))\ \textsc{wp-for}$$

Figure 4.1: `wp` proof rules of Lilac

following splitting:

$$\begin{pmatrix} \omega \leftarrow \mu; \\ \mathrm{ret}\big(D_{\mathrm{ext}}(\omega)\big) \end{pmatrix} = \begin{pmatrix} \omega \leftarrow \mu'; \\ \mathrm{ret}\big(D_{\mathrm{ext}}(\omega)\big) \end{pmatrix} \qquad \begin{pmatrix} \omega \leftarrow \mu; \\ M(\omega) \end{pmatrix} = \begin{pmatrix} \omega \leftarrow \mu'; \\ \mathrm{ret}\big(X(\omega)\big) \end{pmatrix} \tag{4.1}$$

$$(D_{\mathrm{ext}})_{\#}\mu = (D_{\mathrm{ext}})_{\#}\mu' \qquad \mu \gg\!= M = X_{\#}\mu' \tag{4.2}$$

Firstly we see that neither measure associated with the resources, $\mathcal{P}.\mu$ or $\mathcal{P}'.\mu$ get used directly. Instead, we use μ and μ' respectively, which is any larger measure on the Borel σ-algebra on the Hilbert cube. On the left, we see that if we choose arbitrarily some random variable D_{ext} we can accommodate for the random variable to be unchanged, with our choice of μ'[2]. This is in line with "no unnecessary changes". On the right we can see that the new measure μ' essentially encodes composition of the old resource and the program.

We will see why D_{ext} is required when explaining the soundness proof for the `wp_bind` rule (Sec 4.2). We notice that μ' is barely constrained; it is existentially quantified and only needs to satisfy $\mathcal{P}' \sqsubseteq (\Sigma_{I^{\mathbb{N}}}, \mu')$. What is $\mathcal{P}'$ constrained by? The clause $\mathcal{P}' \vDash Q$. $\mathcal{P}'$ must satisfy Q which having access to X will likely make assertions about it.

We have not yet mentioned $\mathcal{P}_{\mathrm{frame}}$. This aspect of the semantics is in line with what is standard in separation logic to establish the frame rule, `wp_frame`. Fig 4.1 lists all the `wp` of Lilac. We have proven the soundness of these rules in Lean. However, we cannot go through all of them in this report.

We conclude this section with a translation of `wp` to Lean:

```
def wp (M : RV (⟪Ty.G A⟫)) (Q : RV ⟪A⟫ → LProp) : LProp :=
  fun Ω ↦
  ∀ Ω_fr : PSp, ∀ Ω_pre : ✓'(Ω_fr ⋆ Ω), ∀ μ,
    (↓Ω_pre).1 ≤ PSpace.mk' μ → ∀ {α: Type} {msα : MeasurableSpace α} (D_ext : RV α),
  ∃ X : RV ⟪A⟫, ∃ Ω' : PSp, ∃ Ω_post: ✓'(Ω_fr ⋆ Ω'), ∃ μ', (↓Ω_post).1 ≤ PSpace.mk' μ' ∧
  (toMK M ×ₖ det D_ext) ∘ₘ μ = (det X ×ₖ det D_ext) ∘ₘ μ' ∧
  (Q X).1 Ω'
```

[2]We are not saying that every random variable will remain unchanged, the ordering of the quantifiers in the semantics of `wp` is crucial

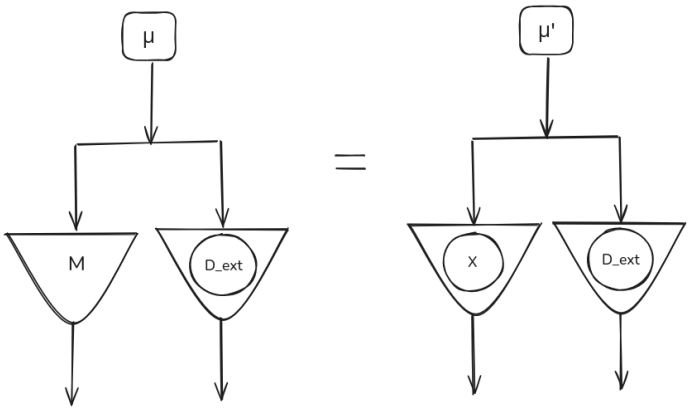


Figure 4.2: Equational reasoning in definition of `wp`

We see quite a lot of new notation. Firstly, $\checkmark'$ is notation for `Option.is_some : Option a → Bool`, stating for example that the independent combination `Ω_fr ⋆ Ω` exists. $\downarrow$ denotes unwrapping of the value within an `Option` when we know it is `some`. We see that the choice of random variable `D_ext` can be to any $(\alpha, ms\alpha)$. As suggested in Equations 4.1 and 4.2, the large "do-notation" based equation from the informal definition of the `wp` connective can be converted to point-free form. In the lean code too we have a point free formulation of the equation, which essentially merges the two point free forms above. This style is adopted since it uses the "kernel API", Mathlib's preferred style [Degenne, 2025] for expressing these kinds of equations. The "kernel API" has vast library of lemmas, and one can imagine how using a point free style facilitates prerequisites for applying a lemma to be more easily inferred (less parameters less things to infer), by the workhorse of type-class inference [Spitters and van der Weegen, 2011]. Again we explain the notation. We have merged the two equations from Eq 4.2, using the $\times_k = \text{Kernel.prod} : \text{Kernel}\,\alpha\,\beta \to \text{Kernel}\,\alpha\,\gamma \to \text{Kernel}\,\alpha\,(\beta \times \gamma)$. Remember that a kernel is defined $\text{Kernel}\,\alpha\,\beta := \alpha \xrightarrow{\text{m}} \mathcal{G}\,\beta$. M has type isomorphic to that of a Kernel[3]. `det` makes a RV $\langle\!\langle A \rangle\!\rangle$ into a Kernel. Remember that RV is a measurable function, and a measurable function $f : \alpha \xrightarrow{\text{m}} \beta$ can be converted to a kernel by lifting the output into the corresponding Dirac measure: $\lambda x \mapsto \text{dirac}\,(f\,x) : \text{Kernel}\,\alpha\,\beta$. Let us expand our diagrammatic representation to include both `det` and $\times_k$. The latter, doesn't have any special shape associated with it and is just a splitting of wires. The former we denote with an oval inside the kernel symbol (which is the triangle, see Sec 2.2). We present in Fig 4.2 the equation in the definition of `wp` that we have just discussed. We modify our notation to be vertical instead of horizontal in preparation for writing the much larger equational reasoning arguments we will be seeing in Sec 4.4.

4.2 wp_bind

This is our first presentation of the soundness of a `wp` proof rule. We will focus on getting the intuition across to the reader, without getting weighed down by the technicalities of the Lean code. `wp_bind` is chosen since it prominently features a different style of proof that is significant in many of the soundness proofs of `wp` rules. Specifically, it employs equational reasoning on kernels, and Mathlib's kernel "API", of lemmas allows us to stay at the level of abstraction of kernels without going down to the integrals in this proof. This is as opposed to some of the other proofs, prominently `wp_unif` and `wp_flip`, which are dominated by measure theoretic proof obligations. We can understand the former as being more familiar to the computer scientist and the latter falling more into the mathematician's plate. We employ the diagrammatic notation again, for a remarkably easy to understand proof of `wp_bind`, translated into this point-free style from the original proof in [Li et al., 2023].

[3] `toMK` is a manual coercion to a kernel, which is required since denotation of Appl programs are not in general kernels, only those with Appl type `Ty.g ty`

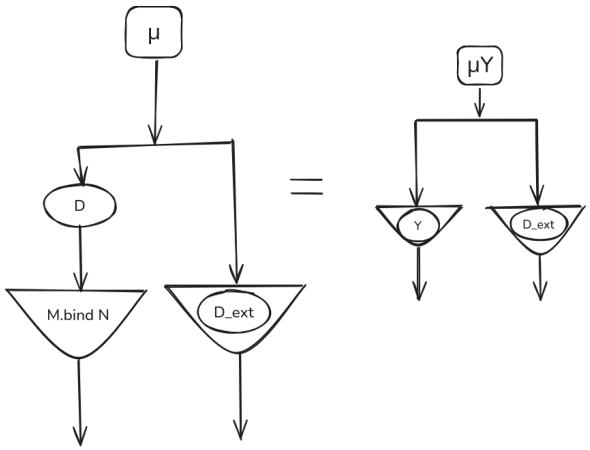
Recall that `wp_bind` is defined:

$$\text{wp}([\![M]\!], X, \text{wp}([\![N]\!], Y, Q)) \vdash \text{wp}([\![X \leftarrow M;\ N]\!], Y, Q) \qquad \text{WP-BIND}$$

where we have colour-coded the premise with the outer `wp`, and inner `wp` each yielding an equation which we will use in establishing the goal. Recalling Sec 4.1 in a nutshell, given a μ and program M, a `wp` gives back a μ' and random variable X (where the μ and μ' are each associated with the old and new resource, $\mathcal{P}$ and $\mathcal{P}'$ respectively). This corresponds to the Lean code:

```
theorem wp_bind (Q : RV ⟪B⟫ → LProp) (D : RV (TProd (⟪·⟫) rs))
    (M : Term rs A.G) (N : Term (A::rs) B.G)
    : wp (M.den ∘ᵣ D) (fun X ↦ (wp (N.den ∘ᵣ (X ;; D)) Q))
      ⊢ wp ((M.bind N).den ∘ᵣ D) Q := by
```

Our goal is to prove:



We first note that we need to construct both a μY (a more sensible name than the usual μ') and Y to satisfy the proof obligation. We notice some new notation, an oval outside a triangle?. This represents a measurable function, in our case D, in a random variable context. D is implicit in the informal presentation but is explicit in the Lean code. We bring clarity through the types denoting, noting that $\circ_r$ is just straightforward function composition. In Lean this composition operation is `Kernel.comap`.

$$\text{(M.bindN).den} : \text{Kernel}\,(\text{TProd}\,\langle\!\langle\cdot\rangle\!\rangle\,\text{rs})\,\langle\!\langle B\rangle\!\rangle$$
$$\text{(M.bindN).den} \circ_r D : \text{Kernel}\,I^{\mathbb{N}}\,\langle\!\langle B\rangle\!\rangle$$

Continuing with the proof, we feed μ into the outer `wp`, to obtain a μX and X such this equation holds:

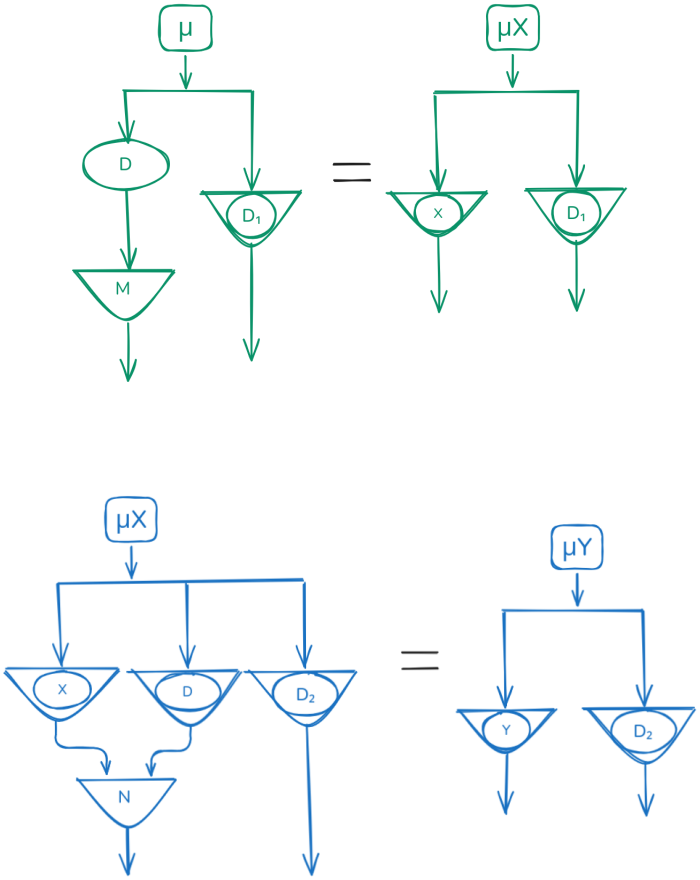
Notice that since `D_ext` is universally quantified for `wp LProp`, we rename to D_1 here to avoid name clashes. The above result can be used with any D_1 instantiation.

From the outer `wp` we will obtain that the resource associated with μX, satisfies the inner `wp`. So feeding μX to the inner `wp`, further provides μY and Y such that this equation holds:

Notice that it seems like N is receiving two inputs but that is just the usual random variable context, extending with the X obtained from running M. We put everything together to achieve the goal (given above in black) in Fig 4.3.

Notice that we refer to application of the premises in our proof using their respective colours. When applying outer `wp`, we instantiate $D_1 =$ `D;;D_ext`, and when applying inner `wp` we instantiate $D_1 =$ `D_ext`.

We conclude with some commentary of how the connection between `wp_bind` with `wp_unif` and `wp_flip` affected the process of mechanisation in practice. Note that we actually prove `wp_meas`, a



generalisation of `wp_unif` and `wp_flip` that we will refer to from here. It is explained immediately in the next section 4.3. Unlike `wp_meas`, `wp_bind` is a standard rule in any weakest precondition based logic. However, it is deeply intertwined with the semantics of `wp`, adding the extra bookkeeping of `D_ext` to the definition of the `wp` connective. Indeed `wp_unif` and `wp_bind` can be understood as reinforcing each other's difficulty! During the mechnaisation we approached these two proofs in an interleaved fashion. We first proved soundness of `wp_unif`, whilst redefining `wp` to remove all occurrences of `D_ext`. Reducing the complexity of `wp` brought the soundness proof of `wp_unif` within approachable distance, so it was deemed necessary, even if we needed to add `D_ext` back in later and rework the proof. Then on attempting to prove `wp_bind` and realising why it is unprovable without `D_ext`, `D_ext` was added back in. Then `wp_unif`'s proof obligation got a little harder but most of the measure theory lemmas where either already there, could be generalised or adapted for the new proof obligations. `wp_unif` and $\wp$ both proofs completed.

4.3 Finite Footprints

When it comes to the proof rules of Lilac, the probability-specific reasoning that can occur in the logic is predominantly present in two rules `wp_unif` and `wp_flip`, which reason about two syntactic constructs from Appl, `unif01` (uniform distribution on unit interval) and `flip` (bernoulli distribution) respectively. There is nothing special about these two distributions, and we can indeed have a general rule `wp_meas`, parametric over any distribution. For this suppose we define a new Appl term, we also introduce a construct for defining a general distribution, like so:

$$(\forall\, X : \mathrm{RV}\ A.\ X \sim \mu \mathrel{-\!\!*} Q) \ \vdash\ \mathrm{wp}(\lambda\ \_ \mapsto \mu)(\lambda X : A.\ \mapsto Q) \qquad \text{(WP-MEAS)}$$

where μ is an arbitrary probability measure. We are specifying a denotation directly instead of taking the denotation of some concrete program. Note that $(\lambda\ \_ \mapsto \mu) : A \xrightarrow{\mathrm{m}} \mathcal{G}B$ which is

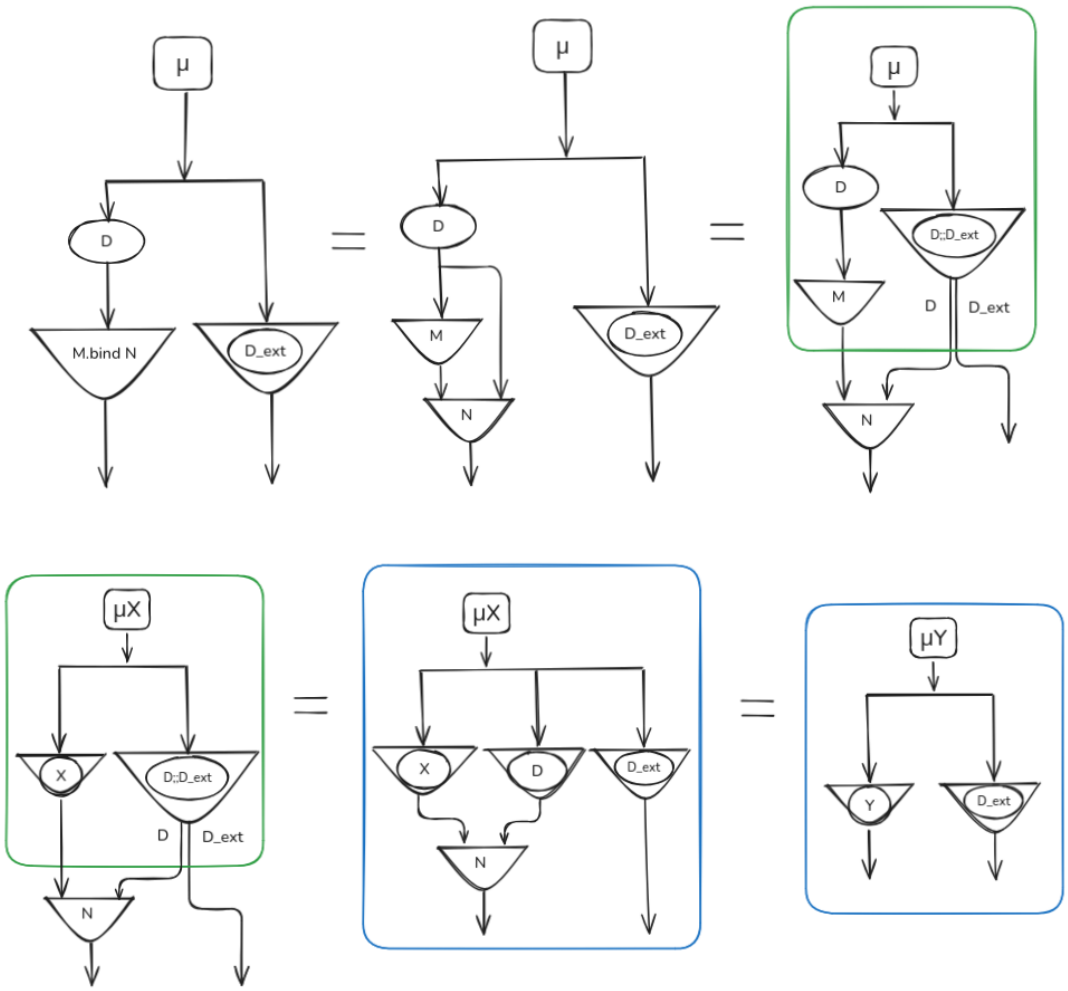


Figure 4.3: A proof of soundness of the `wp_bind` rule in our custom diagrammatic notation for (point-free) equational reasoning

Figure 4.4: A suggestion for visualising a Finite footprint measurable space over a hilbert cube. The entire diagram is a single measurable space. Each strip makes up the measurable sets (or events) in the space. Each line in represents an infinite tuple of reals in the unit interval, connecting dots which single reals, on a vertical number line. Note that beyond the finite footprint, the lines get dense. In a fully faithful diagram, that part would be entirely filled in.

correctly typed as a denotation of a program (in other words it is a kernel).

The soundness of `wp_meas` is the general theorem that we prove in lean and instantiate to `wp_unif` and `wp_flip`. The choice of the Hilbert Cube with finite footprint as a probability space was made to validate `wp_meas`. In order to prove the soundness of this result, we first had to establish a substantial body of fundamental results in Lean about Hilbert cubes with finite footprint, to express transformations which are so intuitive that they do not even get mentioned in proof in Lilac's appendix, but nevertheless, took weeks of effort. The reason for this difficulty boils down to the fact that Finite footprints were invented for validating these proof rules, and isn't of general mathematical interest to be supported by Mathlib.

Definition B.5 [Li et al., 2023, Appendix B] A sub-σ-algebra $\mathcal{F}$ of $\Sigma_{I^{\mathbb{N}}}$ has finite footprint if there is some n such that every $F \in \mathcal{F}$ is of the form $F' \times [0,1]^{\mathbb{N}}$ for some $F' \subseteq [0,1]^n$.

The intuition behind this definition goes back to the slogan "measurability is ownership". Finite footprint restricts the set of measurable functions from the Hilbert cube (thereby restricting ownership over random variables). We have adopted the below equivalent definition which makes the intuition explicit, at the cost of introducing a few more concepts (the change motivated making the proofs clearer and succinct). The following snippets are from Bppl.Lilac.Std.

```
/-- The σ-algebra is only "interesting" in the first `n` coordinates for
some finite `n`.  In the rest of the coordinates, its the `univ` set. -/
def FiniteFootprint (n : ℕ) (ms : MeasurableSpace (ℕ → I)) : Prop :=
  ∃ ms' : MeasurableSpace (Fin n → I), ms = unSplitBi (ms' ×ₘ (⊥ : ℕ → I))
```

First let's look at the types involved $\mathbb{N} \to I$ is exactly the Hilbert cube, while Fin $n \to I$ (I^n) is a finite portion of it. There exists an isomorphism $(\text{Fin } n \to I) \times (\mathbb{N} \to I) \simeq \mathbb{N} \to I$, which just says that we can split something infinite to a finite and infinite part.

We define finite footprint based on this isomorphism, defined in Lean as `splitBi`:

```
noncomputable def splitBi (n : ℕ) : (ℕ → I) ≃ᵐ (Fin n → I) × (ℕ → I) :=
  (MeasurableEquiv.piCongrLeft (fun _ ⇒ I) (finSumNatEquiv n)).symm.trans
  (MeasurableEquiv.sumPiEquivProdPi (fun _ ⇒ I))
```

We focus on the type of splitBi; $\simeq^m$ is notation in mathlib for measurable equivalence, lifting the isomorphism just presented above for the types, to measurable spaces, i.e. there are measurable functions going back and forth, with composition in each way giving the left or right identity functions. (The body of the definition is using results from Mathlib, and is just constructing such an isomorphism)

`splitBi` is the natural notion of isomorphism between measurable spaces that we chose for defining finite footprints, finally here is unsplitBi, which is applying the splitBi backwards, from the split measurable space to the "unsplit" space by taking the pullback σ-algebra, which is called `comap` in Lean.

```
abbrev unSplitBi {n : ℕ} (ms : MeasurableSpace ((Fin n → I) × (ℕ → I)))
    : MeasurableSpace (ℕ → I) := ms.comap (splitBi n)
```

Now returning to the definition `FiniteFootprint`, we see that we are choosing some arbitrary measurable space on the finite part, and the measurable space $\bot = \{\phi, I^{\mathbb{N}}\}$ on the infinite part. This smallest/trivial measurable space on the infinite part of the hilbert cube, makes that part effectively meaningless. A random variable mapping from a finite footprint hilbert cube (thought of as an infinite list), would be forced to ignore all indices greater than n (from the infinite part). We offer a way to visualise a measurable space over the hilbert cube in 4.4.

Let's take a quick example, to see why indices after n must be ignored. Take $X : I^{\mathbb{N}} \to Bool$, and a finite footprint measurable space over the domain and the powerset measurable space over the codomain $(\{\phi, \{false\}, \{true\}, \{false, true\}\})$.

Let's say we know $X(0.1, 0.1, 0.1, \ldots) = false$. Then what is the pre-image of the measurable set false constrained to be, to remain measurable? The smallest possible answer is $X^{-1}(\{false\}) = (0.1, 0.1, a_0, a_1, a_2, \ldots)$ for arbitrary $a_i \in I$. Referring to Fig 4.4, this is the same as saying that the region past n, is dense. Hopefully, this convinces the reader of the key fact that a random variable (which is a measurable function) from a finite footprint Hilbert cube, must ignore all the dimensions beyond n, in other words Finite footprint hilbert cube (FFHC) can't own every random variable and are restricted in some way. In particular, we are always able to easily construct random variables that a FFHC does not own simply by having some Y consider the values of dimensions greater than n.

The above FFHC property of measurability interpretable as ownership, only used the fact that FFHC can be decomposed into a product. So it is just a property of product measurable spaces. The key property of a lilac resource $\mathcal{P} = (\Sigma_{I^{\mathbb{N}}}, \mathcal{F}, \mu)$ (FFHC along with a probability measure) is one can create a new $\mathcal{P}' = (\Sigma_{I^{\mathbb{N}}}, \mathcal{F}', \mu')$ by "adding a random variable Y to $\mathcal{P}$". Let's make that intuition precise.

1. For all random variables $X : (I^{\mathbb{N}}, \mathcal{F}) \xrightarrow{\text{m}} A$ (we explicitly state the measurable space of the domain), which we refer to as random variables owned by $\mathcal{P}$, X must also be measurable w.r.t. $\mathcal{F}'$.

2. Say we have a probability measure ν in the space $B : \texttt{Meas}$. We are able to construct $\mathcal{P}'$ such that with a suitable choice of $Y : (I^{\mathbb{N}}, \mathcal{F}') \xrightarrow{\text{m}} A$, $Y_{\#}\mu' = \nu$!

Let's show concretely how to construct both $\mathcal{P}'$ and $Y : I^{\mathbb{N}} \xrightarrow{\text{m}} \mathbb{R}$, taking $\mathbb{R}$ as the co-domain and the target distribution $\nu = \text{Unif}[0, 1]$ to be the uniform distribution on the unit interval, in this example. Let $\mathcal{P}$ have finite footprint n. The idea is to first construct a $\mathcal{P}_n = (I^{\mathbb{N}}, \mathcal{F}_n, \mu_n)$, where we can define Y as a projection $Y := (\omega : \mathbb{N} \to I) \mapsto \omega n$ to obtain the target distribution $Y_{\#}\mu_n = \nu$. $\mathcal{P}_n$ satisfies condition (2) above while the original $\mathcal{P}$ of course satisfies (1), but neither satisfies both. We claim that taking the independent combination $\mathcal{P}' := \mathcal{P} \star \mathcal{P}_n$ will satisfy both conditions. Deferring the justification of this claim to the next section 4.4, we provide the construction of $\mathcal{P}'$ here.

The Hilbert cube can be decomposed into product measurable spaces at arbitrary indices n. The construction of $\mathcal{P}_n = (I^{\mathbb{N}}, \mathcal{F}_n, \mu_n)$, is defined using a decomposition of the hilbert cube into 3 pieces: before n, at n and after n.

$$(I^{\mathbb{N}}, \mathcal{F}_n) \simeq^m (I^n, \bot) \times_m (I, \Sigma_I) \times_m (I^{\mathbb{N}}, \bot)$$

We also need to define μ_n. On the trivial measurable space $\bot = (I^x, \{\phi, I^x\})$ measurable space there is only one possible probability measure defined as $\mu_\bot := \{\phi \mapsto 0, \text{univ} \mapsto 1\}$. $\mu_n := \mu_\bot \otimes \lambda_I \otimes \mu_\bot$. Where λ_I is the Lebesgue measure on I and $\otimes$ is the product of measures (see Sec 2.1).

Note that λ_I expanded into a measure on $\mathbb{R}$ (mapping all new elements from its domain to 0), is precisely $\nu = \text{Unif}[0, 1]$. So if Y is a map from $\mathcal{P}_n$, we get $Y_{\#}\mu_n = \text{Unif}[0, 1]$ as required. We wish that if we take the independent combination $\mathcal{P}' := \mathcal{P} \star \mathcal{P}_n$, then we will also have $Y_{\#}\mu' = \text{Unif}[0, 1]$

We are now in a position to state a 3rd slogan: "Hilbert Cubes with Finite Footprint are infinite random number generators". Given some resource $\mathcal{P}$ representing the state of a program, we are always able to modify this state to some $\mathcal{P}'$ which supports one more random variable Y of our choice. Each of these Ys are just straightforward projections of some dimension n, i.e. measure on each dimension is the source of randomness. And we can repeat this process arbitrarily, because of the finite footprint condition, obtaining a resource which simultaneously supports an arbitrary number of random variables Y_i.

We end this section with the definition of a finite footprint for a measurable function:

Definition B.6 [Li et al., 2023, Appendix B] A random variable $X : (I^{\mathbb{N}}, \Sigma_{I^{\mathbb{N}}}) \to (A, \Sigma_A)$ has finite footprint if the pullback σ-algebra $\{X^{-1}(E) \mid E \in \Sigma_A\}$ has finite footprint.

The need for this definition is a technical detail: sometimes we will not always have access to a measurable space with finite footprint to define a measurable map from. Even when a measurable function is from the Borel σ-algebra on the Hilbert cube, this definition allows us to only consider those measurable maps, which still behave as if they were mapping from some σ-algebra with a finite footprint. We will see an example of this in use in Sec 4.2.

4.4 wp_meas

In Sec 4.3 on Finite Footprints we gave most of the intuition for the proof of the rule `wp_meas` followed by defining precisely the semantics of `wp`, which appears in `wp_meas`, in Sec 4.1. Now we can discuss the proof of soundness of `wp_meas`, elaborating on Sec 4.3 and showing parts of the Lean mechanisation.

In our presentation of transformations on the Hilbert cube, we are being quite verbose with stating the isomorphisms (measurable equivalence), by "splitting" and "unsplitting" the Hilbert cube. In the informal proof this isomorphism is not mentioned, and it is natural to fluidly move in between the two representations, not even noticing that there must in fact be two different representations in a type theory (since the types $I^{\mathbb{N}}$ and $I^n \times I^{\mathbb{N}}$ are not equal). At the base there is a measurable equivalence on measurable spaces, and on top of that there is also an isomorphism of measures over each representation. In Sec 4.3 we had brushed over this as is done in the informal proof, appealing to intuition. However, this isomorphism between the measures is not covered by Mathlib, so as part of this project we have had to develop some deep measure-theoretic results which are not directly related to finite footprints. This is one of the factors which makes the proof of `wp_meas` the hardest in the entire development, and ended up taking weeks instead of days. We had hoped to talk here about the specific gaps in Mathlib's infrastructure, presenting it in full detail. However, this introduces a lot of notation and details of the reasoning steps in the proofs, to reach a rather technical point in the grand scheme of the entire formalisation. So instead, we will not present the details of

we talk specifically about the measure-theoretic result which lets us make some remarks on some incompatibilities of Mathlib (or mainstream mathematics) with our requirements which has plagued most proofs across the development to some extent.

$$(\forall X : \mathrm{RV}\ A.\ X \sim \mu \mathrel{-\!\!*} Q) \ \vdash\ \mathrm{wp}(\lambda\ \_ \mapsto \mu)(\lambda X : A. \mapsto Q) \qquad \text{(WP-MEAS)}$$

We want to prove a `wp` statement.

Taking some $\mathcal{P}$ for which the premise holds, our goal is to validate $\mathcal{P} \vDash \mathrm{wp}(\lambda\ \_ \mapsto \mu)(\lambda X : A. \mapsto Q)$. To validate a `wp` connective, one of the things we require is to construct a new $\mathcal{P}'$ which owns a random variable X whose distribution is μ. Mathematically stated, given $\mathcal{P}' = (\Sigma_{I^{\mathbb{N}}}, \mathcal{F}, \mu')$,

In order to see what is required to achieve this rule we first need to remember the semantics of the $-\!\!*$. $X \sim \mu \mathrel{-\!\!*} Q$ holds for some $\mathcal{P}$ if we have $\mathcal{R} \vDash X \sim \mu$, and $\mathcal{R} \star \mathcal{P}$ exists then $\mathcal{R} \star \mathcal{P} \vDash Q$. If the premise above holds, then

We need to

$\mathcal{R} \vDash P \mathrel{-\!\!*} Q$ if there is some $\mathcal{P} \vDash P$ Todo: How do I talk about anything hard without getting bogged down with explaining easy stuff. I think I should just give up on re-explaining proofs. It makes no sense there's already a paper, and I've formalized it so why another version :(

We begin with this equality at the heart of the proof.

```
lemma commute_with_add_dim {N : ℕ} (N_ms : MeasurableSpace (Fin N → I))
    : unSplitTri ((N_nil N) ×ₘ I_borel ×ₘ Inf_nil) ⊔ unSplitBi (N_ms ×ₘ Inf_nil)
      = unSplitTri (N_ms ×ₘ I_borel ×ₘ Inf_nil) := by ...
```

Recalling the development from Sec 4.3 this proof is essentially $\mathcal{P}_n \star \mathcal{P} = \mathcal{P}'$ (in that order in the Lean code) but only showing the equality for the underlying measurable space, forgetting the measure. For example, if `.ms` denotes the underlying measurable space then $\mathcal{P}.\mathrm{ms} = \mathrm{unSplitTri}((\mathrm{N\_nil}\ N) \times_m \mathrm{I\_borel} \times_m \mathrm{Inf\_nil}) = (I^{\mathbb{N}}, \mathcal{F}_n) \simeq^m (I^n, \bot) \times_m (I, \Sigma_I) \times_m (I^{\mathbb{N}}, \bot)$. Note

that the independent combination $\star$, cast down to measurable spaces, is the sum of the two measurable spaces: the smallest σ-algebra generated from the union of the two spaces. This is also `join` $\sqcup$ (measurable spaces form a lattice), which is what we see in Lean.

There was a slight omission in the above presentation, regarding the presence of $\mathcal{P}_{\text{frame}}$

We want this result to extend to the relevant measures on these spaces clearly. This is the contents of the file Bppl.Lilac.ProofRules.MeasureProduct. There we prove lemmas to establish this result as stated. However ideally we could generalise this result much further to just be about an isomorphism between measures when there is a measurable equivalence between two spaces, which are some combination of products and sums. Our current proofs are tied to the definitions of finite footprint and the specifics of PSLs like Lilac, but it would be much nicer, to add these general theorems about how measures are isomorphic over different measurable equivalences over spaces constructed with sums and products in Mathlib. This is important future work, to ensure the readability and extensibility to newer logics for the Lean codebase.

4.4.1 Use of Aristotle in this proof

Here we outline our usage of Aristotle, a public free Generative model, which is intended specifically for writing Lean code. There has been astonishing progress in the successfulness of AI in writing valid Lean proofs, within the duration of the project. We tried Claude, at the end of 2025 to try to fill in some proof holes, and it was not particularly useful and abandoned, along with all AI generated code. However, when attempting to use these tools again in within the previous month or two (specifically we have tried only Aristotle) these tools are now consistently at the level of automating the proofs of time-consuming but trivial helper lemmas of a few lines each. Specifically by "helper lemma" we actually refer to `have := ...`, statements peppered throughout all the proofs. This being the first mechanisation project of this scale in my experience, I had learned at some point that it was a terrible idea to fill every hole in every proof, and always keep the codebase in a coherent state. It is instead a much better approach to repeatedly quash the biggest uncertainties of the shape of the mechanisation: for example the interface between the language and logic, the interface between logic and Iris, whether verification of an open program (containing a free variable) is possible, the soundness proofs of probability specific proof rules, and their usage. It was important to trust intuition and develop the project as a patchwork of proofs that slowly come together, to get the most accurate idea of how much time each part would take and avoid wasted effort on a proof, that might need to be reworked. The approach naturally left behind a lot of trivial holes, which we're exactly the right size for having Aristotle fill, since morally, there was only 1 right answer.

The biggest issue with using Aristotle with larger or difficult tasks, it may pursue terrible ideas, and give a very wrong impression of the difficulty and the best approach of proving a difficult theorem, especially when there are ambiguities and choices to be made. The proof of `wp_meas` is where Aristotle was inadvertently used the most, hence we include this reflection here. Firstly, as mentioned before the pure measure theoretic results, where so intuitive that they were stated as "by definition" in the informal proofs

The goals of this project are two-fold, build an extensible foundation for mechanising future PSLs and get the most thorough intuition of the working of a PSL.

AI generated code and proofs verified by a foundational theorem prover, used to be a gold-standard for reliable code-generation from inherently untrustworthy systems, that was out of reach. But within the span of the year it is essentially in reach. Reflecting on my experience with it so far, I expect it would catalyse foundational verification, making formally verified code common place in the general software engineering landscape and also a basic requirement for many critical applications. I believe that research into verificaiton of unconventional software systems such as probabilistic programming then becomes more important.

The companies which work on these tools mostly focus on Lean, Todofootnote for one reason or another. And it is reason or another

4.4.2 Difficulties with type class inference and upstreaming to Mathlib

An unavoidable difficulty with much of this Lean development was the issue of incorrect type-class inference, in particular the inference of MeasurableSpace α associated to a type α. To see the probem recall that we noticed a tension in the standard mathematical notation for measurable spaces, for example $I^{\mathbb{N}} \xrightarrow{\text{m}} A$. Both the measurable spaces in $I^{\mathbb{N}}$ and A are implicit. In this situation typeclasses used, with the definition of measurable functions given like so:

```
structure MeasurableFun (α β : Type*) [MeasurableSpace α] [MeasurableSpace β] where
  /-- underlying function -/
  toFun : α → β
  meas: Measurable toFun
```

[...] represents an argument that should be filled by *typeclass search*. This is different from the usual *type inference*. The success of a typeclass search is whether a suitable typeclass instance of in this case MeasurableSpace α which could be found/constructed. *typeclass search* does not consider whether the term thus generated, type checks. So if we have multiple instances of typeclasses for $I^{\mathbb{N}}$, it will pick one randomly and the resulting term of proof will not be expected since it won't have the correct type. This issue can be fixed by pass the typeclass argument explicitly, but this only work 1 level deep. What we mean by that is that if a lemma uses an internal lemma, the internal lemma also using typeclass search to find a MeasurableSpace α, if the internal typeclass search yields the wrong lemma, there is nothing we can do about that. The proof will fail. The workaround to prevent this from happening, is to be very careful not to place the wrong MeasurableSpace instance into the ambient proof context. This is however unavoidable sometimes as we realised in

was to keep a single type class have also been used in the above definition with this notation. The latter is conservative, only giving type to a term if

In Mathlib, typeclasses are used to fill in these implicit parameters /citepspitters. We can specify it explicitly if But $I^{\mathbb{N}}$ has various measurable spaces associated with it. So in our own notation we sometimes write $(I^{\mathbb{N}}, \Sigma_{I^{\mathbb{N}}}) \xrightarrow{\text{m}} A$ to specify it explicitly. We can do the same in Mathlib lemmas as well, but the issue is that we can only explicitly specify the typeclass level down to one level of unfolding. For all "typeclass searches"

In particular, in Mathlib, indeed in mathematics, it is normal

Chapter 5

Case Studies

In this chapter we demonstrate a few simple examples of how our embedding of Lilac in Iris can be used for interactively proving specifications of APPL programs. We develop some proof automation, and reflect on more divergences from the Lilac paper [Li et al., 2023]

First in sec 5.1 we verify the most basic programs which we had struggled to verify under the original definitions in Chapter 3. Next in sec 5.2 we outline difficulties faced with reasoning about `LProp`s which used the `expectation` ($\mathbb{E}[X] = e$) connective in the logic. We develop some proof rules regarding expectation and some custom tactics for ergonomic reasoning.

Most notably we use this example to remark on how in our mechanised proofs we completely sidestep the usage of the almost surely equals connective ($\overset{\circ}{=}$) and substitutions using the congruence rule. This is not specific to this particular example and is a general phenomenon. Indeed, the Lilac paper [Li et al., 2023] has not provided proof rules for introduction of $\overset{\circ}{=}$[1], and uses it rather intuitively in the proof sketches. In our formalisation, we have not required it at all.

5.1 Sampling Uniform Distributions

We recall the proof from Sec 3.4.2, which we had failed to complete under the old definitions. The informal and lean version, taken from Bppl.Lilac.Examples, are provided side-by-side.

```
abbrev unif1 : Term [] Ty.real.G :=
  unif01.bind (
  ret (#0)
)
```

$x \leftarrow \text{unif}\ [0,1]; \text{ret}\ x$

Since we are using de Brujin indices we cannot name variables bound by the `bind` construct, but instead need to access these variables from an implicit context. `#0` is shorthand for `Term.var Member.head`, which refers to the syntactically closest (right most) variable bound. In this case it is obvious which variable we refer to since there is only a single variable. Below the informal specification is followed by the formalised Lean specification:

$$\{\top\}\ \textsc{Unif1}\ \{X.\ X \sim \text{Unif}[0,1]\}$$

```
nil : RV (List.TProd (⟪·⟫) []) := ...
abbrev post1 : LProp :=
  wp (unif1.den ∘ᵣ nil) (λ X : RV ⟪Ty.real⟫ ↦ (X ~ unif01_sem))
```

[1] the original notation from the paper for the almost surely equals connective is $\overset{as}{=}$. We use $\overset{\circ}{=}$ in the lean formalisation

We see that the lean code of the specification has some additional components implicit in the informal version. Recall that `wp` has type, `def wp (M : RV (⟪Ty.G A⟫)) (Q : RV ⟪A⟫ → LProp) : LProp`. We have assigned `(M := unif1.den ∘ᵣ nil)`. Why is it not just `unif1.den`, the denotation of the unif1 program we had just defined above? Inspecting the types, unif1.den : $[\![\Delta]\!] \xrightarrow{\text{m}} \mathcal{G}\mathbb{R}$, but we require $I^{\mathbb{N}} \xrightarrow{\text{m}} \mathcal{G}\mathbb{R}$[2]. Note that in this case $\Delta = [] =$, an empty list so $[\![\Delta]\!] = \text{Unit}$ which is what `List.TProd (⟪·⟫) []` is. We will have an empty context Δ for closed programs, which is generally what we will see in a top level `wp` specification. We compose `unif1.den` with the map `nil` to resolve the discrepancy. But why do we bother with maps from $I^{\mathbb{N}}$ in the logic in the first place, when this is completely absent in the semantics of APPL? We refer the reader back to Sec 3.2.4, where this insightful question is answered.

$$\{\top\}\ \textsc{Unif1}\ \{X.\ X \sim \text{Unif}[0,1]\}$$

The proof in Lilac using the Iris proof mode is as follows taken from Bppl.Lilac.Examples:

```
theorem unif1_spec :
    ⊢ wp (unif1.den ∘ᵣ nil) (λ X : RV ⟪Ty.real⟫ ↦ (X ~ unif01_sem)) := by
  iapply wp_bind
  iapply wp_unif
  iintro %X HX
  iapply wp_ret
  rw [hrw]
  iexact HX
```

We present the proof state after each tactic[3]. Unlike the failed attempt in Sec 3.4.2 where the proof state was artificially simplified for readability, here we faithfully reproduce the proof state displayed by Lean's info-view, as seen by a user.

```
iapply wp_bind

⊢
⊢ wp (unif01.den ∘ᵣ nil) fun X ↦
  wp (#0.ret.den ∘ᵣ X ;; nil) fun X ↦ (X ~ unif01_sem)
```

Application of the `wp_bind` rule deconstructs the syntactic `bind` in the program `unif1` into nested `wp` statements on the constituent programs `unif01` and `Term.ret #0`. As expected the program of the inner `wp` is not a closed-program and has the newly bound random variable X in its context, at the head of the context. We can see already that `#0` is referring to X, but `#0` is wrapped by `Term.ret`, so there is no reduction that can be applied yet.

```
iapply wp_unif

⊢
⊢ ∀ X, X ~ unif01_sem -*
  wp (#0.ret.den ∘ᵣ X ;; nil) fun X ↦ (X ~ unif01_sem)

iintro %X HX

X : RV ⟪Ty.real⟫.carrier
⊢
*HX : X ~ unif01_sem
⊢ wp ((var head).ret.den ∘ᵣ X ;; nil) fun X ↦ (X ~ unif01_sem)
```

Crucially here we have no `substLProp` unlike in Sec 3.4.2, unblocking further application of rules. This is as far as we had previously gotten.

```
iapply wp_ret
```

[2] actually, `RV (⟪Ty.G Ty.real⟫)`, so we had omitted the finite footprint property for understanding

[3] The interested reader is encouraged to clone the repository and view the file Bppl.Lilac.Examples directly, in a Lean LSP enabled IDE such as vs-code. Interactive proofs are more informative when interacted with.

```
X : RV ⟪Ty.real⟫.carrier
⊢
*HX : X ~ unif01_sem
⊢ (#0.den ∘ₓ X ;; nil) ~ unif01_sem
```

We have removed the `Term.ret` allowing us to simplify the composition of program denotation and context.

```
rw [hrw]
```

```
X : RV ⟪Ty.real⟫.carrier
⊢
*HX : X ~ unif01_sem
⊢ X ~ unif01_sem
```

`hrw` is defined as

```
lemma hrw {A : Ty} {X : RV ⟪A⟫} : ((#0).den ∘ₓ (X ;; nil)) = X := by rfl
```

as seen the proof is `rfl` (reflexivity) which only holds for two terms which are *definitionally equal*, meaning they reduce to the same normal form[4]. We can see a well defined procedure for constructing such rewrite lemmas, so this step can be automated using a custom simplification tactic for our logic. However we didn't pursue this, as we would like to solve the deeper problem of de Brujin indices being unintuitive, and custom tactics to automate rewrites would likely become redundant then.

```
iexact HX
```

```
No goals
```

This concludes the first ever mechanised Lilac proof.

5.2 Specifications with Expectations

Here we explore a more nuanced example where we first encountered difficulty with a notational side-stepping of an issue in the Lilac paper [Li et al., 2023]. Next, we use this opportunity to also comment on a pervasive simplification in proofs in our mechanisation and informal proofs provided in the paper. Here is a Hoare triple style proof annotation of the program `Halve` which we will next mechanise.

$$
\begin{array}{l}
\{\, \mathrm{own}\, X \,\} \\
Y \leftarrow \mathrm{unif}\ [0,1]; \\
\{\, \mathrm{own}\, X * Y \sim \mathrm{Unif}[0,1] \,\} \\
\mathrm{ret}\ (XY) \\
\{\, Z.\ \mathrm{own}\, X * Y \sim \mathrm{Unif}[0,1] * Z \mathrel{\hat{=}} XY \,\} \\
\{\, Z.\ \mathbb{E}[Y] = 1/2 \wedge \mathbb{E}[XY] = \mathbb{E}[X]\mathbb{E}[Y] \ * Z \mathrel{\hat{=}} XY \,\} \\
\{\, Z.\ \mathbb{E}[Y] = 1/2 \wedge \mathbb{E}[Z] = \mathbb{E}[X]\mathbb{E}[Y] \,\} \\
\{\, Z.\ \mathbb{E}[Z] = \mathbb{E}[X]/2 \,\}
\end{array}
$$

We choose the `Halve` program, as not only does it exercise one of the salient features of the logic, reasoning about expectations, it would also test whether the mechanisation can support this rather unconventional goal of verifying a specification on an open variable. We are very much interested in whether our approach to variables is robust to these kinds of examples, as it was a major difficulty that was overcome in this project.

The immediate difficulty to notice is that the expectation is a logical connective, $\mathbb{E}[X] = e$, includes $=$ in the syntax of the connective which can be misleading. It expects arguments typed as $E : \mathrm{RV}\ \mathbb{R}$ and $e : \mathbb{R}$. So in particular, none of the expectations in the above proof, are using valid syntax. For example the post condition must be expanded to:

$\{\, Z.\ \exists e,\ \mathbb{E}[Z] = e/2 \wedge \mathbb{E}[X] = e \,\}$

[4] there's quite a bit of nuance to what is considered a normal form and which reductions need to be considered, for the purposes of computational efficiency of the `defEq` algorithm, that we don't go into

Is this merely an inconvenience or a more fundamental problem, which might make this theorem unprovable in the logic? The latter was in-fact a concern since it is not actually possible to instantiate the above existential! For clarity, we emphasize that $\mathbb{E}[X]$ is not well-defined in the logic. But the $\mathbb{E}[X]$ value is well-defined in the meta-logic (Lean); it is defined as an integral of the random variable X. We want to instantiate e with this semantic $\mathbb{E}[X]$. Let us first present the partial proof until we hit this roadblock.

```
lemma hrw (X Y : RV ⟪Ty.real⟫) :
    (Term.den ((#1 * #0)) ∘ᵣ Y ;; X ;; nil) = X * Y := by rfl

abbrev post_half (X : RV ⟪Ty.real⟫) : LProp := wp (half.den ∘ᵣ (X ;; nil)) (λ Y : RV ℝ ↦
    iprop(∃ e, 𝔼[Y]=e/2 ∧ 𝔼[X]=e))

theorem half_spec (X : RV ⟪Ty.real⟫) : own X ⊢ post_half X := by
  iintro hX
  rw [post_half]
  iapply wp_bind
  iapply wp_unif
  iintro %Y hY
  iapply wp_ret
  rw [hrw]
```

but that is not possible as we demonstrate, with in this partia i.e. an expression in Lean. This is well-defined. But we still aren't able to construc We would of course want to instantiate it with the semantic (Lean)

5.2.1

5.3 Removing the almost surely equals connective ($\overset{\circ}{=}$)

As we have seen in the previous section we have completely removed the need for usage of the almost surely equals connective and analogous situations appear all throughout the paper, both in examples using Core Lilac (Lilac without the conditioning modality) and full Lilac. We give one more example from the paper to justify this claim, presenting an example using the conditioning modality. Simultaneously, we are demonstrating that the set of definitions for Core Lilac will translate naturally into full Lilac. We explain just what is relevant to showing that almost surely equals is not necessary, omitting the conditioning-specific proof rules. This is the first instance in this report where we present something which does not have corresponding end-to-end proofs in Lean. Below is a condensed version of Li et al. [2023, Appendix C]:

$$
\begin{array}{l}
\{\,\top\,\} \\
Z \leftarrow \mathrm{flip}(1/2); \quad X \leftarrow \mathrm{flip}(1/2); \quad Y \leftarrow \mathrm{flip}(1/2); \\
\{\,Z \sim \mathrm{Ber}(1/2) * X \sim \mathrm{Ber}(1/2) * Y \sim \mathrm{Ber}(1/2)\,\} \\
A \leftarrow \mathrm{ret}\,(X \vee Z); \quad B \leftarrow \mathrm{ret}\,(Y \vee Z); \\
\{\,Z \sim \mathrm{Ber}(1/2) * X \sim \mathrm{Ber}(1/2) * Y \sim \mathrm{Ber}(1/2) * A = (X \vee Z) * B = (Y \vee Z)\,\} \\
\mathrm{ret}\,(Z, X, Y, A, B) \\
\{\,\underset{z \leftarrow Z}{\mathbf{C}}(\mathrm{own}(A) * \mathrm{own}(B))\,\}
\end{array}
$$

We translate the Hoare style specific to a `wp` style specification, and show the stages of reduction as we apply the `wp` proof rules:

$$
\begin{array}{l}
\vdash \mathrm{wp} \\
\left(\begin{array}{l}
Z \leftarrow \mathrm{flip}(1/2); \quad X \leftarrow \mathrm{flip}(1/2); \quad Y \leftarrow \mathrm{flip}(1/2); \\
A \leftarrow \mathrm{ret}\,(X \vee Z); \quad B \leftarrow \mathrm{ret}\,(Y \vee Z); \\
\mathrm{ret}\,(Z, X, Y, A, B)
\end{array}\right) \\
\{\,(Z, X, Y, A, B).\ \underset{z \leftarrow Z}{\mathbf{C}}(\mathrm{own}(A) * \mathrm{own}(B))\,\}
\end{array}
$$

$$
\begin{array}{l}
X\ Y\ Z : \mathrm{RV}\ \mathbb{R} \\
X \sim Ber(1/2) \\
Y \sim Ber(1/2) \\
Z \sim Ber(1/2) \\
\vdash \mathrm{wp} \\
\left(\begin{array}{l} A \leftarrow \mathrm{ret}\,(X \vee Z); \quad B \leftarrow \mathrm{ret}\,(Y \vee Z); \\ \mathrm{ret}\,(Z, X, Y, A, B) \end{array} \right) \\
\{\, (Z, X, Y, A, B).\ \underset{z \leftarrow Z}{\mathbf{C}}(\mathrm{own}(A) * \mathrm{own}(B)) \,\}
\end{array}
$$

$$
\begin{array}{l}
X\ Y\ Z : \mathrm{RV}\ \mathbb{R} \\
X \sim Ber(1/2) \\
Y \sim Ber(1/2) \\
Z \sim Ber(1/2) \\
\vdash \mathrm{wp}\ \ \mathrm{ret}\,(Z, X, Y, (X \vee Z), (Y \vee Z)) \\
\{\, (Z, X, Y, A, B).\ \underset{z \leftarrow Z}{\mathbf{C}}(\mathrm{own}(A) * \mathrm{own}(B)) \,\}
\end{array}
$$

$$
\begin{array}{l}
X\ Y\ Z : \mathrm{RV}\ \mathbb{R} \\
X \sim Ber(1/2) \\
Y \sim Ber(1/2) \\
Z \sim Ber(1/2) \\
\vdash \{\ \underset{z \leftarrow Z}{\mathbf{C}}(\mathrm{own}(X \vee Z) * \mathrm{own}(Y \vee Z))\ \}
\end{array}
$$

Todo: Condense into a diagram with arrows stating the proof rules applying to achieve the transformations

We have made the program context (of the form `∘ᵣ (... ;; nil)`) implicit, and given explicit names rather than de Brujin indices to variables, as a consequence of this omission, for readability. We color code the program in black and the LProp logical assertions in blue to show the entire reduction of of the program using weakest precondition reasoning and our proof rules, never needs to use $\stackrel{\circ}{=}$ the almost surely equals connective. The proof is not completed, and we have stopped at the point that we would have needed proof rules about the conditioning modality, the mechanisation of whose soundness proofs we leave as future work[5]

5.3.1 Comparison with Bluebell

Todo: This section is incomplete. What I aim to talk about is that the definition of almost surely equals in Lilac, and as a consequence the proof its properties such as being an equivalence relation, got somewhat convoluted, and in Bluebell a fundamentally alternative approach was pursued in part to offer a cleaner solution to this problem.

In particular Bluebell includes variables in its resource algebra. @Shing Hin can you confirm if these claims are correct?

Because of this observation, I just want to state that it is strange I ended up not needing almost surely equals at all. And I would want to try out more examples and implement full Lilac (with conditioning) before confirming if almost surely equals is indeed not necessary....

[5]This is expected to be similar in difficulty to the formalisation of `wp_unif` possibly requiring the mechanisation of relevant measure theoretic lemmas first before we can proceed with the soundness proof itself.

Chapter 6

Verification of Probabilistic Inference using Separation Logic

This chapter outlines a separate effort from the previous chapters where we had mechanised Core Lilac [Li et al., 2023]. Here we sketch a novel theoretical contribution, proposing to apply probabilistic separation logic to the verification of not only PPLs but also their compiler/interpreter. We argue why a successor of Lilac, Bayesian Separation Logic (BaSL) [Ho et al., 2025], brings us one step closer to the verification of probabilistic inference algorithms in a separation logic. Two well-motivated `research problems` *which lie at the intersection of measure theory and the semantics of PPLs are posed.*

The importance of verification of probabilistic inference, which lies at the heart of the compilers or interpreters of PPLs has already been established by Ścibior et al. [2017] and Tassarotti and Tristan [2023]. The former undertakes more foundational work expressing probabilistic inference in higher order probabilistic programs and verifying correctness properties directly using the denotational semantics. The latter verifies correctness of key passes in the compiler of the Stan PPL [Carpenter et al., 2017]. Both approaches prove properties of the denotational semantics directly and do not work in a PSL. Our hypothesis is that it is beneficial to work at the higher level of abstraction of a separation logic, which has not been attainable before the recent introduction of BaSL [Ho et al., 2025]. This is because reasoning about unnormalised probability distributions is crucial for probabilistic inference, as we will explain below, and no logic before BaSL was capable of that. If our hypothesis that PSL offers a scalable framework for reasoning about probabilistic inference is correct, then we would unlock being able to produce foundational correctness proofs for the latest probabilistic inference algorithms published every year in an active field of research **?**..

6.1 Motivation

Bayesian inference is the problem of updating our beliefs about some unknown quantity once we have seen some data. As a concrete example, suppose we receive a bent coin and want to infer its bias $\theta \in [0, 1]$, the probability that it lands heads. Before tossing it we have some prior belief about θ, given by the prior distribution $p(\theta)$; a convenient choice here is the $\mathrm{Beta}(\alpha, \beta)$ distribution, whose two parameters let us tune how confident we are that the coin is fair. We then flip the coin n times and observe k heads, which we collect into our data $\mathcal{D}$. The probability of seeing this data for a given bias is the likelihood $p(\mathcal{D} \mid \theta) = \theta^k(1-\theta)^{n-k}$. What we actually want is the *posterior* $p(\theta \mid \mathcal{D})$, our updated belief about the bias having seen the flips, and Bayes' rule tells us how to get it from the prior and the likelihood:

$$p(\theta \mid \mathcal{D}) = \frac{p(\mathcal{D} \mid \theta)\, p(\theta)}{p(\mathcal{D})}, \qquad \text{where} \qquad p(\mathcal{D}) = \int_0^1 p(\mathcal{D} \mid \theta)\, p(\theta)\, d\theta.$$

Notice that the numerator is just the likelihood times the prior, and we can evaluate both at any θ easily. The denominator $p(\mathcal{D})$, called the *model evidence*, doesn't depend on θ at all[1]; its only job is to normalise the posterior so that it integrates to one. It is this normalising constant, an integral over the whole space of θ, which is in general intractable. It involves exploring the entire sample space to compute this integral[2].

This is a central motivation for the field of probabilistic inference: *given an unnormalised probability distribution (usually obtained through an application of Bayes' rule), how do we obtain the normalised distribution, possibly through approximate methods?*

Say we are given an unnormalised distribution $\widetilde{\pi}(x)$. Then we would like to obtain some information on the normalised $\pi(x)$, without explicitly computing the normalising constant, the integral in the denominator in the above equation. We can ask 3 concrete questions [Murphy, 2023; van de Meent et al., 2021]

1. How to determine a closed form approximation $q(x)$ from a parametrised family of distributions $q \in Q_\theta$ such that q is "closest" to π, $q \approx \pi$ (Variational Inference)
2. Drawing samples distributed as $\pi(x)$ (MCMC: Markov Chain Monte Carlo)
3. Approximating the expectation (or some other integral) over π (SMC: Sequential Monte Carlo)

In what follows we focus exclusively on MCMC, which is what is used in the interpreter or compiler of a PPL.

6.2 Measure vs density

Let's make the goal of MCMC precise. We had stated that we wanted to draw samples given an unnormalised distribution $\widetilde{\pi}(x)$ which corresponds to a true distribution $\pi(x)$ which cannot be stated in closed form[3].

"Distribution" could mean a few things. It can either mean a *measure*, $\mu : \Sigma_A \to \mathbb{R}^+$ or a *density*, $\rho : A \to \mathbb{R}^+$. Most common measures *admit a density*, in which case we can treat the two interchangeably, hence the terminology "distribution" refers to both. MCMC is only defined for those measures which admit a density.

Below we describe the conditions for a measure to admit a density.

To even speak of a density we first need to fix a *base measure* λ on the same space, against which the density is measured. On $\mathbb{R}^n$ the natural choice is the Lebesgue measure, which simply assigns to each set its usual length (or area, or volume). The density ρ is then understood relative to λ: the measure of a set A is recovered by integrating the density over it,

$$\mu(A) = \int_A \rho \, d\lambda.$$

We say μ *admits a density* (with respect to λ) exactly when some such ρ exists. When can we find one? The obstruction is easy to spot. If there is a set A that λ considers negligible, $\lambda(A) = 0$, then the integral above is forced to give $\mu(A) = 0$ too, no matter which density ρ we pick. So if μ places any mass on a set that λ ignores, no density can possibly exist. A measure μ is *absolutely continuous* with respect to λ, written $\mu \ll \lambda$, when it avoids exactly this situation:

[1] θ has been integrated out

[2] In this particular example the *posterior* can in-fact be computed since it is a fact that the numerator,the product of a binomial and beta distribution is also beta distribution. The *posterior* is thus a Beta distribution whose parameters we can calculate algebraically. This trick, known as *conjugacy*, only applies in very special cases like our coin flip example though

[3] "in closed form" morally means that intractable operations like integrals are not allowed, but includes most other common mathematical operations (trigonometry, arithmetic, ...) are allowed

$$\lambda(A) = 0 \implies \mu(A) = 0 \qquad \text{for every measurable } A.$$

It turns out this necessary condition is also sufficient. This is the content of the *Radon-Nikodym theorem* [Axler, 2020]: provided λ is σ-finite, μ admits a density with respect to λ if and only if $\mu \ll \lambda$. Moreover the density is essentially unique, with any two choices agreeing λ-almost everywhere. This density is called the *Radon-Nikodym derivative* and written $\frac{d\mu}{d\lambda}$.

A simple measure which fails to be absolutely continuous is the Dirac measure δ_0 on $\mathbb{R}$, which places all of its mass on the single point 0. The point $\{0\}$ has Lebesgue measure zero, yet $\delta_0(\{0\}) = 1$, so $\delta_0 \not\ll \lambda$ and no density exists. This is precisely why MCMC, which manipulates densities, is only defined for those measures which are absolutely continuous.

6.3 Markov Chain Monte Carlo in a PPL

Having established the problem precisely, let us describe the solution. An MCMC algorithm constructs a sequence of values, $x\_i : S$, where the state space S, is defined as the space over which our target distribution is defined. Intuitively it guarantees that after some "burn in values" at the beginning, each element of the rest of the sequence is all distributed according to the target distribution. The way in which these values are generated, is using a kernel[4], known as the transition kernel, $T : S \xrightarrow{\text{m}} \mathcal{G}S$. Suppose we have some starting value $x_0 : S$, then subsequent values are generated by sampling the kernel like so, using Haskell like do notation in the Giry Monad:

$$x_1 \leftarrow T(x_0); x_2 \leftarrow T(x_1); x_3 \leftarrow T(x_2); \cdots$$

This is exactly an unrolling of the for loop construct from Sec 2.2. We recall the syntax here: $for(N, M_i, iX.M_s)$ where N is the number of times to iterate, $M_i : \mathcal{G}A$ from which the initial value is sampled and $\lambda iX.M_s : \mathbb{N} \to S \xrightarrow{\text{m}} \mathcal{G}S$. The last argument is a family of *step kernels* indexed by the iteration index, i. In MCMC we only use a single step kernel ignoring the iteration index i. The previous value is fed into the step kernel, and the next value is sampled. So restating MCMC as a for loop in PPL we have:

$X_N : \mathcal{G}S := for(N, dirac(0), \_ix.T(x))$

The choice of M_i is arbitrary and does not affect correctness of MCMC. Above, we arbitrarily chose 0 as an example. The concrete values $x_1, x_2, \ldots$ obtained on an individual run, associated with random variables $X_1, X_2, \ldots$, representing the distributions of those values across all possible runs. These variables $X_1, X_2, \ldots$ form what are called Markov chains because when each x_{i+1} is sampled, the transition kernel is only passed the immediately preceding value x_i. This property of being dependent on just the previous step, is the Markovian property. For loops in Appl, by their definition, generate Markov chains if we ignore the iteration index i as above.

We had claimed that as $N \to \infty$, $X_N \sim \pi$, or equivalently $\lim_{N\to\infty} T^N(X_0) \sim \pi$. But we haven't even mentioned how π or $\widetilde{\pi}$ gets involved in the construction of $T(x)$ for any of this to work out! In this presentation we lack the space to develop the basics of *ergodic theory* which go into providing the condition on the transition kernel that ensures that the random variable X_N is eventually distributed according to π. So we state the result at a high level. As developed by Meyn et al. [2009], if a transition kernel is *ergodic* then it has an invariant measure, i.e. $\exists \pi'. \lim_{N\to\infty} T^N(X_0) \sim \pi$. We omit the definition of *ergodic* which is quite involved, as it is sufficient to state the following predicate on Kernels and know that it is a provable property with much of the required theory already developed in Mathlib[5]:

```
def HasInvariantMeasure (π' : Measure S) (T : Kernel S S) : Prop
```

(Note that we intend for π' to mean $\widetilde{pi}$ in Lean code.) The astute reader may notice that in the above definition, in Appl, we must have

[4] Extremely convenient for us since the semantics of our programs is also a kernel!

[5]

`(π' : ProbabilityMeasure S)`. BaSL [Ho et al., 2025] is a program logic which is able to reason about unnormalised measures as well, which do indeed arise exactly in the manner they did in our bent coin example from Sec 6.1, through the application of Bayes' rule. The programming language which BaSL reasons about, Bppl, is a generalisation of Appl. So to understand this chapter, it suffices to think of BaSL as Lilac but with support for constructing unnormalised distributions as programs.

We now have the components to establish a new `wp` proof rule that will aid in reasoning about probabilistic inference in a separation logic.

$$\text{HasInvariantMeasure}\widetilde{\pi} \vdash \text{wp}(\llbracket \text{for}(n, e, \_i\, X.\, M) \rrbracket, X : A.\, X \sim \pi) \text{ WP-LIM}$$

Is that true? Not quite since N has to approach infinity for this rule to hold. And in any executable program we cannot have N approach infinity. We could prove the correctness of an inference algorithm by proving the above given $N \to \infty$. However, that is done entirely in the meta-logic and there is nothing that using a separation logic would contribute here. We want to reason about the `for` loop in the above program when it is part of a larger program, as realistic inference algorithms can indeed involve sub-inference procedures as part of it. N not going to ∞ in practice is why these inference methods are called "approximate".

Research Problem 1 : Can we develop a modification of the above rule which is sound by introducing into the logic a method for tracking and bounding the approximation error? The pure measure theoretic component, quantitative convergence of Markov chains, is well-established [Meyn et al., 2009]. The logical component can be based on similar previous "error-bounding" logics such as Approxis [Haselwarter et al., 2025].

6.4 Metropolis Hastings

MCMC is seldom used directly. It is generally hard to find a transition kernel T, which has an invariant measure which we can specify. The Metropolis-Hastings algorithm is perhaps the simplest family of MCMC algorithms. In general there are several parametrised families of MCMC algorithms where the parameters are not as difficult to provide as directly providing a transition kernel T with an invariant measure. We start first with an example, adapted from the *island problem* of Huang [2025].

Teleportation Museum: To celebrate the advances in instantaneous teleportation, a museum dedicated to the technology has just opened in town. There are no lifts or stairs in the building, and instead people can go from one floor to any other using the teleportation portals built into the entire floorspace of all floors. The museum is instantaneously the biggest attraction in town, and museum staff are tasked with devising a method of keeping the population on each floor at the maximum capacity. And it is not possible to count the number of people on any floor exactly, there are too many! The museum staff implement into the teleportation system a feature where a teleportation request can be accepted or rejected. Here's where the example gets slightly artificial. The visitors to the museum are principled people, operating on their own internal transition kernels `Q : Kernel Floor Floor` (which they graciously shared with the museum staff). After every unit of time, they take their current floor x, and sample a new floor $x' \sim Q\,x$. Let us focus on a single visitor Alice (transition kernel Q_A), and the museum's protocol for automatically handling her teleportation requests. The algorithm itself is simple: the museum wishes that the ratio of people across the floors which can be understood as a distribution of people π be maintained. They devise an acceptance ratio A which depends on Q_A, π, x and x'. If the acceptance ratio is high the protocol is more likely to accept the proposed move and if not Alice must stay at the current floor x. The museum staff, with a lot of clever mathematics, prove that they can pick a particular acceptance ratio A, such that after infinitely long, the distribution of Alice's positions is exactly π. They can repeat their protocol for every visitor to reach their requirements. The protocol was scrapped since no one could wait infinitely long for a guarantee.

The analogy provides some core intuition on MCMC: obtain the probability distribution, by observing a single person's movements, which at any point in time approximates the true distribution, becoming exact at infinity. And it also provides the intuition behind Metropolis-Hastings, which

breaks building a transition kernel into sub-problems: building a *proposal transition kernel* and an *acceptance ratio*, which put together creates the actual transition kernel. It turns out that the *proposal transition kernel* can be arbitrary. Let's fill in the details. Under the Metropolis-Hastings algorithm, the acceptance ratio is given as:

$$A(Q, \widetilde{\pi}, x, x') = min\left(1, \frac{\text{density}_{\widetilde{\pi}}(x')}{\text{density}_{\widetilde{\pi}}(x)} \frac{\text{density}_{(Q\,x)}(x')}{\text{density}_{(Q\,x')}(x)}\right)$$

We have introduced some new notation, particularly by density_{μ} we refer to the density associated with a measure. A density may not always be admitted by a measure though, so Metropolis-Hastings is only well-defined when for all x, Qx admits a density and $\widetilde{\pi}$ admits a density (see Sec 6.2). It is normal to omit this distinction between the density and measure and simply refer to $\text{density}_{\widetilde{\pi}}(x')$ as $\widetilde{\pi}(x')$, but we prefer precise notation here.

Secondly it is interesting to note that $\frac{\text{density}_{\pi}(x')}{\text{density}_{\pi}(x)} = \frac{\text{density}_{\widetilde{\pi}}(x')}{\text{density}_{\widetilde{\pi}}(x)}$ since the normalising constants cancel. In the correctness proof of Metropolis-Hastings we would use the right-hand side (non-computable) but for computation we can use the left-hand side which is easy to compute. That is the core innovation of Metropolis-Hastings. We now provide Metropolis-Hastings as a Bppl program:

$$\begin{aligned} &T : S \xrightarrow{\text{m}} \mathcal{G}S \\ &Tx = x' \leftarrow Qx \\ &\quad u \leftarrow \texttt{unif01} \\ &\quad if u \leq A(Q, \widetilde{\pi}, x, x') then (ret x') else (ret x) \end{aligned}$$

Using the acceptance rejection procedure, we have transformed the *proposal transition kernel* Q, into a different transition kernel T. Metropolis-Hastings is a valid MCMC method since it is proven that when T is constructed in such a way, it has the invariant distribution π [].

We now motivate our 2nd research problem. A complex target distribution, $\widetilde{\pi}$, is required to admit a density for Metropolis-Hastings (and any MCMC method in general) to be well-defined. And $\widetilde{\pi}$ is the semantics of a program constructed using Bppl. Can we guarantee that the denotation of every program in Bppl (a measure) admits a density? Not in general. Say that $[\![M]\!]\pi$ admits a density w.r.t. a base measure, λ:

$$\forall E \in B_{\mathbb{R}}.\ \lambda(E) = 0 \rightarrow \widetilde{\pi}(E) = 0 \tag{6.1}$$

To ask if sequencing in Bppl preserves the property of admitting a density, is to ask whether taking the Giry Monadic bind with an arbitrary kernel κ preserves admitting a density. Remembering $(\pi \gg\!= \kappa)(E) = \int (\lambda\, \mu \mapsto \mu(E))\ d(\kappa_{\#}\pi)$ (Sec 2.2), that amounts to proving:

$$\forall E \in B_{\mathbb{R}}.\ \lambda(E) = 0 \rightarrow (\pi \gg\!= \kappa)(E) = 0 \tag{6.2}$$

We would like to prove (6.2) given the assumption (6.1), but it does not hold in general. To clarify, proving this would be a single inductive case in proving that all denotations of Bppl programs admit a density. **Research Problem 2**: Can we track automatically in the type system whether a program admits a density? Can we design an density primitive which can be applied when the density is known to exist? Is verification of probabilistic inference in separation logic a strong motivator for developing higher order PSLs, noting that $\text{density} : \mathcal{G}\,A \rightarrow \mathbb{R}$ can be phrased as a higher order function? Or is a workaround more elegant/satisfactory?

We conclude this chapter, with these two identified research problems for unlocking verification of probabilistic inference in separation logic, as future work.

Chapter 7

Discussion

7.1 Limitations

In the previous section 6, we outlined a vision for where continuous PSLs could go. Here we list some more

- We have not yet mechanised conditioning, Lilac's most interesting feature. We attempted to structure the codebase with extensibility[1] especially to Lilac's conditioning modality which is the nearest goal. BaSL [Ho et al., 2025] would be a natural next goal.
- Due to our use of de Bruijn indices, larger programs written in our mechanisation of Appl, can quickly become unreadable. This is the representation that the user of the Iris interface would see. It is not clear how to solve this issue without some involved metaprogramming to translate to a more human-readable variable notation before display in the Iris interface or with a different approach to variables in Appl.
- The language Appl, makes heavy use of noncomputable constructs in Mathlib, so we do not automatically obtain a foundationally verified executable, unfortunately. This is indeed expected due to formal theory working with many idealised mathematical constructs, most obviously, Real numbers and not floating point numbers. So it is in fact not straightforward to obtain a formally verified artefact when we write an Appl program in Lean. This would recover accounting for the floating-point error, an operational semantics and form of correspondence theorem.
- For long-term relevance of such mechanisation projects, it is important to bring the codebase to the highest standards so that it may be upstreamed to relevant more visible repositories, for others to build upon. There are two relevant upstream repositories: Mathlib and Iris-lean. The first step is to generalise several (ideally all) of the lemmas in Bppl.Lilac.ProofRules.MeasureProduct, so that they no longer mention `FiniteFootprint` (see Sec 4.3), which is specific to Lilac/BaSL. The reason for doing so is to get these proofs merged into Mathlib. The measure theory proofs don't really belong in this repo and likely would not be reviewed by the Iris-lean maintainers. Such a generalisation may be harder than our current proof, but at the very least requires a rewrite of the relevant proofs. The second step is to condense and clarify the remaining Lilac repository, and attempt to have it merged into Iris-lean[2].
- Probabilistic separation logic is a tiny but rapidly growing field. It is generally quite hard to give a convincing pitch of it to someone who has never heard of it. One of the factors is the steep learning curve of having to learn a considerable amount of measure theory[3]. Getting an intuitive understanding of a proof system is easiest when allowed to play with it. This is exactly what our mechanisation now allows, albeit to a limited extent. It would be great to have a repository of many more examples. We expect this may involve the invention and

[1] For example with the `closed_subtype_krm` in Bppl.Lilac.Krm, which we ran out of space to discuss in the report

[2] Guidelines for proof quality are that each line of proof should represent some idea like a line of a pen-paper proof

[3] Some of which (Giry monads, Kernels) is not covered by a standard first course in measure theory

soundness proofs of several new proof rules, given our experience with expectation (Sec 5.2). Ultimately it would be nice to develop a tutorial in the style of the Iris tutorial.

7.2 Conclusion

By mechanising Core Lilac in Lean we have provided the first ever mechanisation of a probabilistic separation logic which supports continuous distributions. Much of the measure theoretic prerequisites were already present in Mathlib, which made this project possible. As a result we have demonstrated that Mathlib is suitable not only for pushing the boundaries of (formalised) mathematics, but will eventually also be of use in producing software with the highest levels of assurance. If and when PSLs get used for large scale probabilistic program verification, we may require many more mathematical results which as suggested in Chapter 6 is likely to already be supported or show strong foundations in Mathlib.

By embedding Lilac into Iris (specifically its frontend MoSeL Krebbers et al. [2018] which was a modularisation of the original Iris proof mode), we empirically show the impressive generality of the Iris framework, as we had to make no modifications whatsoever to Iris, which was originally designed with the verification of concurrent, heap-manipulating programs in mind. Furthermore, this project is one of the first significant instantiations of what was at the beginning of this project a nascent port of Iris Lean.

Bibliography

Peter O'Hearn, John Reynolds, and Hongseok Yang. Local reasoning about programs that alter data structures. In Laurent Fribourg, editor, *Computer Science Logic*, pages 1–19, Berlin, Heidelberg, 2001. Springer Berlin Heidelberg. ISBN 978-3-540-44802-0.

Bart Jacobs, Jan Smans, Pieter Philippaerts, Frédéric Vogels, Willem Penninckx, and Frank Piessens. Verifast: A powerful, sound, predictable, fast verifier for c and java. In Mihaela Bobaru, Klaus Havelund, Gerard J. Holzmann, and Rajeev Joshi, editors, *NASA Formal Methods*, pages 41–55, Berlin, Heidelberg, 2011. Springer Berlin Heidelberg. ISBN 978-3-642-20398-5.

Cristiano Calcagno, Dino Distefano, Peter W. O'Hearn, and Hongseok Yang. Compositional shape analysis by means of bi-abduction. *J. ACM*, 58(6), December 2011. ISSN 0004-5411. doi: 10.1145/2049697.2049700. URL https://doi.org/10.1145/2049697.2049700.

Quang Loc Le, Azalea Raad, Jules Villard, Josh Berdine, Derek Dreyer, and Peter W. O'Hearn. Finding real bugs in big programs with incorrectness logic. *Proc. ACM Program. Lang.*, 6 (OOPSLA1), April 2022. doi: 10.1145/3527325. URL https://doi.org/10.1145/3527325.

Stephen Brookes and Peter W. O'Hearn. Concurrent separation logic. *ACM SIGLOG News*, 3(3): 47–65, August 2016. doi: 10.1145/2984450.2984457. URL https://doi.org/10.1145/2984450.2984457.

Ralf Jung, Robbert Krebbers, Jacques-henri Jourdan, Aleš Bizjak, Lars Birkedal, and Derek Dreyer. Iris from the ground up: A modular foundation for higher-order concurrent separation logic. *Journal of Functional Programming*, 28:e20, 2018. doi: 10.1017/S0956796818000151.

Ralf Jung, Jacques-Henri Jourdan, Robbert Krebbers, and Derek Dreyer. Rustbelt: securing the foundations of the rust programming language. *Proc. ACM Program. Lang.*, 2(POPL), December 2017. doi: 10.1145/3158154. URL https://doi.org/10.1145/3158154.

Robbert Krebbers, Amin Timany, and Lars Birkedal. Interactive proofs in higher-order concurrent separation logic. *SIGPLAN Not.*, 52(1):205–217, January 2017. ISSN 0362-1340. doi: 10.1145/3093333.3009855. URL https://doi.org/10.1145/3093333.3009855.

Robbert Krebbers, Jacques-Henri Jourdan, Ralf Jung, Joseph Tassarotti, Jan-Oliver Kaiser, Amin Timany, Arthur Charguéraud, and Derek Dreyer. Mosel: a general, extensible modal framework for interactive proofs in separation logic. *Proc. ACM Program. Lang.*, 2(ICFP), July 2018. doi: 10.1145/3236772. URL https://doi.org/10.1145/3236772.

Gilles Barthe, Justin Hsu, and Kevin Liao. A probabilistic separation logic. *Proceedings of the ACM on Programming Languages*, 4(POPL):1–30, December 2019. ISSN 2475-1421. doi: 10.1145/3371123. URL http://dx.doi.org/10.1145/3371123.

John M. Li, Amal Ahmed, and Steven Holtzen. Lilac: A modal separation logic for conditional probability. *Proc. ACM Program. Lang.*, 7(PLDI), June 2023. doi: 10.1145/3591226. URL https://doi.org/10.1145/3591226.

Jialu Bao, Emanuele D'Osualdo, and Azadeh Farzan. Bluebell: An alliance of relational lifting and independence for probabilistic reasoning. *Proc. ACM Program. Lang.*, 9(POPL), January 2025. doi: 10.1145/3704894. URL https://doi.org/10.1145/3704894.

Janine Lohse, Tim Rohde, Jimmy Xin, Niklas Mück, Iona Kuhn, Derek Dreyer, Deepak Garg, and Emanuele D'Osualdo. First steps towards probabilistic iris: Harmonizing independence, conditioning, and dynamic heap allocation, 2026. URL https://arxiv.org/abs/2605.13765.

Bob Carpenter, Andrew Gelman, Matthew D. Hoffman, Daniel Lee, Ben Goodrich, Michael Betancourt, Marcus Brubaker, Jiqiang Guo, Peter Li, and Allen Riddell. Stan: A probabilistic programming language. *Journal of Statistical Software*, 76(1):1–32, 2017. doi: 10.18637/jss.v076.i01. URL https://www.jstatsoft.org/index.php/jss/article/view/v076i01.

Praveen Narayanan, Jacques Carette, Wren Romano, Chung-chieh Shan, and Robert Zinkov. Probabilistic inference by program transformation in hakaru (system description). In *International Symposium on Functional and Logic Programming - 13th International Symposium, FLOPS 2016, Kochi, Japan, March 4-6, 2016, Proceedings*, pages 62–79. Springer, 2016. doi: 10.1007/978-3-319-29604-3_5. URL http://dx.doi.org/10.1007/978-3-319-29604-3_5.

Frank Wood, Jan Willem van de Meent, and Vikash Mansinghka. A new approach to probabilistic programming inference. In *Proceedings of the 17th International conference on Artificial Intelligence and Statistics*, pages 1024–1032, 2014.

Jan-Willem van de Meent, Brooks Paige, Hongseok Yang, and Frank Wood. An introduction to probabilistic programming, 2021. URL https://arxiv.org/abs/1809.10756.

Robert J. Aumann. Borel structures for function spaces. *Illinois Journal of Mathematics*, 5(4), December 1961. ISSN 0019-2082. doi: 10.1215/ijm/1255631584. URL http://dx.doi.org/10.1215/ijm/1255631584.

Sam Staton. Commutative semantics for probabilistic programming. In *Programming Languages and Systems: 26th European Symposium on Programming, ESOP 2017, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2017, Uppsala, Sweden, April 22–29, 2017, Proceedings*, page 855–879, Berlin, Heidelberg, 2017. Springer-Verlag. ISBN 978-3-662-54433-4. doi: 10.1007/978-3-662-54434-1_32. URL https://doi.org/10.1007/978-3-662-54434-1_32.

A. M. Stuart. Inverse problems: A bayesian perspective. *Acta Numerica*, 19:451–559, 2010. doi: 10.1017/S0962492910000061.

Kevin P. Murphy. *Probabilistic machine learning : advanced topics*. Adaptive computation and machine learning. The MIT Press, Cambridge, Massachusetts, 2023. ISBN 9780262048439.

Zoubin Ghahramani. Probabilistic machine learning and artificial intelligence. *Nature*, 521(7553): 452–459, May 2015. ISSN 1476-4687. doi: 10.1038/nature14541. URL http://dx.doi.org/10.1038/nature14541.

Alessandro Chiesa and Eylon Yogev. *Building Cryptographic Proofs from Hash Functions*. 2024. URL https://github.com/hash-based-snargs-book.

Devon Tuma, Quang Dao, James Waters, Alexander Hicks, and Nicholas Hopper. VCVio: Verified cryptography in lean via oracle effects and handlers. Cryptology ePrint Archive, Paper 2026/899, 2026. URL https://eprint.iacr.org/2026/899.

Verified-zkEVM. iris-lean. https://github.com/Verified-zkEVM/iris-lean, 2025. Fork of the Iris Lean repository, instantiating a probabilistic separation logic, Bluebell.

leanprover-community. iris-lean: Lean 4 port of iris, a higher-order concurrent separation logic framework. https://github.com/leanprover-community/iris-lean/, 2026. Accessed: 18th Jan 2026.

Shing Hin Ho, Nicolas Wu, and Azalea Raad. Bayesian separation logic, 2025. URL https://arxiv.org/abs/2507.15530.

Joseph Tassarotti and Jean-Baptiste Tristan. Verified density compilation for a probabilistic programming language. *Proc. ACM Program. Lang.*, 7(PLDI), June 2023. doi: 10.1145/3591245. URL https://doi.org/10.1145/3591245.

Adam Ścibior, Ohad Kammar, Matthijs Vákár, Sam Staton, Hongseok Yang, Yufei Cai, Klaus Ostermann, Sean K. Moss, Chris Heunen, and Zoubin Ghahramani. Denotational validation of higher-order bayesian inference. *Proceedings of the ACM on Programming Languages*, 2(POPL): 1–29, December 2017. ISSN 2475-1421. doi: 10.1145/3158148. URL http://dx.doi.org/10.1145/3158148.

Sheldon Axler. *Measures*, pages 13–72. Springer International Publishing, Cham, 2020. ISBN 978-3-030-33143-6. doi: 10.1007/978-3-030-33143-6_2. URL https://doi.org/10.1007/978-3-030-33143-6_2.

Michèle Giry. A categorical approach to probability theory. In B. Banaschewski, editor, *Categorical Aspects of Topology and Analysis*, pages 68–85, Berlin, Heidelberg, 1982. Springer Berlin Heidelberg. ISBN 978-3-540-39041-1.

Johannes Borgström, Ugo Dal Lago, Andrew D. Gordon, and Marcin Szymczak. A lambda-calculus foundation for universal probabilistic programming, 2017. URL https://arxiv.org/abs/1512.08990.

Cristiano Calcagno, Peter W. O'Hearn, and Hongseok Yang. Local action and abstract separation logic. In *22nd Annual IEEE Symposium on Logic in Computer Science (LICS 2007)*, pages 366–378, 2007. doi: 10.1109/LICS.2007.30.

David J. Pym, Peter W. O'Hearn, and Hongseok Yang. Possible worlds and resources: the semantics of bi. *Theoretical Computer Science*, 315(1):257–305, 2004. ISSN 0304-3975. doi: https://doi.org/10.1016/j.tcs.2003.11.020. URL https://www.sciencedirect.com/science/article/pii/S0304397503006248. Mathematical Foundations of Programming Semantics.

D. Galmiche, D. Méry, and D. Pym. The semantics of bi and resource tableaux. *Mathematical Structures in Computer Science*, 15(6):1033–1088, 2005. doi: 10.1017/S0960129505004858.

Peter W. O'Hearn and David J. Pym. The logic of bunched implications. *Bulletin of Symbolic Logic*, 5(2):215–244, 1999. doi: 10.2307/421090.

Christine Paulin-Mohring. Introduction to the calculus of inductive constructions. 11 2014.

Thierry Coquand. An analysis of girard's paradox. In *Logic in Computer Science*, 1986. URL https://api.semanticscholar.org/CorpusID:5533529.

Mario Carneiro. The type theory of lean. Master's thesis, Carnegie Mellon University, 2019. URL https://github.com/digama0/lean-type-theory/releases/download/v1.0/main.pdf.

leanprover community. Metaprogramming in lean 4. https://leanprover-community.github.io/lean4-metaprogramming-book/, 2024. Online book.

Adam Chlipala. *Certified Programming with Dependent Types: A Pragmatic Introduction to the Coq Proof Assistant*. The MIT Press, 2013. ISBN 0262026651.

Rémy Degenne. Markov kernels in mathlib's probability library, 2025. URL https://arxiv.org/abs/2510.04070.

Bas Spitters and Eelis van der Weegen. Type classes for mathematics in type theory, 2011. URL https://arxiv.org/abs/1102.1323.

Sean Meyn, Richard L. Tweedie, and Peter W. Glynn. *Markov Chains and Stochastic Stability*. Cambridge Mathematical Library. Cambridge University Press, 2 edition, 2009.

Philipp G. Haselwarter, Kwing Hei Li, Alejandro Aguirre, Simon Oddershede Gregersen, Joseph Tassarotti, and Lars Birkedal. Approximate relational reasoning for higher-order probabilistic programs. *Proc. ACM Program. Lang.*, 9(POPL), January 2025. doi: 10.1145/3704877. URL https://doi.org/10.1145/3704877.

Eliot Huang. Markov chain monte carlo: The metropolis-hastings algorithm. https://math.uchicago.edu/~may/REU2025/REUPapers/Huang,Eliot.pdf, 2025.

Appendix A

Declarations

A.1 Use of Generative AI

Generative AI was not used in the writing of this report other than as a glorified search engine: finding appropriate citations and guidance with formatting in latex. It was however used to speed up theorem proving in Lean and a discussion of my experience is provided in Sec 4.4. For the latter, the only use of Generative AI was through Aristotle, a tool publicly available online, free of cost, which was developed specifically for writing Lean code. Usage is made transparent by marking every commit in which it was used as "co-authored by Aristotle" which is marked as so on GitHub.

A.2 Ethical Considerations

This project pushes the boundary of formal correctness guarantees of a certain class of programs called probabilistic programs. The applications are strictly in increasing the safety of software systems, so there are no notable ethical considerations.

A.3 Sustainability

The most computationally resource intensive tool used in this project is expected to be Aristotle, a Generative AI based tool for writing Lean code. The energy usage of this tool is not publicly made available, however it is completely free to the public, which suggests that its energy usage cannot be too high. In line with the goals of this project for extending the Lean development into more complex logics, Aristotle was only used for "trivial constructions" or "obvious sub-lemmas", since it produces convoluted proofs which are not in line with our requirements for more significant tasks. A detailed discussion can be found in Sec 4.4. But the takeaway here is that the principle followed, should curtail compute-intensive situations where the system is unable to solve the tasks and goes in circles till timeout. Other than that, and the compilation of Lean and Latex files on a local PC, there are no other sustainability considerations.

A.4 Availability of Materials

This report is submitted with a GitHub repository containing the Lean artefact, which is made publicly available: `https://github.com/edwin1729/bppl-logic/tree/submission`.